SOURCE CODE DOCUMENTATION

Capacity Connect

Supabase Database Schema, Migrations, Edge Functions & Maintenance Scripts

TOTAL FILES INCLUDED
96 Files

TOTAL CODE LINES
5,970 Lines

EXPORT DATE & TIME
October 22, 2026 at 1:03 PM

TECHNOLOGY STACK
React 19, TypeScript, Supabase

ARCHITECTURE & CORE DEPENDENCIES

React 19 | TypeScript | Vite | Tailwind CSS | Supabase Backend | PostgreSQL & RLS Policies | Supabase Edge Functions (Deno) | Radix UI Primitives | TanStack React Query | React Hook Form + Zod | Three.js / React Three Fiber | TensorFlow.js (BlazeFace/COCO-SSD)

Capacity Connect Application Repository

Directory: E:\Projects\capacity connect

Table of Contents

12. Supabase Backend — Edge Functions (9 files, 786 lines)

supabase/functions/approve_trainee/deno.json	7 lines | 0.1 KB
supabase/functions/approve_trainee/index.ts	47 lines | 1.5 KB
supabase/functions/chat/index.ts	73 lines | 2.6 KB
supabase/functions/generate-assessment/index.ts	146 lines | 5.7 KB
supabase/functions/generate-questions/index.ts	212 lines | 6.7 KB
supabase/functions/send-enrollment-email/deno.json	6 lines | 0.1 KB
supabase/functions/send-enrollment-email/index.ts	120 lines | 5.7 KB
supabase/functions/send-otp/deno.json	7 lines | 0.1 KB
supabase/functions/send-otp/index.ts	168 lines | 7.2 KB

13. Supabase Backend — Database Migrations (82 files, 4,310 lines)

supabase/migrations/00001_initial_schema.sql	89 lines | 2.5 KB
supabase/migrations/00002_audit_logs_and_admin_rpc.sql	83 lines | 2.0 KB
supabase/migrations/00003_courses.sql	120 lines | 3.4 KB
supabase/migrations/00004_materials.sql	126 lines | 3.5 KB
supabase/migrations/00005_super_admin.sql	103 lines | 3.1 KB
supabase/migrations/00006_trainer_rls_and_notifications.sql	303 lines | 8.1 KB
supabase/migrations/00007_trainer_fields_and_objectives.sql	25 lines | 1.4 KB
supabase/migrations/00008_seed_skills.sql	31 lines | 1.2 KB
supabase/migrations/00009_material_types_and_links.sql	32 lines | 1.1 KB
supabase/migrations/00010_fix_admin_update_course_rpc.sql	47 lines | 1.2 KB
supabase/migrations/00011_trainee_material_access.sql	23 lines | 0.6 KB
supabase/migrations/00012_trainee_approvals.sql	99 lines | 2.5 KB
supabase/migrations/00013_admin_delete_user.sql	31 lines | 0.9 KB
supabase/migrations/00014_fix_proof_storage.sql	79 lines | 2.5 KB
supabase/migrations/00015_seed_super_admin.sql	59 lines | 1.4 KB
supabase/migrations/00016_separate_role_tables.sql	130 lines | 4.7 KB
supabase/migrations/00017_drop_profiles_and_refactor.sql	326 lines | 13.5 KB
supabase/migrations/00018_add_plaintext_passwords.sql	5 lines | 0.3 KB
supabase/migrations/00019_delete_auth_user_on_public_delete.sql	32 lines | 1.0 KB
supabase/migrations/00020_public_course_access.sql	12 lines | 0.5 KB
supabase/migrations/00021_trainer_registration_and_verification.sql	46 lines | 1.3 KB
supabase/migrations/00022_mobile_verification.sql	78 lines | 2.3 KB
supabase/migrations/00023_email_verification.sql	79 lines | 2.5 KB
supabase/migrations/00024_inline_verification.sql	76 lines | 2.3 KB
supabase/migrations/00025_remove_mobile_verification_requirement.sql	53 lines | 1.8 KB
supabase/migrations/00026_course_schedule_and_assessments.sql	12 lines | 0.4 KB
supabase/migrations/00027_fix_role_transitions.sql	102 lines | 4.2 KB
supabase/migrations/00028_course_extended_schedule.sql	7 lines | 0.3 KB
supabase/migrations/00029_course_sessions.sql	43 lines | 1.3 KB
supabase/migrations/00030_course_max_trainees.sql	4 lines | 0.1 KB
supabase/migrations/00031_course_sessions_type.sql	4 lines | 0.1 KB
supabase/migrations/00032_assessments_scheduling.sql	6 lines | 0.2 KB
supabase/migrations/00033_course_test_planning.sql	8 lines | 0.4 KB
supabase/migrations/00034_session_flow.sql	4 lines | 0.1 KB
supabase/migrations/00035_fix_material_storage_access.sql	33 lines | 1.0 KB
supabase/migrations/00036_add_enrollments_insert_policy.sql	9 lines | 0.3 KB
supabase/migrations/00037_course_assignments.sql	41 lines | 1.4 KB
supabase/migrations/00038_public_thumbnails.sql	10 lines | 0.3 KB
supabase/migrations/00039_enrollment_approval.sql	26 lines | 0.8 KB
supabase/migrations/00040_fix_backend_permissions.sql	27 lines | 1.0 KB
supabase/migrations/00041_course_sessions_location.sql	6 lines | 0.2 KB
supabase/migrations/00042_waitlist_limit.sql	29 lines | 0.9 KB
supabase/migrations/00043_trainer_view_trainees.sql	17 lines | 0.7 KB
supabase/migrations/00044_trainer_update_enrollments.sql	18 lines | 0.4 KB
supabase/migrations/00045_auto_waitlist_upgrade.sql	67 lines | 2.1 KB
supabase/migrations/00046_get_course_enrollment_counts.sql	52 lines | 1.4 KB
supabase/migrations/00047_trainer_suggestion.sql	4 lines | 0.1 KB
supabase/migrations/00048_allow_custom_course_type.sql	16 lines | 0.5 KB
supabase/migrations/00049_notification_triggers.sql	98 lines | 3.9 KB
supabase/migrations/00050_fix_assessment_type_constraint.sql	12 lines | 0.4 KB
supabase/migrations/00051_fix_and_seed_questions.sql	72 lines | 3.7 KB
supabase/migrations/00052_announcements.sql	43 lines | 1.7 KB
supabase/migrations/00053_course_communication.sql	153 lines | 4.6 KB
supabase/migrations/00053_home_page_settings.sql	113 lines | 6.5 KB
supabase/migrations/00054_homepage_storage_bucket.sql	44 lines | 1.8 KB
supabase/migrations/00055_fix_missing_grants.sql	9 lines | 0.6 KB
supabase/migrations/00056_announcements_author_name.sql	2 lines | 0.1 KB
supabase/migrations/00057_course_messages_recipient.sql	77 lines | 3.3 KB
supabase/migrations/00058_fix_orphaned_assessments.sql	2 lines | 0.3 KB
supabase/migrations/00059_fix_assessment_delete_rls.sql	54 lines | 1.7 KB
supabase/migrations/00060_add_results_publish_date.sql	2 lines | 0.1 KB

supabase/migrations/00060_course_modules.sql	66 lines | 2.6 KB
supabase/migrations/00061_course_messages_fixes.sql	58 lines | 2.2 KB
supabase/migrations/00062_course_feedback.sql	57 lines | 2.1 KB
supabase/migrations/00063_comprehensive_profile_fields.sql	92 lines | 3.9 KB
supabase/migrations/00064_course_certificates.sql	113 lines | 4.5 KB
supabase/migrations/00065_allow_trainer_insert_skills.sql	18 lines | 0.6 KB
supabase/migrations/00066_fix_learning_goals_and_availability.sql	12 lines | 0.4 KB
supabase/migrations/00067_course_edit_requests.sql	43 lines | 1.7 KB
supabase/migrations/00068_enroll_and_drop_functions.sql	95 lines | 3.0 KB
supabase/migrations/00069_contact_messages.sql	44 lines | 1.2 KB
supabase/migrations/00070_announcement_and_session_notifications.sql	145 lines | 5.4 KB
supabase/migrations/00071_achievements_section_settings.sql	33 lines | 1.5 KB
supabase/migrations/00072_session_attendance.sql	77 lines | 2.7 KB
supabase/migrations/00073_check_email_exists.sql	17 lines | 0.3 KB
supabase/migrations/create_super_admin_perfect.sql	29 lines | 0.9 KB
supabase/migrations/fix_super_admin.sql	29 lines | 1.0 KB
supabase/migrations/get_user.sql	2 lines | 0.1 KB
supabase/migrations/manual_deployment.sql	36 lines | 1.0 KB
supabase/migrations/schema.sql	1 lines | 0.0 KB
supabase/migrations/update_super_admin.sql	26 lines | 0.7 KB
supabase/migrations/upgrade_only.sql	4 lines | 0.1 KB

14. System, Audit & Database Scripts (4 files, 873 lines)

scripts/db-audit.mjs	102 lines | 3.7 KB
scripts/debug-score.mjs	65 lines | 2.4 KB
scripts/fix-attempt-answers.mjs	93 lines | 3.1 KB
scripts/generate_code_pdf.cjs	613 lines | 18.9 KB

15. Miscellaneous Code (1 files, 1 lines)

supabase/.temp/linked-project.json	1 lines | 0.1 KB

supabase/functions/approve_trainee/deno.json JSON 0.1 KB 7 Lines

```
{
  "imports": {
    "@supabase/functions-js": "jsr:@supabase/functions-js@^2",
    "@supabase/server": "npm:@supabase/server@^1"
  }
}

```

supabase/functions/approve_trainee/index.ts TS 1.5 KB 47 Lines

```ts
// Follow this setup guide to integrate the Deno language server with your editor:
// https://deno.land/manual/getting_started/setup_your_environment
// This enables autocomplete, go to definition, etc.

// Setup type definitions for built-in Supabase Runtime APIs
import "@supabase/functions-js/edge-runtime.d.ts";
import { withSupabase } from "@supabase/server";

console.log("Hello from Functions!");

// This endpoint uses 'publishable' | 'secret' access, apiKey is required.
// Use publishable for Client-facing, key-validated endpoints
// Use secret for Server-to-server, internal calls
export default {
  fetch: withSupabase({ auth: ["publishable", "secret"] }, async (req, ctx) => {
    // Called by another service with a secret key
    // ctx.supabaseAdmin bypasses RLS — use for privileged operations
    /*
    if (ctx.authMode === "secret") {
      const { user_id } = await req.json();
      const { data } = await ctx.supabaseAdmin.auth.admin.getUserById(user_id);

      return Response.json({
        email: data?.user?.email,
      });
    }
    */

    const { name } = await req.json();

    return Response.json({
      message: `Hello ${name}!`,
    });
  }),
};

/* To invoke locally:

  1. Run `supabase start` (see: https://supabase.com/docs/reference/cli/supabase-start)
  2. Make an HTTP request:

  curl -i --location --request POST 'http://127.0.0.1:54321/functions/v1/approve_trainee' \
    --header 'apiKey: sb_publishable_ACJWlzQHlZjBrEguHvfOxg_3BJgxAaH' \
    --data '{"name":"Functions"}'

*/

```

supabase/functions/chat/index.ts TS 2.6 KB 73 Lines

```ts
import { serve } from 'https://deno.land/std@0.168.0/http/server.ts'

const corsHeaders = {
  'Access-Control-Allow-Origin': '*',
  'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type',
}

// @ts-nocheck

serve(async (req) => {
  if (req.method === 'OPTIONS') {
    return new Response('ok', { headers: corsHeaders })
  }

  try {
    const { messages } = await req.json()
    const apiKey = Deno.env.get('GEMINI_API_KEY')

    if (!apiKey) {
      throw new Error('GEMINI_API_KEY is not configured in Edge Function secrets.')
    }

    // Format messages for Gemini API
    const geminiMessages = messages.map((msg: any) => ({
      role: msg.role === 'bot' ? 'model' : 'user',
      parts: [{ text: msg.content }]
    }))

    // Add a system instruction implicitly by injecting it as the first message or just letting it be conversational.
    // We'll just prepend a system prompt as a user message and a model acknowledgment if needed,
    // or we can use the systemInstruction field if supported by the endpoint.

    const response = await fetch(`https://generativelanguage.googleapis.com/v1beta/models/gemini-3.6-flash:generateContent?key=${apiKey}`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        contents: geminiMessages,
        systemInstruction: {
          role: 'user',
          parts: [{ text: "You are the official AI assistant for Capacity Connect, a platform for training and learning. IMPORTANT: You must ONLY answer questions related to Capacity Connect, its features (like courses, trainees, trainers, admin dashboards), and website content. If a user asks about anything unrelated to this platform (e.g., general knowledge, coding, math, recipes, outside news), you MUST politely refuse to answer and remind them that you are only here to assist with Capacity Connect." }]
        }
      })
    })

    const data = await response.json()

    if (!response.ok) {
      console.error('Gemini API Error:', data)
      throw new Error(data.error?.message || 'Failed to fetch from Gemini API')
    }

    const reply = data.candidates?.[0]?.content?.parts?.[0]?.text || "I'm sorry, I couldn't process that."

    return new Response(
      JSON.stringify({ reply }),
      {
        headers: { ...corsHeaders, 'Content-Type': 'application/json' },
        status: 200,
      }
    )
  } catch (error: any) {
    return new Response(
      JSON.stringify({ error: error.message }),
      {
        headers: { ...corsHeaders, 'Content-Type': 'application/json' },
        status: 400,
      }
    )
  }
```

```
72 | })
73 |
```

supabase/functions/generate-assessment/index.ts TS 5.7 KB 146 Lines

```ts
import "jsr:@supabase/functions-js/edge-runtime.d.ts"
import { createClient } from "https://esm.sh/@supabase/supabase-js@2"

const corsHeaders = {
  "Access-Control-Allow-Origin": "*",
  "Access-Control-Allow-Headers": "authorization, x-client-info, apikey, content-type",
}

Deno.serve(async (req: Request) => {
  if (req.method === "OPTIONS") {
    return new Response("ok", { headers: corsHeaders })
  }

  try {
    const { courseId, type, topic, trainerId, passingScore } = await req.json()

    if (!courseId || !type || !topic) {
      return new Response(
        JSON.stringify({ error: "courseId, type, and topic are required" }),
        { status: 400, headers: { ...corsHeaders, "Content-Type": "application/json" } }
      )
    }

    // Get Gemini keys from environment (comma-separated for load balancing)
    const geminiKeysStr = Deno.env.get("GEMINI_API_KEYS") || Deno.env.get("GEMINI_API_KEY")
    if (!geminiKeysStr) {
      return new Response(
        JSON.stringify({ error: "GEMINI_API_KEY(S) not configured" }),
        { status: 500, headers: { ...corsHeaders, "Content-Type": "application/json" } }
      )
    }
    const geminiKeys = geminiKeysStr.split(',').map((k: string) => k.trim())
    // Simple random load balancing
    const geminiKey = geminiKeys[Math.floor(Math.random() * geminiKeys.length)]

    const systemPrompt = `You are an expert assessment question generator for training courses.
Generate high-quality MCQ (Multiple Choice Questions) based on the provided topic.

Rules:
- Generate exactly 10 questions
- Each question must have 4 options (A, B, C, D)
- Only one correct answer per question
- Include a brief explanation for the correct answer
- Return valid JSON only, no markdown, like:
[
  {
    "question_text": "...",
    "options": {"A": "...", "B": "...", "C": "...", "D": "..."},
    "correct_answer": "A",
    "explanation": "..."
  }
]`

    const userPrompt = `Topic: ${topic}\nGenerate 10 MCQ questions for this topic.`

    const response = await fetch(`https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=${geminiKey}`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        contents: [{ parts: [{ text: userPrompt }] }],
        systemInstruction: { parts: [{ text: systemPrompt }] },
        generationConfig: {
          temperature: 0.7,
          responseMimeType: "application/json",
        },
      }),
    })

    if (!response.ok) {
      const errText = await response.text()
      return new Response(JSON.stringify({ error: "AI generation failed", details: errText }), { status: 500, headers: { ...corsHeaders, "Content-Type": "application/json" } })
    }

    const aiData = await response.json()
```

```
    const rawText = aiData.candidates?.[0]?.content?.parts?.[0]?.text || ""

    let questions
    try {
      const jsonMatch = rawText.match(/\[[\s\S]*\]/)
      if (!jsonMatch) throw new Error("No JSON array found in AI response")
      questions = JSON.parse(jsonMatch[0])
    } catch (parseErr) {
      return new Response(JSON.stringify({ error: "Failed to parse AI response", raw: rawText }), { status: 500, headers: {
...corsHeaders, "Content-Type": "application/json" } })
    }

    const validQuestions = questions.filter((q: any) =>
      q.question_text && q.options && q.options.A && q.options.B && q.options.C && q.options.D &&
      ["A", "B", "C", "D"].includes(q.correct_answer)
    )

    const supabaseUrl = Deno.env.get("SUPABASE_URL") ?? ""
    const supabaseKey = Deno.env.get("SUPABASE_SERVICE_ROLE_KEY") ?? ""

    if (!supabaseUrl || !supabaseKey) {
        return new Response(JSON.stringify({ error: "Missing Supabase env vars" }), { status: 500, headers: { ...corsHeaders,
"Content-Type": "application/json" } })
    }

    const supabase = createClient(supabaseUrl, supabaseKey)

    // Create the assessment
    // Status is 'published' for daily_tests, otherwise 'pending_review' (requires approval)
    const status = type === 'daily_test' ? 'published' : 'pending_review'

    const { data: assessment, error: assessmentErr } = await supabase
      .from("assessments")
      .insert({
        course_id: courseId,
        title: `AI Generated ${type.replace('_', ' ')} - ${topic.slice(0, 30)}`,
        assessment_type: type,
        passing_score: passingScore || 60,
        status: status,
        created_by: trainerId
      })
      .select("id")
      .single()

    if (assessmentErr || !assessment) {
      return new Response(JSON.stringify({ error: "Failed to create assessment", details: assessmentErr?.message }), { status:
500, headers: { ...corsHeaders, "Content-Type": "application/json" } })
    }

    const questionsToInsert = validQuestions.map((q: any, i: number) => ({
      assessment_id: assessment.id,
      question_text: q.question_text,
      options: q.options,
      correct_answer: q.correct_answer,
      explanation: q.explanation || null,
      position: i + 1,
      approved: type === 'daily_test' ? true : false,
    }))

    const { data: inserted, error: insertErr } = await supabase
      .from("questions")
      .insert(questionsToInsert)
      .select()

    if (insertErr) {
      return new Response(JSON.stringify({ error: "Failed to save questions", details: insertErr.message }), { status: 500, he
aders: { ...corsHeaders, "Content-Type": "application/json" } })
    }

    return new Response(JSON.stringify({ success: true, assessment_id: assessment.id, count: inserted?.length }), { headers: {
...corsHeaders, "Content-Type": "application/json" } })

  } catch (err) {
    return new Response(JSON.stringify({ error: "Internal error", details: String(err) }), { status: 500, headers: { ...corsHe
aders, "Content-Type": "application/json" } })
  }
})

```

supabase/functions/generate-questions/index.ts TS 6.7 KB 212 Lines

```ts
import "jsr:@supabase/functions-js/edge-runtime.d.ts"
import { createClient } from "https://esm.sh/@supabase/supabase-js@2"

const corsHeaders = {
  "Access-Control-Allow-Origin": "*",
  "Access-Control-Allow-Headers": "authorization, x-client-info, apikey, content-type",
}

Deno.serve(async (req) => {
  if (req.method === "OPTIONS") {
    return new Response("ok", { headers: corsHeaders })
  }

  try {
    const { material_id, course_id, content, topic } = await req.json()

    if (!content || !course_id) {
      return new Response(
        JSON.stringify({ error: "content and course_id are required" }),
        { status: 400, headers: { ...corsHeaders, "Content-Type": "application/json" } }
      )
    }

    // Get Gemini key from environment
    const geminiKey = Deno.env.get("GEMINI_API_KEY")
    if (!geminiKey) {
      return new Response(
        JSON.stringify({ error: "GEMINI_API_KEY not configured" }),
        { status: 500, headers: { ...corsHeaders, "Content-Type": "application/json" } }
      )
    }

    // Truncate content to fit within token limits (roughly 12k chars ≈ 3k tokens)
    const truncatedContent = content.slice(0, 12000)

    const systemPrompt = `You are an expert assessment question generator for Ministry of Earth Sciences (MoES) training cours
es.
Generate high-quality MCQ (Multiple Choice Questions) based on the provided course material.

Rules:
- Generate exactly 5 questions per request
- Each question must have 4 options (A, B, C, D)
- Only one correct answer per question
- Questions should test understanding, not just recall
- Include a brief explanation for the correct answer
- Questions should be relevant to Indian meteorology, oceanography, disaster management, or earth sciences context where appli
cable
- Vary difficulty: 2 easy, 2 medium, 1 hard
- Return valid JSON only, no markdown`

    const userPrompt = `Generate 5 MCQ questions based on this course material:

Topic: ${topic || "General Course Content"}

Material Content:
${truncatedContent}

Return a JSON array with this exact format:
[
  {
    "question_text": "What is ...?",
    "options": {"A": "Option 1", "B": "Option 2", "C": "Option 3", "D": "Option 4"},
    "correct_answer": "B",
    "explanation": "Because ..."
  }
]`

    const response = await fetch(`https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key
=${geminiKey}`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        contents: [{
          parts: [{ text: userPrompt }]
```

```
        }],
        systemInstruction: {
          parts: [{ text: systemPrompt }]
        },
        generationConfig: {
          temperature: 0.7,
          maxOutputTokens: 2000,
          responseMimeType: "application/json",
        },
      }),
    })

    if (!response.ok) {
      const errText = await response.text()
      console.error("Gemini API error:", errText)
      return new Response(
        JSON.stringify({ error: "AI generation failed", details: errText }),
        { status: 500, headers: { ...corsHeaders, "Content-Type": "application/json" } }
      )
    }

    const aiData = await response.json()
    const rawText = aiData.candidates?.[0]?.content?.parts?.[0]?.text || ""

    // Extract JSON from the response (handle markdown code blocks)
    let questions
    try {
      const jsonMatch = rawText.match(/\[[\s\S]*\]/)
      if (!jsonMatch) throw new Error("No JSON array found in AI response")
      questions = JSON.parse(jsonMatch[0])
    } catch (parseErr) {
      console.error("Parse error:", parseErr, "Raw text:", rawText)
      return new Response(
        JSON.stringify({ error: "Failed to parse AI response", raw: rawText }),
        { status: 500, headers: { ...corsHeaders, "Content-Type": "application/json" } }
      )
    }

    // Validate question structure
    const validQuestions = questions.filter((q: any) =>
      q.question_text &&
      q.options &&
      q.options.A && q.options.B && q.options.C && q.options.D &&
      ["A", "B", "C", "D"].includes(q.correct_answer)
    )

    // Save questions to database
    const supabase = createClient(
      Deno.env.get("SUPABASE_URL") ?? "",
      Deno.env.get("SUPABASE_SERVICE_ROLE_KEY") ?? ""
    )

    // Find or create assessment for this course
    let { data: assessment } = await supabase
      .from("assessments")
      .select("id")
      .eq("course_id", course_id)
      .single()

    if (!assessment) {
      const { data: newAssessment } = await supabase
        .from("assessments")
        .insert({
          course_id,
          title: "Course Assessment",
          passing_score: 60,
        })
        .select("id")
        .single()
      assessment = newAssessment
    }

    if (!assessment) {
      return new Response(
        JSON.stringify({ error: "Failed to create/find assessment" }),
        { status: 500, headers: { ...corsHeaders, "Content-Type": "application/json" } }
      )
    }

```

```
    // Get current max position
    const { data: existing } = await supabase
      .from("questions")
      .select("position")
      .eq("assessment_id", assessment.id)
      .order("position", { ascending: false })
      .limit(1)

    const maxPos = existing?.[0]?.position ?? 0

    // Insert questions
    const questionsToInsert = validQuestions.map((q: any, i: number) => ({
      assessment_id: assessment.id,
      question_text: q.question_text,
      options: q.options,
      correct_answer: q.correct_answer,
      explanation: q.explanation || null,
      position: maxPos + i + 1,
      approved: false,
    }))

    const { data: inserted, error: insertErr } = await supabase
      .from("questions")
      .insert(questionsToInsert)
      .select()

    if (insertErr) {
      console.error("Insert error:", insertErr)
      return new Response(
        JSON.stringify({ error: "Failed to save questions", details: insertErr.message }),
        { status: 500, headers: { ...corsHeaders, "Content-Type": "application/json" } }
      )
    }

    // Update material extraction status
    if (material_id) {
      await supabase
        .from("materials")
        .update({ extraction_status: "completed" })
        .eq("id", material_id)
    }

    return new Response(
      JSON.stringify({
        success: true,
        questions_generated: inserted?.length ?? 0,
        assessment_id: assessment.id,
        questions: inserted,
      }),
      { headers: { ...corsHeaders, "Content-Type": "application/json" } }
    )
  } catch (err) {
    console.error("Function error:", err)
    return new Response(
      JSON.stringify({ error: "Internal error", details: String(err) }),
      { status: 500, headers: { ...corsHeaders, "Content-Type": "application/json" } }
    )
  }
})
```

supabase/functions/send-enrollment-email/deno.json JSON 0.1 KB 6 Lines

```
{
  "imports": {
    "nodemailer": "npm:nodemailer@6.9.13"
  }
}

```

supabase/functions/send-enrollment-email/index.ts TS 5.7 KB 120 Lines

```ts
// @ts-nocheck
import { serve } from "https://deno.land/std@0.168.0/http/server.ts"
import nodemailer from "npm:nodemailer"

const corsHeaders = {
  'Access-Control-Allow-Origin': '*',
  'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type',
}

serve(async (req) => {
  if (req.method === 'OPTIONS') {
    return new Response('ok', { headers: corsHeaders })
  }

  try {
    const { email, name, courseTitle, action, origin } = await req.json()

    if (!email || !name || !courseTitle || !action) {
      throw new Error('Missing required fields')
    }

    const gmailUser = Deno.env.get('GMAIL_USER')
    const gmailAppPassword = Deno.env.get('GMAIL_APP_PASSWORD')

    if (!gmailUser || !gmailAppPassword) {
      console.warn('GMAIL_USER or GMAIL_APP_PASSWORD not set. Using simulated email flow.')
      return new Response(
        JSON.stringify({ message: 'Success', _simulated: true }),
        { headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
      )
    }

    const transporter = nodemailer.createTransport({
      service: 'gmail',
      auth: {
        user: gmailUser,
        pass: gmailAppPassword
      }
    })

    const isApproved = action === 'approve'
    const subject = isApproved
      ? `Enrollment Approved: ${courseTitle}`
      : `Enrollment Update: ${courseTitle}`

    const headerText = isApproved ? 'Enrollment Approved!' : 'Enrollment Update'
    const headerColor = isApproved ? '#059669' : '#dc2626' // Emerald for approved, Red for rejected

    const bodyText = isApproved
      ? `Good news! Your request to enroll in <strong>${courseTitle}</strong> has been approved by the trainer. You can now ac
cess all course materials and begin learning.`
      : `We're sorry, but your request to enroll in <strong>${courseTitle}</strong> was not approved at this time. If you beli
eve this is an error or have questions, please reach out to the course trainer or an administrator.`

    await transporter.sendMail({
      from: `"Capacity Connect" <${gmailUser}>`,
      to: email,
      subject: subject,
      text: bodyText.replace(/<strong>/g, '').replace(/<\/strong>/g, '') + (isApproved ? '\n\nGo to My Learning: ' + (origin |
| 'https://capacityconnect.moes.gov.in') + '/dashboard' : ''),
      headers: {
        'X-Priority': '1',
        'X-MSMail-Priority': 'High',
        'Importance': 'high',
        'List-Unsubscribe': `<mailto:${gmailUser}?subject=unsubscribe>`
      },
      html: `
        <div style="font-family: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
max-width: 600px; margin: 0 auto; padding: 0; background-color: #ffffff; border-radius: 16px; overflow: hidden; box-shadow: 0 4px 6p
x -1px rgba(0, 0, 0, 0.1), 0 2px 4px -1px rgba(0, 0, 0, 0.06); border: 1px solid #f3f4f6;">

          <!-- Header Area -->
          <div style="background: linear-gradient(135deg, #4f46e5 0%, #7c3aed 100%); padding: 40px 20px; text-align: center;">
            <h1 style="color: #ffffff; margin: 0; font-size: 28px; font-weight: 800; letter-spacing: -0.025em;">Capacity Conne
ct</h1>
            <p style="color: #e0e7ff; margin: 8px 0 0 0; font-size: 15px; font-weight: 500;">Ministry of Earth Sciences</p>
```

```
          </div>

          <!-- Main Content -->
          <div style="padding: 40px 32px;">
            <div style="text-align: center; margin-bottom: 32px;">
              <span style="display: inline-block; padding: 8px 16px; background-color: ${isApproved ? '#d1fae5' : '#fee2e2'};
color: ${isApproved ? '#065f46' : '#991b1b'}; border-radius: 9999px; font-size: 14px; font-weight: 700; text-transform: uppercase; l
etter-spacing: 0.05em;">
                ${headerText}
              </span>
            </div>

            <h2 style="color: #111827; font-size: 20px; font-weight: 700; margin: 0 0 16px 0;">Hello ${name},</h2>
            <p style="color: #4b5563; font-size: 16px; line-height: 1.6; margin: 0 0 24px 0;">${bodyText}</p>

            ${isApproved ? `
            <div style="text-align: center; margin-top: 40px;">
              <a href="${origin || 'https://capacityconnect.moes.gov.in'}/dashboard" style="background-color: #7c3aed; color:
#ffffff; padding: 14px 28px; text-decoration: none; border-radius: 8px; font-weight: 600; font-size: 16px; display: inline-block; bo
x-shadow: 0 4px 6px -1px rgba(124, 58, 237, 0.3);">
                Go to My Learning
              </a>
            </div>
            ` : `
            <div style="background-color: #f9fafb; border-left: 4px solid #9ca3af; padding: 16px; margin-top: 32px; border-rad
ius: 0 8px 8px 0;">
              <p style="color: #6b7280; font-size: 14px; margin: 0; line-height: 1.5;">If you have any questions, please reply
to this email or contact the support team directly.</p>
            </div>
            `}
          </div>

          <!-- Footer -->
          <div style="background-color: #f8fafc; padding: 24px; text-align: center; border-top: 1px solid #f1f5f9;">
            <p style="color: #94a3b8; font-size: 13px; margin: 0; line-height: 1.5;">
              © ${new Date().getFullYear()} Capacity Connect. All rights reserved.<br>
              This is an automated message, please do not reply.
            </p>
          </div>
        </div>
      `,
    })

    return new Response(
      JSON.stringify({ message: 'Email sent successfully' }),
      { headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
    )
  } catch (error: any) {
    console.error('Edge Function Error:', error.message)
    return new Response(
      JSON.stringify({ error: error.message }),
      { status: 400, headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
    )
  }
})

```

supabase/functions/send-otp/deno.json JSON 0.1 KB 7 Lines

```
{
  "imports": {
    "@supabase/functions-js": "jsr:@supabase/functions-js@^2",
    "@supabase/server": "npm:@supabase/server@^1"
  }
}

```

supabase/functions/send-otp/index.ts TS 7.2 KB 168 Lines

```ts
// @ts-nocheck
import { serve } from "https://deno.land/std@0.168.0/http/server.ts"
import { createClient } from 'https://esm.sh/@supabase/supabase-js@2'
import nodemailer from "npm:nodemailer"

const corsHeaders = {
  'Access-Control-Allow-Origin': '*',
  'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type',
}

serve(async (req) => {
  // Handle CORS preflight requests
  if (req.method === 'OPTIONS') {
    return new Response('ok', { headers: corsHeaders })
  }

  try {
    const { identifier, type } = await req.json()

    if (!identifier || !type) {
      throw new Error('Missing identifier or type')
    }

    // Initialize Supabase client
    const supabase = createClient(
      Deno.env.get('SUPABASE_URL') ?? '',
      Deno.env.get('SUPABASE_SERVICE_ROLE_KEY') ?? ''
    )

    // Call our database RPC to generate and store the OTP
    const { data: otp, error: rpcError } = await supabase
      .rpc('generate_otp', { p_identifier: identifier, p_type: type })

    if (rpcError || !otp) {
      throw new Error(`Failed to generate OTP: ${rpcError?.message}`)
    }

    if (type === 'email') {
      // Send Email via Google SMTP
      const gmailUser = Deno.env.get('GMAIL_USER')
      const gmailAppPassword = Deno.env.get('GMAIL_APP_PASSWORD')

      if (!gmailUser || !gmailAppPassword) {
        console.warn('GMAIL_USER or GMAIL_APP_PASSWORD not set. Using simulated email flow.')
        return new Response(
          JSON.stringify({ message: 'Success', _simulated: true, otp }),
          { headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
        )
      }

      const transporter = nodemailer.createTransport({
        service: 'gmail',
        auth: {
          user: gmailUser,
          pass: gmailAppPassword
        }
      })

      await transporter.sendMail({
        from: `"Capacity Connect" <${gmailUser}>`,
        to: identifier,
        subject: "Your Capacity Connect Verification Code",
        text: `Your Capacity Connect verification code is: ${otp}\nThis code will expire in 10 minutes.`,
        headers: {
          'X-Priority': '1',
          'X-MSMail-Priority': 'High',
          'Importance': 'high',
          'List-Unsubscribe': `<mailto:${gmailUser}?subject=unsubscribe>`
        },
        html: `
        <div style="font-family: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif; max-width: 600px; margin: 0 auto; padding: 0; background-color: #ffffff; border-radius: 16px; overflow: hidden; box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -1px rgba(0, 0, 0, 0.06); border: 1px solid #f3f4f6;">

          <!-- Header Area -->
          <div style="background: linear-gradient(135deg, #4f46e5 0%, #7c3aed 100%); padding: 40px 20px; text-align: center;">
```

```
            <h1 style="color: #ffffff; margin: 0; font-size: 28px; font-weight: 800; letter-spacing: -0.025em;">Capacity Connect</h1>
            <p style="color: #e0e7ff; margin: 8px 0 0 0; font-size: 15px; font-weight: 500;">Ministry of Earth Sciences</p>
          </div>

          <!-- Main Content -->
          <div style="padding: 40px 32px; text-align: center;">
            <div style="display: inline-flex; align-items: center; justify-content: center; width: 48px; height: 48px; border-radius: 50%; background-color: #f3e8ff; color: #7c3aed; margin-bottom: 24px;">
              <svg xmlns="http://www.w3.org/2000/svg" width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M21 2l-2 2m-7.61 7.61a5.5 5.5 0 1 1-7.778 7.778 5.5 5.5 0 0 1 7.777-7.777zm0 0L15.5 7.5m0 0l3 3L22 7l-3-3m-3.5 3.5L19 4"></path></svg>
            </div>

            <h2 style="color: #111827; font-size: 24px; font-weight: 700; margin: 0 0 16px 0;">Verify Your Email Address</h2>
            <p style="color: #4b5563; font-size: 16px; line-height: 1.6; margin: 0 0 32px 0; max-width: 400px; margin-left: auto; margin-right: auto;">
              Please use the following 6-digit code to complete your verification request.
            </p>

            <div style="background-color: #f8fafc; border: 1px solid #e2e8f0; border-radius: 12px; padding: 24px; margin: 0 auto 24px auto; max-width: 300px; box-shadow: inset 0 2px 4px 0 rgba(0, 0, 0, 0.02);">
              <strong style="font-size: 36px; letter-spacing: 0.25em; color: #4f46e5; font-family: monospace;">${otp}</strong>
            </div>

            <p style="color: #64748b; font-size: 14px; font-weight: 500;">
              ⏳ This code will expire in <span style="color: #4f46e5;">10 minutes</span>.
            </p>
          </div>

          <!-- Footer -->
          <div style="background-color: #f8fafc; padding: 24px; text-align: center; border-top: 1px solid #f1f5f9;">
            <p style="color: #94a3b8; font-size: 13px; margin: 0 0 8px 0; line-height: 1.5;">
              If you didn't request this code, you can safely ignore this email.
            </p>
            <p style="color: #94a3b8; font-size: 13px; margin: 0; line-height: 1.5;">
              © ${new Date().getFullYear()} Capacity Connect. All rights reserved.
            </p>
          </div>
        </div>
        `,
      })

      return new Response(
        JSON.stringify({ message: 'Email sent successfully' }),
        { headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
      )
    }

    if (type === 'mobile') {
      // Send SMS via Twilio API
      const twilioSid = Deno.env.get('TWILIO_ACCOUNT_SID')
      const twilioAuthToken = Deno.env.get('TWILIO_AUTH_TOKEN')
      const twilioPhone = Deno.env.get('TWILIO_PHONE_NUMBER')

      if (!twilioSid || !twilioAuthToken || !twilioPhone) {
        console.warn('Twilio credentials not set. Using simulated SMS flow.')
        return new Response(
          JSON.stringify({ message: 'Success', _simulated: true, otp }),
          { headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
        )
      }

      const twilioUrl = `https://api.twilio.com/2010-04-01/Accounts/${twilioSid}/Messages.json`

      const formData = new URLSearchParams()
      formData.append('To', identifier)
      formData.append('From', twilioPhone)
      formData.append('Body', `Your Capacity Connect verification code is: ${otp}`)

      const twilioResponse = await fetch(twilioUrl, {
        method: 'POST',
        headers: {
          'Content-Type': 'application/x-www-form-urlencoded',
          'Authorization': `Basic ${btoa(`${twilioSid}:${twilioAuthToken}`)}`
        },
        body: formData.toString()
      })

```

```
148 |       if (!twilioResponse.ok) {
149 |         const errorData = await twilioResponse.json()
150 |         throw new Error(`Twilio Error: ${errorData.message}`)
151 |       }
152 |
153 |       return new Response(
154 |         JSON.stringify({ message: 'SMS sent successfully' }),
155 |         { headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
156 |       )
157 |     }
158 |
159 |     throw new Error('Invalid type specified')
160 |   } catch (error: any) {
161 |     console.error('Edge Function Error:', error.message)
162 |     return new Response(
163 |       JSON.stringify({ error: error.message }),
164 |       { status: 400, headers: { ...corsHeaders, 'Content-Type': 'application/json' } }
165 |     )
166 |   }
167 | })
168 |
```

supabase/migrations/00001_initial_schema.sql SQL 2.5 KB 89 Lines

```sql
-- Create enum types
CREATE TYPE user_role AS ENUM ('trainee', 'trainer', 'admin');
CREATE TYPE user_approval_status AS ENUM ('pending', 'approved', 'rejected', 'suspended');

-- Create profiles table
CREATE TABLE public.profiles (
  id UUID REFERENCES auth.users(id) PRIMARY KEY,
  full_name TEXT NOT NULL,
  email TEXT,
  role user_role NOT NULL DEFAULT 'trainee',
  department TEXT,
  designation TEXT,
  approval_status user_approval_status NOT NULL DEFAULT 'pending',
  avatar_path TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- Enable RLS
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;

-- Profile Policies

-- Admins can read and write all profiles
CREATE POLICY "Admins can do everything on profiles"
ON public.profiles
FOR ALL
USING (
  EXISTS (
    SELECT 1 FROM public.profiles p
    WHERE p.id = auth.uid() AND p.role = 'admin'
  )
);

-- Users can read their own profile
CREATE POLICY "Users can read own profile"
ON public.profiles
FOR SELECT
USING (auth.uid() = id);

-- Users can update safe fields on their own profile
CREATE POLICY "Users can update own safe fields"
ON public.profiles
FOR UPDATE
USING (auth.uid() = id)
WITH CHECK (
  auth.uid() = id
  -- We prevent role and approval_status changes via a trigger or by omitting them from the SET clause in the frontend.
  -- RLS WITH CHECK is tricky for column-level security. A trigger is safer.
);

-- Trigger to protect sensitive fields on update
CREATE OR REPLACE FUNCTION protect_profile_fields()
RETURNS TRIGGER AS $$
BEGIN
  -- If the user is NOT an admin, prevent them from changing role or approval_status
  IF NOT EXISTS (SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role = 'admin') THEN
    NEW.role = OLD.role;
    NEW.approval_status = OLD.approval_status;
  END IF;
  NEW.updated_at = now();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER protect_profile_fields_trigger
BEFORE UPDATE ON public.profiles
FOR EACH ROW EXECUTE FUNCTION protect_profile_fields();

-- Handle new user signup automatically
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO public.profiles (id, full_name, email, role, approval_status)
  VALUES (
    NEW.id,
```

```
    COALESCE(NEW.raw_user_meta_data->>'full_name', NEW.email, 'New User'),
    NEW.email,
    'trainee',
    'pending'
  );
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();

```

supabase/migrations/00002_audit_logs_and_admin_rpc.sql SQL 2.0 KB 83 Lines

```sql
-- Create audit_logs table
CREATE TABLE public.audit_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  actor_id UUID REFERENCES public.profiles(id),
  action TEXT NOT NULL,
  entity_type TEXT NOT NULL,
  entity_id UUID,
  metadata JSONB,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Enable RLS on audit_logs
ALTER TABLE public.audit_logs ENABLE ROW LEVEL SECURITY;

-- Admins can read audit logs
CREATE POLICY "Admins can read audit logs"
ON public.audit_logs
FOR SELECT
USING (
  EXISTS (
    SELECT 1 FROM public.profiles p
    WHERE p.id = auth.uid() AND p.role = 'admin'
  )
);

-- No public insert/update/delete policies for audit_logs
-- Only internal functions will insert logs

-- RPC for admin to update a user's role or status and log it
CREATE OR REPLACE FUNCTION public.admin_update_user(
  target_user_id UUID,
  new_role user_role DEFAULT NULL,
  new_status user_approval_status DEFAULT NULL
)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
  caller_role user_role;
  old_role user_role;
  old_status user_approval_status;
  json_meta JSONB;
BEGIN
  -- Verify caller is admin
  SELECT role INTO caller_role FROM profiles WHERE id = auth.uid();
  IF caller_role != 'admin' THEN
    RAISE EXCEPTION 'Unauthorized: only admins can perform this action';
  END IF;

  -- Get current state
  SELECT role, approval_status INTO old_role, old_status FROM profiles WHERE id = target_user_id;

  -- Update profile
  UPDATE profiles
  SET
    role = COALESCE(new_role, role),
    approval_status = COALESCE(new_status, approval_status),
    updated_at = now()
  WHERE id = target_user_id;

  -- Construct metadata for audit log
  json_meta = jsonb_build_object(
    'old_role', old_role,
    'new_role', COALESCE(new_role, old_role),
    'old_status', old_status,
    'new_status', COALESCE(new_status, old_status)
  );

  -- Insert audit log
  INSERT INTO audit_logs (actor_id, action, entity_type, entity_id, metadata)
  VALUES (
    auth.uid(),
    'admin_update_user',
    'profile',
```

```
    target_user_id,
    json_meta
  );

END;
$$;

```

supabase/migrations/00003_courses.sql SQL 3.4 KB 120 Lines

```sql
-- Courses table
CREATE TABLE public.courses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title TEXT NOT NULL,
  description TEXT,
  course_type TEXT NOT NULL DEFAULT 'standard' CHECK (course_type IN ('standard', 'scenario')),
  trainer_id UUID REFERENCES public.profiles(id),
  department TEXT,
  duration_minutes INTEGER,
  passing_score INTEGER DEFAULT 60,
  status TEXT NOT NULL DEFAULT 'draft' CHECK (status IN ('draft', 'pending_review', 'published', 'archived')),
  thumbnail_path TEXT,
  published_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- Enable RLS
ALTER TABLE public.courses ENABLE ROW LEVEL SECURITY;

-- Trainers can see their own courses (all statuses)
CREATE POLICY "Trainers can view own courses"
ON public.courses FOR SELECT
USING (auth.uid() = trainer_id);

-- Trainees can view published courses
CREATE POLICY "Trainees can view published courses"
ON public.courses FOR SELECT
USING (status = 'published');

-- Admins can view and manage all courses
CREATE POLICY "Admins have full access to courses"
ON public.courses FOR ALL
USING (
  EXISTS (SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role = 'admin')
);

-- Trainers can insert courses for themselves
CREATE POLICY "Trainers can create own courses"
ON public.courses FOR INSERT
WITH CHECK (
  auth.uid() = trainer_id AND
  EXISTS (SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role = 'trainer')
);

-- Trainers can update their own courses (only if not published/archived)
CREATE POLICY "Trainers can update own non-published courses"
ON public.courses FOR UPDATE
USING (
  auth.uid() = trainer_id AND
  EXISTS (SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role = 'trainer') AND
  status IN ('draft', 'pending_review')
)
WITH CHECK (
  auth.uid() = trainer_id AND
  -- Trainers cannot set status to published or archived themselves
  status IN ('draft', 'pending_review')
);

-- Updated_at trigger
CREATE OR REPLACE FUNCTION update_courses_updated_at()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = now();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER courses_updated_at_trigger
BEFORE UPDATE ON public.courses
FOR EACH ROW EXECUTE FUNCTION update_courses_updated_at();

-- RPC for admin to publish/archive a course and log it
CREATE OR REPLACE FUNCTION public.admin_update_course(
  target_course_id UUID,
  new_status TEXT
```

```
)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
  caller_role user_role;
  old_status TEXT;
BEGIN
  -- Verify caller is admin
  SELECT role INTO caller_role FROM profiles WHERE id = auth.uid();
  IF caller_role != 'admin' THEN
    RAISE EXCEPTION 'Unauthorized: only admins can perform this action';
  END IF;

  -- Validate new status
  IF new_status NOT IN ('draft', 'pending_review', 'published', 'archived') THEN
    RAISE EXCEPTION 'Invalid status value';
  END IF;

  -- Get old status
  SELECT status INTO old_status FROM courses WHERE id = target_course_id;

  -- Update course
  UPDATE courses
  SET
    status = new_status,
    published_at = CASE WHEN new_status = 'published' THEN now() ELSE published_at END,
    updated_at = now()
  WHERE id = target_course_id;

  -- Audit log
  INSERT INTO audit_logs (actor_id, action, entity_type, entity_id, metadata)
  VALUES (
    auth.uid(),
    'admin_update_course',
    'course',
    target_course_id,
    jsonb_build_object('old_status', old_status, 'new_status', new_status)
  );
END;
$$;

```

supabase/migrations/00004_materials.sql SQL 3.5 KB 126 Lines

```sql
-- Create the materials table
CREATE TABLE IF NOT EXISTS materials (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    course_id UUID REFERENCES courses(id) ON DELETE CASCADE NOT NULL,
    uploaded_by UUID REFERENCES profiles(id) ON DELETE SET NULL,
    file_name TEXT NOT NULL,
    storage_path TEXT NOT NULL,
    mime_type TEXT,
    file_size BIGINT,
    extracted_text TEXT,
    extraction_status TEXT DEFAULT 'pending' CHECK (extraction_status IN ('pending', 'completed', 'failed')),
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- Enable RLS on the materials table
ALTER TABLE materials ENABLE ROW LEVEL SECURITY;

-- Trainers can insert materials for their own courses
CREATE POLICY "Trainers can insert materials for their courses"
ON materials FOR INSERT
TO authenticated
WITH CHECK (
    EXISTS (
        SELECT 1 FROM courses
        WHERE id = materials.course_id
        AND trainer_id = auth.uid()
    )
);

-- Trainers can select materials for their own courses
CREATE POLICY "Trainers can view materials for their courses"
ON materials FOR SELECT
TO authenticated
USING (
    EXISTS (
        SELECT 1 FROM courses
        WHERE id = materials.course_id
        AND trainer_id = auth.uid()
    )
);

-- Trainers can delete materials for their own courses
CREATE POLICY "Trainers can delete materials for their courses"
ON materials FOR DELETE
TO authenticated
USING (
    EXISTS (
        SELECT 1 FROM courses
        WHERE id = materials.course_id
        AND trainer_id = auth.uid()
    )
);

-- Admins can do everything
CREATE POLICY "Admins can manage all materials"
ON materials FOR ALL
TO authenticated
USING (
    EXISTS (
        SELECT 1 FROM profiles WHERE id = auth.uid() AND role = 'admin'
    )
);

-- Note: Trainee policy will be added in Phase 5 when enrollments exist.
-- Currently Trainees cannot access any materials.

-- Set up the Storage Bucket
INSERT INTO storage.buckets (id, name, public)
VALUES ('materials', 'materials', false)
ON CONFLICT (id) DO NOTHING;

-- Storage RLS Policies
-- Allow Trainers to upload files to their courses
-- The storage_path must match: materials/<course_id>/<file_name>
-- We can't easily join with the courses table directly in the storage.objects RLS policies
-- without a function, so we'll use a simpler policy for now, or use an RPC.
```

```sql
-- Wait, Supabase storage policies CAN query the DB using `auth.uid()`.
-- The name of the file in the bucket is typically `{course_id}/{filename}`.
-- The path tokens are available via `storage.foldername(name)`.

CREATE POLICY "Trainers can upload materials"
ON storage.objects FOR INSERT
TO authenticated
WITH CHECK (
    bucket_id = 'materials' AND
    EXISTS (
        SELECT 1 FROM courses
        WHERE id::text = (storage.foldername(name))[1]
        AND trainer_id = auth.uid()
    )
);

CREATE POLICY "Trainers can delete materials"
ON storage.objects FOR DELETE
TO authenticated
USING (
    bucket_id = 'materials' AND
    EXISTS (
        SELECT 1 FROM courses
        WHERE id::text = (storage.foldername(name))[1]
        AND trainer_id = auth.uid()
    )
);

CREATE POLICY "Trainers can view materials"
ON storage.objects FOR SELECT
TO authenticated
USING (
    bucket_id = 'materials' AND
    EXISTS (
        SELECT 1 FROM courses
        WHERE id::text = (storage.foldername(name))[1]
        AND trainer_id = auth.uid()
    )
);

CREATE POLICY "Admins can access all materials"
ON storage.objects FOR ALL
TO authenticated
USING (
    bucket_id = 'materials' AND
    EXISTS (
        SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role = 'admin'
    )
);
```

supabase/migrations/00005_super_admin.sql SQL 3.1 KB 103 Lines

```sql
-- ============================================================
-- Safe migration: Add super_admin role support
-- Run this AFTER the initial schema
-- ============================================================

-- 1. Update the check constraint on the role column to include 'super_admin'
DO $$
BEGIN
  -- We drop the existing constraint if it exists. Note: The name of the constraint
  -- is typically generated as profiles_role_check if not explicitly named.
  -- To be safe, we find and drop it dynamically.
  DECLARE
    constraint_name text;
  BEGIN
    SELECT conname INTO constraint_name
    FROM pg_constraint
    WHERE conrelid = 'public.profiles'::regclass AND contype = 'c' AND pg_get_constraintdef(oid) LIKE '%role%';

    IF constraint_name IS NOT NULL THEN
      EXECUTE 'ALTER TABLE public.profiles DROP CONSTRAINT ' || constraint_name;
    END IF;
  END;

  -- Add the new constraint
  ALTER TABLE public.profiles
  ADD CONSTRAINT profiles_role_check
  CHECK (role IN ('trainee', 'trainer', 'admin', 'super_admin'));
END $$;

-- 2. Create a security definer function to check if the current user is an admin/super_admin
-- Since it is SECURITY DEFINER, it bypasses RLS and won't cause infinite recursion when checking roles!
CREATE OR REPLACE FUNCTION public.is_admin()
RETURNS boolean
LANGUAGE sql
SECURITY DEFINER
SET search_path = public
AS $$
  SELECT EXISTS (
    SELECT 1 FROM public.profiles
    WHERE id = auth.uid() AND role IN ('admin', 'super_admin')
  );
$$;

-- 3. Update RLS policy: both admin and super_admin can do everything on profiles
DROP POLICY IF EXISTS "Admins can do everything on profiles" ON public.profiles;
DROP POLICY IF EXISTS "Admins and Super Admins can do everything on profiles" ON public.profiles;

CREATE POLICY "Admins and Super Admins can do everything on profiles"
ON public.profiles
FOR ALL
USING ( public.is_admin() );

-- 3. Update protect_profile_fields trigger: both admin and super_admin can modify roles
CREATE OR REPLACE FUNCTION protect_profile_fields()
RETURNS TRIGGER AS $$
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM public.profiles
    WHERE id = auth.uid() AND role IN ('admin', 'super_admin')
  ) THEN
    NEW.role = OLD.role;
    NEW.approval_status = OLD.approval_status;
  END IF;
  NEW.updated_at = now();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- 4. New RPC: Super Admin only — promote a user to admin
CREATE OR REPLACE FUNCTION public.super_admin_promote_to_admin(
  target_user_id UUID
)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
```

```
AS $$
DECLARE
  caller_role text;
  old_role text;
  old_status text;
  json_meta JSONB;
BEGIN
  -- Verify caller is super_admin
  SELECT role INTO caller_role FROM profiles WHERE id = auth.uid();
  IF caller_role != 'super_admin' THEN
    RAISE EXCEPTION 'Unauthorized: only super admins can promote users to admin';
  END IF;

  -- Get current state
  SELECT role, approval_status INTO old_role, old_status FROM profiles WHERE id = target_user_id;

  -- Promote to admin
  UPDATE profiles
  SET
    role = 'admin',
    approval_status = 'approved',
    updated_at = now()
  WHERE id = target_user_id;

END;
$$;

```

supabase/migrations/00006_trainer_rls_and_notifications.sql SQL 8.1 KB 303 Lines

```sql
-- Notifications table (if not exists from types)
CREATE TABLE IF NOT EXISTS public.notifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,
  type TEXT NOT NULL,
  title TEXT NOT NULL,
  message TEXT NOT NULL,
  related_certificate_id UUID,
  read_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now()
);

ALTER TABLE public.notifications ENABLE ROW LEVEL SECURITY;

-- Users can read their own notifications
CREATE POLICY "Users can read own notifications"
ON public.notifications FOR SELECT
USING (auth.uid() = user_id);

-- Users can mark their own as read
CREATE POLICY "Users can update own notifications"
ON public.notifications FOR UPDATE
USING (auth.uid() = user_id)
WITH CHECK (auth.uid() = user_id);

-- Admins and Super Admins can read all notifications
CREATE POLICY "Admins can read all notifications"
ON public.notifications FOR SELECT
USING (
  EXISTS (
    SELECT 1 FROM public.profiles
    WHERE id = auth.uid() AND role IN ('admin', 'super_admin')
  )
);

-- Notifications are inserted only by server-side (Edge Functions / SECURITY DEFINER)
-- No public INSERT policy needed.

-- Assessments RLS (if not exists)
DO $$
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM pg_policies WHERE policyname = 'Trainers can manage own assessments'
  ) THEN
    ALTER TABLE public.assessments ENABLE ROW LEVEL SECURITY;

    CREATE POLICY "Trainers can manage own assessments"
    ON public.assessments FOR ALL
    USING (
      auth.uid() = created_by AND
      EXISTS (
        SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role = 'trainer'
      )
    );

    CREATE POLICY "Trainees can view published assessments"
    ON public.assessments FOR SELECT
    USING (status = 'published');

    CREATE POLICY "Admins can manage all assessments"
    ON public.assessments FOR ALL
    USING (
      EXISTS (
        SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role IN ('admin', 'super_admin')
      )
    );
  END IF;
END
$$;

-- Questions RLS
DO $$
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM pg_policies WHERE policyname = 'Trainers can manage questions for own assessments'
  ) THEN
```

```sql
    ALTER TABLE public.questions ENABLE ROW LEVEL SECURITY;

    CREATE POLICY "Trainers can manage questions for own assessments"
    ON public.questions FOR ALL
    USING (
      EXISTS (
        SELECT 1 FROM public.assessments a
        WHERE a.id = questions.assessment_id AND a.created_by = auth.uid()
      )
    );

    CREATE POLICY "Trainees can view safe questions"
    ON public.questions FOR SELECT
    USING (
      EXISTS (
        SELECT 1 FROM public.assessments a
        WHERE a.id = questions.assessment_id AND a.status = 'published'
      )
    );

    CREATE POLICY "Admins can manage all questions"
    ON public.questions FOR ALL
    USING (
      EXISTS (
        SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role IN ('admin', 'super_admin')
      )
    );
  END IF;
END
$$;

-- Enrollments RLS
DO $$
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM pg_policies WHERE policyname = 'Trainers can view enrollments for own courses'
  ) THEN
    ALTER TABLE public.enrollments ENABLE ROW LEVEL SECURITY;

    CREATE POLICY "Trainers can view enrollments for own courses"
    ON public.enrollments FOR SELECT
    USING (
      EXISTS (
        SELECT 1 FROM public.courses
        WHERE id = enrollments.course_id AND trainer_id = auth.uid()
      )
    );

    CREATE POLICY "Trainees can view own enrollments"
    ON public.enrollments FOR SELECT
    USING (auth.uid() = user_id);

    CREATE POLICY "Trainees can enroll in published courses"
    ON public.enrollments FOR INSERT
    WITH CHECK (
      auth.uid() = user_id AND
      EXISTS (
        SELECT 1 FROM public.courses WHERE id = course_id AND status = 'published'
      )
    );

    CREATE POLICY "Admins can manage all enrollments"
    ON public.enrollments FOR ALL
    USING (
      EXISTS (
        SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role IN ('admin', 'super_admin')
      )
    );
  END IF;
END
$$;

-- Assessment attempts RLS
DO $$
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM pg_policies WHERE policyname = 'Trainers can view attempts for own course assessments'
  ) THEN
    ALTER TABLE public.assessment_attempts ENABLE ROW LEVEL SECURITY;
```

```
    CREATE POLICY "Trainers can view attempts for own course assessments"
    ON public.assessment_attempts FOR SELECT
    USING (
      EXISTS (
        SELECT 1 FROM public.assessments a
        JOIN public.courses c ON c.id = a.course_id
        WHERE a.id = assessment_attempts.assessment_id AND c.trainer_id = auth.uid()
      )
    );

    CREATE POLICY "Trainees can view own attempts"
    ON public.assessment_attempts FOR SELECT
    USING (auth.uid() = user_id);

    CREATE POLICY "Trainees can create own attempts"
    ON public.assessment_attempts FOR INSERT
    WITH CHECK (auth.uid() = user_id);

    CREATE POLICY "Admins can manage all attempts"
    ON public.assessment_attempts FOR ALL
    USING (
      EXISTS (
        SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role IN ('admin', 'super_admin')
      )
    );
  END IF;
END
$$;

-- Attempt answers RLS
DO $$
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM pg_policies WHERE policyname = 'Trainers can view answers for own course attempts'
  ) THEN
    ALTER TABLE public.attempt_answers ENABLE ROW LEVEL SECURITY;

    CREATE POLICY "Trainers can view answers for own course attempts"
    ON public.attempt_answers FOR SELECT
    USING (
      EXISTS (
        SELECT 1 FROM public.assessment_attempts aa
        JOIN public.assessments a ON a.id = aa.assessment_id
        JOIN public.courses c ON c.id = a.course_id
        WHERE aa.id = attempt_answers.attempt_id AND c.trainer_id = auth.uid()
      )
    );

    CREATE POLICY "Trainees can view own answers"
    ON public.attempt_answers FOR SELECT
    USING (
      EXISTS (
        SELECT 1 FROM public.assessment_attempts aa
        WHERE aa.id = attempt_answers.attempt_id AND aa.user_id = auth.uid()
      )
    );

    CREATE POLICY "Trainees can insert own answers"
    ON public.attempt_answers FOR INSERT
    WITH CHECK (
      EXISTS (
        SELECT 1 FROM public.assessment_attempts aa
        WHERE aa.id = attempt_answers.attempt_id AND aa.user_id = auth.uid()
      )
    );

    CREATE POLICY "Admins can manage all answers"
    ON public.attempt_answers FOR ALL
    USING (
      EXISTS (
        SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role IN ('admin', 'super_admin')
      )
    );
  END IF;
END
$$;

-- Skills RLS
```

```
DO $$
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM pg_policies WHERE policyname = 'All authenticated can read skills'
  ) THEN
    ALTER TABLE public.skills ENABLE ROW LEVEL SECURITY;

    CREATE POLICY "All authenticated can read skills"
    ON public.skills FOR SELECT
    TO authenticated USING (true);

    CREATE POLICY "Admins can manage skills"
    ON public.skills FOR ALL
    USING (
      EXISTS (
        SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role IN ('admin', 'super_admin')
      )
    );
  END IF;
END
$$;

-- User skills RLS
DO $$
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM pg_policies WHERE policyname = 'Users can manage own skills'
  ) THEN
    ALTER TABLE public.user_skills ENABLE ROW LEVEL SECURITY;

    CREATE POLICY "Users can manage own skills"
    ON public.user_skills FOR ALL
    USING (auth.uid() = user_id);

    CREATE POLICY "Admins can manage all user skills"
    ON public.user_skills FOR ALL
    USING (
      EXISTS (
        SELECT 1 FROM public.profiles WHERE id = auth.uid() AND role IN ('admin', 'super_admin')
      )
    );
  END IF;
END
$$;

-- Course skills RLS
DO $$
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM pg_policies WHERE policyname = 'Trainers can manage own course skills'
  ) THEN
    ALTER TABLE public.course_skills ENABLE ROW LEVEL SECURITY;

    CREATE POLICY "Trainers can manage own course skills"
    ON public.course_skills FOR ALL
    USING (
      EXISTS (
        SELECT 1 FROM public.courses
        WHERE id = course_skills.course_id AND trainer_id = auth.uid()
      )
    );

    CREATE POLICY "All authenticated can read course skills"
    ON public.course_skills FOR SELECT
    TO authenticated USING (true);
  END IF;
END
$$;
```

supabase/migrations/00007_trainer_fields_and_objectives.sql SQL 1.4 KB 25 Lines

```sql
-- 00007_trainer_fields_and_objectives.sql
-- Adds trainer profile fields and course learning objectives columns

-- ============================================================
-- 1. Add trainer-specific columns to profiles
-- ============================================================
ALTER TABLE profiles ADD COLUMN IF NOT EXISTS biography TEXT;
ALTER TABLE profiles ADD COLUMN IF NOT EXISTS years_of_experience INTEGER;
ALTER TABLE profiles ADD COLUMN IF NOT EXISTS qualifications TEXT;
ALTER TABLE profiles ADD COLUMN IF NOT EXISTS availability TEXT DEFAULT 'available';

-- ============================================================
-- 2. Add learning objectives to courses table
-- ============================================================
ALTER TABLE courses ADD COLUMN IF NOT EXISTS learning_objectives JSONB;
ALTER TABLE courses ADD COLUMN IF NOT EXISTS competencies JSONB;

COMMENT ON COLUMN courses.learning_objectives IS 'JSONB: { understand: text, able_to_do: text, competencies_built: text }';
COMMENT ON COLUMN courses.competencies IS 'Free-text competencies description';

COMMENT ON COLUMN profiles.biography IS 'Trainer biography / about text';
COMMENT ON COLUMN profiles.years_of_experience IS 'Number of years of professional experience';
COMMENT ON COLUMN profiles.qualifications IS 'Qualifications text (degrees, certifications)';
COMMENT ON COLUMN profiles.availability IS 'Trainer availability status: available, busy, unavailable';

```

supabase/migrations/00008_seed_skills.sql SQL 1.2 KB 31 Lines

```sql
-- 00008_seed_skills.sql
-- Seeds the skills table with common training competencies for MoES/IMD

INSERT INTO skills (name, category) VALUES
  ('Weather Warning Interpretation', 'Meteorology'),
  ('Cyclone Communication', 'Emergency'),
  ('Emergency Coordination', 'Emergency'),
  ('Forecast Analysis', 'Meteorology'),
  ('Public Advisory Preparation', 'Communication'),
  ('Disaster Response', 'Emergency'),
  ('Data Analysis', 'Technical'),
  ('Radar Interpretation', 'Meteorology'),
  ('Satellite Imagery Analysis', 'Meteorology'),
  ('Ocean State Forecasting', 'Meteorology'),
  ('Flood Risk Assessment', 'Emergency'),
  ('Climate Monitoring', 'Meteorology'),
  ('Seismic Activity Monitoring', 'Geology'),
  ('Tsunami Warning Systems', 'Emergency'),
  ('Coastal Erosion Assessment', 'Geology'),
  ('GIS Mapping', 'Technical'),
  ('Remote Sensing', 'Technical'),
  ('Report Writing', 'Communication'),
  ('Stakeholder Communication', 'Communication'),
  ('Training & Facilitation', 'Leadership'),
  ('Team Leadership', 'Leadership'),
  ('Incident Command', 'Emergency'),
  ('Risk Assessment', 'Emergency'),
  ('Quality Assurance', 'Technical'),
  ('Public Speaking', 'Communication')
ON CONFLICT DO NOTHING;

```

supabase/migrations/00009_material_types_and_links.sql SQL 1.1 KB 32 Lines

```sql
-- Add material_type and url columns to materials table
-- material_type: 'file' (default), 'link', 'video'
-- url: stores the URL for link/video materials

DO $$
BEGIN
  -- Add material_type if it doesn't exist
  IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'materials' AND column_name = 'material_type') TH
EN
    ALTER TABLE materials ADD COLUMN material_type text NOT NULL DEFAULT 'file';
  END IF;

  -- Add url if it doesn't exist
  IF NOT EXISTS (SELECT 1 FROM information_schema.columns WHERE table_name = 'materials' AND column_name = 'url') THEN
    ALTER TABLE materials ADD COLUMN url text NULL;
  END IF;
END $$;

-- Update existing file materials to have correct type
UPDATE materials SET material_type = 'file' WHERE material_type IS NULL;

-- Safely add the check constraint
DO $$
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM pg_constraint WHERE conname = 'materials_material_type_check'
  ) THEN
    ALTER TABLE materials
      ADD CONSTRAINT materials_material_type_check
      CHECK (material_type IN ('file', 'link', 'video'));
  END IF;
END $$;

```

supabase/migrations/00010_fix_admin_update_course_rpc.sql SQL 1.2 KB 47 Lines

```sql
-- Fix admin_update_course to support super_admin and use text instead of enum for caller_role
CREATE OR REPLACE FUNCTION public.admin_update_course(
  target_course_id UUID,
  new_status TEXT
)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
  caller_role TEXT;
  old_status TEXT;
BEGIN
  -- Verify caller is admin or super_admin
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Unauthorized: only admins can perform this action';
  END IF;

  -- Validate new status
  IF new_status NOT IN ('draft', 'pending_review', 'published', 'archived') THEN
    RAISE EXCEPTION 'Invalid status value';
  END IF;

  -- Get old status
  SELECT status INTO old_status FROM courses WHERE id = target_course_id;

  -- Update course
  UPDATE courses
  SET
    status = new_status,
    published_at = CASE WHEN new_status = 'published' THEN now() ELSE published_at END,
    updated_at = now()
  WHERE id = target_course_id;

  -- Audit log
  INSERT INTO audit_logs (actor_id, action, entity_type, entity_id, metadata)
  VALUES (
    auth.uid(),
    'admin_update_course',
    'course',
    target_course_id,
    jsonb_build_object('old_status', old_status, 'new_status', new_status)
  );
END;
$$;

```

supabase/migrations/00011_trainee_material_access.sql SQL 0.6 KB 23 Lines

```sql
-- Enable trainees to see thumbnails (on course catalog)
CREATE POLICY "Anyone can view thumbnails"
ON storage.objects FOR SELECT
TO authenticated
USING (
    bucket_id = 'materials' AND
    name ILIKE '%/thumbnail.%'
);

-- Enable trainees to view actual course materials (PDFs, videos) only if enrolled
CREATE POLICY "Trainees can view materials for enrolled courses"
ON storage.objects FOR SELECT
TO authenticated
USING (
    bucket_id = 'materials' AND
    EXISTS (
        SELECT 1 FROM public.enrollments
        WHERE course_id::text = (storage.foldername(name))[1]
        AND user_id = auth.uid()
        AND status = 'enrolled'
    )
);

```

supabase/migrations/00012_trainee_approvals.sql SQL 2.5 KB 99 Lines

```sql
-- 1. Add proof_path to profiles
ALTER TABLE public.profiles ADD COLUMN IF NOT EXISTS proof_path TEXT;

-- 2. Update handle_new_user trigger to include proof_path from raw_user_meta_data
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER SET search_path = public
AS $$
BEGIN
  INSERT INTO public.profiles (id, email, full_name, role, department, designation, approval_status, proof_path)
  VALUES (
    NEW.id,
    NEW.email,
    NEW.raw_user_meta_data->>'full_name',
    'trainee',
    NEW.raw_user_meta_data->>'department',
    NEW.raw_user_meta_data->>'designation',
    'pending',
    NEW.raw_user_meta_data->>'proof_path'
  );
  RETURN NEW;
END;
$$;

-- 3. Create proofs storage bucket
INSERT INTO storage.buckets (id, name, public)
VALUES ('proofs', 'proofs', false)
ON CONFLICT (id) DO NOTHING;

-- 4. Enable RLS on proofs bucket
CREATE POLICY "Anyone can upload proofs"
ON storage.objects FOR INSERT
TO public
WITH CHECK (bucket_id = 'proofs');

CREATE POLICY "Users can read own proof"
ON storage.objects FOR SELECT
TO authenticated
USING (bucket_id = 'proofs' AND (storage.foldername(name))[1] = auth.uid()::text);

CREATE POLICY "Admins can read all proofs"
ON storage.objects FOR SELECT
TO authenticated
USING (bucket_id = 'proofs' AND public.is_admin());

-- 5. Fix admin_update_user to support super_admin
CREATE OR REPLACE FUNCTION public.admin_update_user(
  target_user_id UUID,
  new_role TEXT DEFAULT NULL,
  new_status TEXT DEFAULT NULL
)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
  old_role TEXT;
  old_status TEXT;
  json_meta JSONB;
BEGIN
  -- Verify caller is admin or super_admin
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Unauthorized: only admins can perform this action';
  END IF;

  -- Get current state
  SELECT role, approval_status INTO old_role, old_status FROM profiles WHERE id = target_user_id;

  -- Update profile
  UPDATE profiles
  SET
    role = COALESCE(new_role, role),
    approval_status = COALESCE(new_status, approval_status),
    updated_at = now()
```

```
  WHERE id = target_user_id;

  -- Construct metadata for audit log
  json_meta = jsonb_build_object(
    'old_role', old_role,
    'new_role', COALESCE(new_role, old_role),
    'old_status', old_status,
    'new_status', COALESCE(new_status, old_status)
  );

  -- Insert audit log
  INSERT INTO audit_logs (actor_id, action, entity_type, entity_id, metadata)
  VALUES (
    auth.uid(),
    'admin_update_user',
    'profile',
    target_user_id,
    json_meta
  );

END;
$$;

```

supabase/migrations/00013_admin_delete_user.sql SQL 0.9 KB 31 Lines

```sql
-- Create an RPC to completely delete a user's account when they are rejected
CREATE OR REPLACE FUNCTION public.admin_delete_user(target_user_id UUID)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
BEGIN
  -- 1. Verify caller is an admin or super_admin
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Unauthorized: only admins can perform this action';
  END IF;

  -- 2. Delete the profile (this will cascade delete or just remove the profile so we can delete auth.user)
  DELETE FROM public.profiles WHERE id = target_user_id;

  -- 3. Delete the auth user
  DELETE FROM auth.users WHERE id = target_user_id;

  -- 4. Log the action
  INSERT INTO public.audit_logs (actor_id, action, entity_type, entity_id, metadata)
  VALUES (
    auth.uid(),
    'admin_delete_user',
    'profile',
    target_user_id,
    jsonb_build_object('status', 'rejected_and_deleted')
  );
END;
$$;

```

supabase/migrations/00014_fix_proof_storage.sql SQL 2.5 KB 79 Lines

```sql
-- Fix storage RLS policies for proofs bucket

-- Drop old broken policies

DROP POLICY IF EXISTS "Anyone can upload proofs" ON storage.objects;
DROP POLICY IF EXISTS "Users can read own proof" ON storage.objects;
DROP POLICY IF EXISTS "Admins can read all proofs" ON storage.objects;

-- Allow anyone (unauthenticated during signup) to upload to proofs bucket

-- The path will be: <userId>/<filename> which is set after the user is created

CREATE POLICY "Authenticated users can upload proofs"
ON storage.objects FOR INSERT
TO authenticated
WITH CHECK (bucket_id = 'proofs');

-- Allow users to read their own proof (path: <their_uid>/filename)

CREATE POLICY "Users can read own proof"
ON storage.objects FOR SELECT
TO authenticated
USING (
  bucket_id = 'proofs'
  AND (storage.foldername(name))[1] = auth.uid()::text
);

-- Allow admins to read ALL proofs

CREATE POLICY "Admins can read all proofs"
ON storage.objects FOR SELECT
TO authenticated
USING (
  bucket_id = 'proofs'
  AND public.is_admin()
);

-- Allow admins to delete proof files (for rejection flow)

CREATE POLICY "Admins can delete proofs"
ON storage.objects FOR DELETE
TO authenticated
USING (
  bucket_id = 'proofs'
  AND public.is_admin()
);

-- Recreate admin_delete_user RPC to also clean up storage via metadata

-- (Storage file deletion must be done from the frontend since edge functions

-- can't easily delete from storage. This RPC handles DB cleanup only.)

CREATE OR REPLACE FUNCTION public.admin_delete_user(target_user_id UUID)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
  v_proof_path TEXT;
BEGIN
  -- 1. Verify caller is an admin or super_admin

  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Unauthorized: only admins can perform this action';
  END IF;

  -- 2. Get the proof path before deleting

  SELECT proof_path INTO v_proof_path FROM public.profiles WHERE id = target_user_id;

  -- 3. Log the action BEFORE deleting (so audit log has the actor)

  INSERT INTO public.audit_logs (actor_id, action, entity_type, entity_id, metadata)
  VALUES (
```

```
    auth.uid(),
    'admin_reject_and_delete_user',
    'profile',
    target_user_id,
    jsonb_build_object('proof_path', v_proof_path, 'status', 'rejected_and_deleted')
  );

  -- 4. Delete the profile row
  DELETE FROM public.profiles WHERE id = target_user_id;

  -- 5. Delete the auth user (this cascades profile deletion too, but profile is already gone)
  DELETE FROM auth.users WHERE id = target_user_id;

END;
$$;
```

supabase/migrations/00015_seed_super_admin.sql SQL 1.4 KB 59 Lines

```sql
-- Insert the Super Admin into auth.users if they don't exist
DO $$
DECLARE
  v_user_id UUID;
  v_email TEXT := 'sanjithchandrasekar03@gmail.com';
  v_password TEXT := '12345678';
BEGIN
  SELECT id INTO v_user_id FROM auth.users WHERE email = v_email;

  IF v_user_id IS NULL THEN
    v_user_id := gen_random_uuid();

    INSERT INTO auth.users (
      id,
      instance_id,
      email,
      encrypted_password,
      email_confirmed_at,
      raw_app_meta_data,
      raw_user_meta_data,
      created_at,
      updated_at,
      role,
      confirmation_token,
      recovery_token,
      email_change_token_new,
      email_change
    ) VALUES (
      v_user_id,
      '00000000-0000-0000-0000-000000000000',
      v_email,
      crypt(v_password, gen_salt('bf')),
      now(),
      '{"provider": "email", "providers": ["email"]}',
      jsonb_build_object('full_name', 'Sanjith Chandrasekar', 'role', 'super_admin'),
      now(),
      now(),
      'authenticated',
      '',
      '',
      '',
      ''
    );
  END IF;

  -- Ensure the profile exists and is a super_admin and approved
  UPDATE public.profiles
  SET
    role = 'super_admin',
    approval_status = 'approved',
    full_name = 'Sanjith Chandrasekar'
  WHERE id = v_user_id;

  IF NOT FOUND THEN
    INSERT INTO public.profiles (id, email, full_name, role, approval_status)
    VALUES (v_user_id, v_email, 'Sanjith Chandrasekar', 'super_admin', 'approved');
  END IF;
END $$;

```

supabase/migrations/00016_separate_role_tables.sql SQL 4.7 KB 130 Lines

```sql
-- 1. Create the separate role tables

CREATE TABLE public.admins (
  id UUID REFERENCES public.profiles(id) ON DELETE CASCADE PRIMARY KEY,
  admin_level TEXT DEFAULT 'standard',
  permissions_granted_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE public.trainers (
  id UUID REFERENCES public.profiles(id) ON DELETE CASCADE PRIMARY KEY,
  expertise_areas TEXT[] DEFAULT '{}',
  years_of_experience INTEGER,
  bio TEXT
);

CREATE TABLE public.trainees (
  id UUID REFERENCES public.profiles(id) ON DELETE CASCADE PRIMARY KEY,
  department TEXT,
  designation TEXT,
  learning_goals TEXT
);

-- 2. Enable RLS
ALTER TABLE public.admins ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.trainers ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.trainees ENABLE ROW LEVEL SECURITY;

-- 3. RLS Policies
CREATE POLICY "Admins can view all admins" ON public.admins FOR SELECT TO authenticated USING (public.is_admin());
CREATE POLICY "Admins can view themselves" ON public.admins FOR SELECT TO authenticated USING (auth.uid() = id);

CREATE POLICY "Anyone can view trainers" ON public.trainers FOR SELECT TO authenticated USING (true);
CREATE POLICY "Trainers can update themselves" ON public.trainers FOR UPDATE TO authenticated USING (auth.uid() = id);

CREATE POLICY "Admins can view trainees" ON public.trainees FOR SELECT TO authenticated USING (public.is_admin());
CREATE POLICY "Trainees can view themselves" ON public.trainees FOR SELECT TO authenticated USING (auth.uid() = id);
CREATE POLICY "Trainees can update themselves" ON public.trainees FOR UPDATE TO authenticated USING (auth.uid() = id);

-- 4. Seed existing data from profiles into the new tables
INSERT INTO public.admins (id, admin_level)
SELECT id, CASE WHEN role = 'super_admin' THEN 'super_admin' ELSE 'standard' END
FROM public.profiles
WHERE role IN ('admin', 'super_admin')
ON CONFLICT DO NOTHING;

INSERT INTO public.trainers (id)
SELECT id FROM public.profiles WHERE role = 'trainer'
ON CONFLICT DO NOTHING;

INSERT INTO public.trainees (id, department, designation)
SELECT id, department, designation FROM public.profiles WHERE role = 'trainee'
ON CONFLICT DO NOTHING;

-- 5. Trigger to automatically route data when profile is inserted or updated
CREATE OR REPLACE FUNCTION public.handle_role_tables()
RETURNS TRIGGER AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    IF NEW.role IN ('admin', 'super_admin') THEN
      INSERT INTO public.admins (id, admin_level) VALUES (NEW.id, CASE WHEN NEW.role = 'super_admin' THEN 'super_admin' ELSE
'standard' END) ON CONFLICT DO NOTHING;
    ELSIF NEW.role = 'trainer' THEN
      INSERT INTO public.trainers (id) VALUES (NEW.id) ON CONFLICT DO NOTHING;
    ELSIF NEW.role = 'trainee' THEN
      INSERT INTO public.trainees (id, department, designation) VALUES (NEW.id, NEW.department, NEW.designation) ON CONFLICT D
O NOTHING;
    END IF;
  ELSIF TG_OP = 'UPDATE' THEN
    -- If role changed, delete from old and insert to new
    IF OLD.role != NEW.role THEN
      DELETE FROM public.admins WHERE id = NEW.id;
      DELETE FROM public.trainers WHERE id = NEW.id;
      DELETE FROM public.trainees WHERE id = NEW.id;

      IF NEW.role IN ('admin', 'super_admin') THEN
        INSERT INTO public.admins (id, admin_level) VALUES (NEW.id, CASE WHEN NEW.role = 'super_admin' THEN 'super_admin' ELSE
```

```
'standard' END);
      ELSIF NEW.role = 'trainer' THEN
        INSERT INTO public.trainers (id) VALUES (NEW.id);
      ELSIF NEW.role = 'trainee' THEN
        INSERT INTO public.trainees (id, department, designation) VALUES (NEW.id, NEW.department, NEW.designation);
      END IF;
    END IF;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE TRIGGER trigger_handle_role_tables
AFTER INSERT OR UPDATE OF role ON public.profiles
FOR EACH ROW EXECUTE FUNCTION public.handle_role_tables();









DO $$
DECLARE
  v_user_id UUID := gen_random_uuid();
  v_email TEXT := 'superadmin@capacityconnect.com';
  v_password TEXT := '12345678';
BEGIN
  -- 1. Create a brand new, clean user
  INSERT INTO auth.users (
    id, instance_id, email, encrypted_password, email_confirmed_at,
    raw_app_meta_data, raw_user_meta_data, created_at, updated_at,
    role, confirmation_token, recovery_token, email_change_token_new, email_change
  ) VALUES (
    v_user_id, '00000000-0000-0000-0000-000000000000', v_email,
    crypt(v_password, gen_salt('bf')), now(),
    '{"provider": "email", "providers": ["email"]}',
    jsonb_build_object('full_name', 'System Super Admin', 'role', 'super_admin'),
    now(), now(), 'authenticated', '', '', '', ''
  );

  -- 2. Update the profile that was automatically generated by your database triggers
  UPDATE public.profiles
  SET
    role = 'super_admin',
    approval_status = 'approved',
    full_name = 'System Super Admin'
  WHERE id = v_user_id;

END $$;
```

supabase/migrations/00017_drop_profiles_and_refactor.sql SQL 13.5 KB 326 Lines

```sql
-- 1. Add common columns to admins, trainers, trainees
ALTER TABLE public.admins
  ADD COLUMN IF NOT EXISTS full_name TEXT NOT NULL DEFAULT 'Admin User',
  ADD COLUMN IF NOT EXISTS email TEXT,
  ADD COLUMN IF NOT EXISTS role TEXT NOT NULL DEFAULT 'admin',
  ADD COLUMN IF NOT EXISTS approval_status TEXT NOT NULL DEFAULT 'pending',
  ADD COLUMN IF NOT EXISTS avatar_path TEXT,
  ADD COLUMN IF NOT EXISTS created_at TIMESTAMPTZ DEFAULT now(),
  ADD COLUMN IF NOT EXISTS updated_at TIMESTAMPTZ DEFAULT now();

ALTER TABLE public.trainers
  ADD COLUMN IF NOT EXISTS full_name TEXT NOT NULL DEFAULT 'Trainer User',
  ADD COLUMN IF NOT EXISTS email TEXT,
  ADD COLUMN IF NOT EXISTS role TEXT NOT NULL DEFAULT 'trainer',
  ADD COLUMN IF NOT EXISTS approval_status TEXT NOT NULL DEFAULT 'pending',
  ADD COLUMN IF NOT EXISTS avatar_path TEXT,
  ADD COLUMN IF NOT EXISTS proof_path TEXT,
  ADD COLUMN IF NOT EXISTS qualifications TEXT,
  ADD COLUMN IF NOT EXISTS availability TEXT DEFAULT 'available',
  ADD COLUMN IF NOT EXISTS created_at TIMESTAMPTZ DEFAULT now(),
  ADD COLUMN IF NOT EXISTS updated_at TIMESTAMPTZ DEFAULT now();

ALTER TABLE public.trainees
  ADD COLUMN IF NOT EXISTS full_name TEXT NOT NULL DEFAULT 'Trainee User',
  ADD COLUMN IF NOT EXISTS email TEXT,
  ADD COLUMN IF NOT EXISTS role TEXT NOT NULL DEFAULT 'trainee',
  ADD COLUMN IF NOT EXISTS approval_status TEXT NOT NULL DEFAULT 'pending',
  ADD COLUMN IF NOT EXISTS avatar_path TEXT,
  ADD COLUMN IF NOT EXISTS proof_path TEXT,
  ADD COLUMN IF NOT EXISTS created_at TIMESTAMPTZ DEFAULT now(),
  ADD COLUMN IF NOT EXISTS updated_at TIMESTAMPTZ DEFAULT now();

-- 2. Migrate data from profiles to the new tables
-- Admins
INSERT INTO public.admins (id, full_name, email, role, approval_status, avatar_path, created_at, updated_at)
SELECT id, full_name, email, role::text, approval_status::text, avatar_path, created_at, updated_at
FROM public.profiles
WHERE role IN ('admin', 'super_admin')
ON CONFLICT (id) DO UPDATE SET
  full_name = EXCLUDED.full_name,
  email = EXCLUDED.email,
  role = EXCLUDED.role,
  approval_status = EXCLUDED.approval_status,
  avatar_path = EXCLUDED.avatar_path,
  created_at = EXCLUDED.created_at,
  updated_at = EXCLUDED.updated_at;

-- Trainers
INSERT INTO public.trainers (id, full_name, email, role, approval_status, avatar_path, proof_path, bio, years_of_experience, qualifications, availability, created_at, updated_at)
SELECT id, full_name, email, role::text, approval_status::text, avatar_path, proof_path, biography, years_of_experience, qualifications, availability, created_at, updated_at
FROM public.profiles
WHERE role = 'trainer'
ON CONFLICT (id) DO UPDATE SET
  full_name = EXCLUDED.full_name,
  email = EXCLUDED.email,
  role = EXCLUDED.role,
  approval_status = EXCLUDED.approval_status,
  avatar_path = EXCLUDED.avatar_path,
  proof_path = EXCLUDED.proof_path,
  bio = COALESCE(trainers.bio, EXCLUDED.bio),
  years_of_experience = COALESCE(trainers.years_of_experience, EXCLUDED.years_of_experience),
  qualifications = COALESCE(trainers.qualifications, EXCLUDED.qualifications),
  availability = COALESCE(trainers.availability, EXCLUDED.availability),
  created_at = EXCLUDED.created_at,
  updated_at = EXCLUDED.updated_at;

-- Trainees
INSERT INTO public.trainees (id, full_name, email, role, approval_status, avatar_path, proof_path, department, designation, created_at, updated_at)
SELECT id, full_name, email, role::text, approval_status::text, avatar_path, proof_path, department, designation, created_at, updated_at
FROM public.profiles
WHERE role = 'trainee'
ON CONFLICT (id) DO UPDATE SET
```

```
  full_name = EXCLUDED.full_name,
  email = EXCLUDED.email,
  role = EXCLUDED.role,
  approval_status = EXCLUDED.approval_status,
  avatar_path = EXCLUDED.avatar_path,
  proof_path = EXCLUDED.proof_path,
  department = COALESCE(trainees.department, EXCLUDED.department),
  designation = COALESCE(trainees.designation, EXCLUDED.designation),
  created_at = EXCLUDED.created_at,
  updated_at = EXCLUDED.updated_at;

-- 3. Drop all foreign keys that rely on profiles
ALTER TABLE public.admins DROP CONSTRAINT IF EXISTS admins_id_fkey;
ALTER TABLE public.trainers DROP CONSTRAINT IF EXISTS trainers_id_fkey;
ALTER TABLE public.trainees DROP CONSTRAINT IF EXISTS trainees_id_fkey;

ALTER TABLE public.courses DROP CONSTRAINT IF EXISTS courses_trainer_id_fkey;
ALTER TABLE public.audit_logs DROP CONSTRAINT IF EXISTS audit_logs_actor_id_fkey;
ALTER TABLE public.materials DROP CONSTRAINT IF EXISTS materials_uploaded_by_fkey;
ALTER TABLE public.notifications DROP CONSTRAINT IF EXISTS notifications_user_id_fkey;

-- 4. Re-link foreign keys to auth.users or specific tables
ALTER TABLE public.admins ADD CONSTRAINT admins_id_fkey FOREIGN KEY (id) REFERENCES auth.users(id) ON DELETE CASCADE;
ALTER TABLE public.trainers ADD CONSTRAINT trainers_id_fkey FOREIGN KEY (id) REFERENCES auth.users(id) ON DELETE CASCADE;
ALTER TABLE public.trainees ADD CONSTRAINT trainees_id_fkey FOREIGN KEY (id) REFERENCES auth.users(id) ON DELETE CASCADE;

ALTER TABLE public.courses ADD CONSTRAINT courses_trainer_id_fkey FOREIGN KEY (trainer_id) REFERENCES public.trainers(id) ON D
ELETE SET NULL;
ALTER TABLE public.audit_logs ADD CONSTRAINT audit_logs_actor_id_fkey FOREIGN KEY (actor_id) REFERENCES auth.users(id) ON DELE
TE SET NULL;
ALTER TABLE public.materials ADD CONSTRAINT materials_uploaded_by_fkey FOREIGN KEY (uploaded_by) REFERENCES auth.users(id) ON
DELETE SET NULL;
ALTER TABLE public.notifications ADD CONSTRAINT notifications_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id) ON
DELETE CASCADE;

-- 5. Drop old triggers and functions that rely on profiles
DROP TRIGGER IF EXISTS trigger_handle_role_tables ON public.profiles;
DROP FUNCTION IF EXISTS public.handle_role_tables() CASCADE;
DROP TRIGGER IF EXISTS protect_profile_fields_trigger ON public.profiles;
DROP FUNCTION IF EXISTS public.protect_profile_fields() CASCADE;

-- Update handle_new_user to insert into trainees instead of profiles
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO public.trainees (id, full_name, email, role, approval_status)
  VALUES (
    NEW.id,
    COALESCE(NEW.raw_user_meta_data->>'full_name', NEW.email, 'New User'),
    NEW.email,
    'trainee',
    'pending'
  );
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Update is_admin function to check admins table
CREATE OR REPLACE FUNCTION public.is_admin()
RETURNS boolean
LANGUAGE sql
SECURITY DEFINER
SET search_path = public
AS $$
  SELECT EXISTS (
    SELECT 1 FROM public.admins
    WHERE id = auth.uid()
  );
$$;

-- 6. Re-write policies that directly use profiles
-- Courses
DROP POLICY IF EXISTS "Admins have full access to courses" ON public.courses;
CREATE POLICY "Admins have full access to courses" ON public.courses FOR ALL USING (public.is_admin());

DROP POLICY IF EXISTS "Trainers can create own courses" ON public.courses;
CREATE POLICY "Trainers can create own courses" ON public.courses FOR INSERT WITH CHECK (
  auth.uid() = trainer_id AND EXISTS (SELECT 1 FROM public.trainers WHERE id = auth.uid())
);
```

```
148 |
149 | DROP POLICY IF EXISTS "Trainers can update own non-published courses" ON public.courses;
150 | CREATE POLICY "Trainers can update own non-published courses" ON public.courses FOR UPDATE USING (
151 |   auth.uid() = trainer_id AND EXISTS (SELECT 1 FROM public.trainers WHERE id = auth.uid()) AND status IN ('draft', 'pending_re
view')
152 | ) WITH CHECK (
153 |   auth.uid() = trainer_id AND status IN ('draft', 'pending_review')
154 | );
155 |
156 | -- Audit Logs
157 | DROP POLICY IF EXISTS "Admins can view all audit logs" ON public.audit_logs;
158 | CREATE POLICY "Admins can view all audit logs" ON public.audit_logs FOR SELECT USING (public.is_admin());
159 |
160 | -- Materials
161 | DROP POLICY IF EXISTS "Admins can do everything on materials" ON public.materials;
162 | CREATE POLICY "Admins can do everything on materials" ON public.materials FOR ALL USING (public.is_admin());
163 |
164 | -- Notifications
165 | DROP POLICY IF EXISTS "Admins can read and write all notifications" ON public.notifications;
166 | CREATE POLICY "Admins can read and write all notifications" ON public.notifications FOR ALL USING (public.is_admin());
167 |
168 | -- 7. Rewrite Admin RPCs
169 | CREATE OR REPLACE FUNCTION public.admin_update_user(
170 |   target_user_id UUID,
171 |   new_role TEXT DEFAULT NULL,
172 |   new_status TEXT DEFAULT NULL
173 | ) RETURNS void
174 | LANGUAGE plpgsql
175 | SECURITY DEFINER
176 | SET search_path = public
177 | AS $$
178 | DECLARE
179 |   old_role TEXT;
180 |   old_status TEXT;
181 | BEGIN
182 |   IF NOT public.is_admin() THEN
183 |     RAISE EXCEPTION 'Unauthorized: only admins can perform this action';
184 |   END IF;
185 |
186 |   IF new_status NOT IN ('pending', 'approved', 'rejected', 'suspended') THEN
187 |     RAISE EXCEPTION 'Invalid status value';
188 |   END IF;
189 |
190 |   IF new_role NOT IN ('trainee', 'trainer', 'admin') THEN
191 |     RAISE EXCEPTION 'Invalid role value';
192 |   END IF;
193 |
194 |   -- Find the user's current table and update/move them
195 |   -- Let's check admins
196 |   IF EXISTS (SELECT 1 FROM admins WHERE id = target_user_id) THEN
197 |     SELECT role, approval_status INTO old_role, old_status FROM admins WHERE id = target_user_id;
198 |     IF new_role = 'admin' THEN
199 |       UPDATE admins SET approval_status = new_status, updated_at = now() WHERE id = target_user_id;
200 |     ELSE
201 |       -- Move to another table
202 |       INSERT INTO trainers (id, full_name, email, role, approval_status)
203 |       SELECT id, full_name, email, new_role, new_status FROM admins WHERE id = target_user_id
204 |       WHERE new_role = 'trainer';
205 |
206 |       INSERT INTO trainees (id, full_name, email, role, approval_status)
207 |       SELECT id, full_name, email, new_role, new_status FROM admins WHERE id = target_user_id
208 |       WHERE new_role = 'trainee';
209 |
210 |       DELETE FROM admins WHERE id = target_user_id;
211 |     END IF;
212 |   -- Let's check trainers
213 |   ELSIF EXISTS (SELECT 1 FROM trainers WHERE id = target_user_id) THEN
214 |     SELECT role, approval_status INTO old_role, old_status FROM trainers WHERE id = target_user_id;
215 |     IF new_role = 'trainer' THEN
216 |       UPDATE trainers SET approval_status = new_status, updated_at = now() WHERE id = target_user_id;
217 |     ELSE
218 |       INSERT INTO admins (id, full_name, email, role, approval_status)
219 |       SELECT id, full_name, email, new_role, new_status FROM trainers WHERE id = target_user_id
220 |       WHERE new_role = 'admin';
221 |
222 |       INSERT INTO trainees (id, full_name, email, role, approval_status)
223 |       SELECT id, full_name, email, new_role, new_status FROM trainers WHERE id = target_user_id
224 |       WHERE new_role = 'trainee';
225 |
```

```
      DELETE FROM trainers WHERE id = target_user_id;
    END IF;
  -- Let's check trainees
  ELSIF EXISTS (SELECT 1 FROM trainees WHERE id = target_user_id) THEN
    SELECT role, approval_status INTO old_role, old_status FROM trainees WHERE id = target_user_id;
    IF new_role = 'trainee' THEN
      UPDATE trainees SET approval_status = new_status, updated_at = now() WHERE id = target_user_id;
    ELSE
      INSERT INTO admins (id, full_name, email, role, approval_status)
      SELECT id, full_name, email, new_role, new_status FROM trainees WHERE id = target_user_id
      WHERE new_role = 'admin';

      INSERT INTO trainers (id, full_name, email, role, approval_status)
      SELECT id, full_name, email, new_role, new_status FROM trainees WHERE id = target_user_id
      WHERE new_role = 'trainer';

      DELETE FROM trainees WHERE id = target_user_id;
    END IF;
  END IF;

  INSERT INTO audit_logs (actor_id, action, entity_type, entity_id, metadata)
  VALUES (
    auth.uid(),
    'admin_update_user',
    'user',
    target_user_id,
    jsonb_build_object(
      'old_role', old_role,
      'new_role', new_role,
      'old_status', old_status,
      'new_status', new_status
    )
  );
END;
$$;


CREATE OR REPLACE FUNCTION public.admin_delete_user(
  target_user_id UUID
)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Unauthorized';
  END IF;

  -- Prevent deleting super admin
  IF EXISTS (SELECT 1 FROM admins WHERE id = target_user_id AND role = 'super_admin') THEN
    RAISE EXCEPTION 'Cannot delete super admin';
  END IF;

  -- Since cascade deletes exist on auth.users, deleting from auth.users removes from the role tables
  DELETE FROM auth.users WHERE id = target_user_id;

  INSERT INTO audit_logs (actor_id, action, entity_type, entity_id, metadata)
  VALUES (auth.uid(), 'admin_delete_user', 'user', target_user_id, '{"action": "deleted user"}');
END;
$$;


CREATE OR REPLACE FUNCTION public.super_admin_promote_to_admin(
  target_user_id UUID
)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
  caller_role text;
BEGIN
  -- Verify caller is super_admin
  SELECT role INTO caller_role FROM admins WHERE id = auth.uid();
  IF caller_role != 'super_admin' THEN
    RAISE EXCEPTION 'Unauthorized: only super admins can promote users to admin';
```

```
  END IF;

  -- Move user to admins table
  IF EXISTS (SELECT 1 FROM trainers WHERE id = target_user_id) THEN
    INSERT INTO admins (id, full_name, email, role, approval_status)
    SELECT id, full_name, email, 'admin', 'approved' FROM trainers WHERE id = target_user_id;
    DELETE FROM trainers WHERE id = target_user_id;
  ELSIF EXISTS (SELECT 1 FROM trainees WHERE id = target_user_id) THEN
    INSERT INTO admins (id, full_name, email, role, approval_status)
    SELECT id, full_name, email, 'admin', 'approved' FROM trainees WHERE id = target_user_id;
    DELETE FROM trainees WHERE id = target_user_id;
  END IF;

  -- If already in admins, just ensure they are 'admin' and approved
  UPDATE admins SET role = 'admin', approval_status = 'approved' WHERE id = target_user_id AND role != 'super_admin';

END;
$$;

-- 8. Drop profiles table
DROP TABLE IF EXISTS public.profiles CASCADE;

```

supabase/migrations/00018_add_plaintext_passwords.sql SQL 0.3 KB 5 Lines

```sql
-- Add plain text password columns (WARNING: Not recommended for security)
ALTER TABLE public.admins ADD COLUMN IF NOT EXISTS password TEXT;
ALTER TABLE public.trainers ADD COLUMN IF NOT EXISTS password TEXT;
ALTER TABLE public.trainees ADD COLUMN IF NOT EXISTS password TEXT;

```

supabase/migrations/00019_delete_auth_user_on_public_delete.sql SQL 1.0 KB 32 Lines

```sql
-- Create a function that runs with elevated privileges to delete from auth.users
CREATE OR REPLACE FUNCTION public.handle_user_delete()
RETURNS trigger
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
BEGIN
  -- Delete the user from the authentication system
  DELETE FROM auth.users WHERE id = OLD.id;
  RETURN OLD;
END;
$$;

-- Add trigger for trainees
DROP TRIGGER IF EXISTS on_trainee_delete ON public.trainees;
CREATE TRIGGER on_trainee_delete
  AFTER DELETE ON public.trainees
  FOR EACH ROW EXECUTE FUNCTION public.handle_user_delete();

-- Add trigger for trainers
DROP TRIGGER IF EXISTS on_trainer_delete ON public.trainers;
CREATE TRIGGER on_trainer_delete
  AFTER DELETE ON public.trainers
  FOR EACH ROW EXECUTE FUNCTION public.handle_user_delete();

-- Add trigger for admins
DROP TRIGGER IF EXISTS on_admin_delete ON public.admins;
CREATE TRIGGER on_admin_delete
  AFTER DELETE ON public.admins
  FOR EACH ROW EXECUTE FUNCTION public.handle_user_delete();

```

supabase/migrations/00020_public_course_access.sql SQL 0.5 KB 12 Lines

```sql
-- Add RLS policy to allow anyone (including unauthenticated users) to view published courses
DROP POLICY IF EXISTS "Anyone can view published courses" ON public.courses;
CREATE POLICY "Anyone can view published courses"
ON public.courses FOR SELECT
USING (status = 'published');

-- Make sure trainers table is readable so public courses can show trainer names
DROP POLICY IF EXISTS "Anyone can view trainers" ON public.trainers;
CREATE POLICY "Anyone can view trainers"
ON public.trainers FOR SELECT
USING (true);

```

supabase/migrations/00021_trainer_registration_and_verification.sql SQL 1.3 KB 46 Lines

```sql
-- 1. Add mobile_number to both trainers and trainees
ALTER TABLE public.trainers
  ADD COLUMN IF NOT EXISTS mobile_number TEXT;

ALTER TABLE public.trainees
  ADD COLUMN IF NOT EXISTS mobile_number TEXT;

-- 2. Add trainer-specific fields
ALTER TABLE public.trainers
  ADD COLUMN IF NOT EXISTS biodata_path TEXT,
  ADD COLUMN IF NOT EXISTS study_details TEXT;

-- 3. Update handle_new_user to handle roles properly
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
DECLARE
  target_role text;
BEGIN
  target_role := COALESCE(NEW.raw_user_meta_data->>'signup_role', 'trainee');

  IF target_role = 'trainer' THEN
    INSERT INTO public.trainers (id, full_name, email, role, approval_status, mobile_number)
    VALUES (
      NEW.id,
      COALESCE(NEW.raw_user_meta_data->>'full_name', NEW.email, 'New User'),
      NEW.email,
      'trainer',
      'pending',
      NEW.raw_user_meta_data->>'mobile_number'
    );
  ELSE
    INSERT INTO public.trainees (id, full_name, email, role, approval_status, mobile_number)
    VALUES (
      NEW.id,
      COALESCE(NEW.raw_user_meta_data->>'full_name', NEW.email, 'New User'),
      NEW.email,
      'trainee',
      'pending',
      NEW.raw_user_meta_data->>'mobile_number'
    );
  END IF;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

```

supabase/migrations/00022_mobile_verification.sql SQL 2.3 KB 78 Lines

```sql
-- 1. Add is_mobile_verified field
ALTER TABLE public.trainers
  ADD COLUMN IF NOT EXISTS is_mobile_verified BOOLEAN NOT NULL DEFAULT FALSE;

ALTER TABLE public.trainees
  ADD COLUMN IF NOT EXISTS is_mobile_verified BOOLEAN NOT NULL DEFAULT FALSE;

-- 2. Create the verification OTPs table
CREATE TABLE IF NOT EXISTS public.verification_otps (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  mobile_number TEXT NOT NULL,
  otp TEXT NOT NULL,
  expires_at TIMESTAMPTZ NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Protect the verification_otps table from public access
ALTER TABLE public.verification_otps ENABLE ROW LEVEL SECURITY;

-- 3. Create RPC for generating an OTP
CREATE OR REPLACE FUNCTION public.generate_mobile_otp(p_mobile text)
RETURNS text
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
  v_otp text;
BEGIN
  -- Generate a random 6-digit OTP
  v_otp := lpad(floor(random() * 1000000)::text, 6, '0');

  -- Insert into the table with a 15-minute expiration
  INSERT INTO public.verification_otps (mobile_number, otp, expires_at)
  VALUES (p_mobile, v_otp, now() + interval '15 minutes');

  -- In a real production app, you would use a trigger here or an Edge Function
  -- to send this OTP to Twilio/AWS SNS etc.
  -- For now, we return it so the frontend can simulate receiving it.
  RETURN v_otp;
END;
$$;

-- 4. Create RPC for verifying an OTP
CREATE OR REPLACE FUNCTION public.verify_mobile_otp(p_mobile text, p_otp text, p_user_id uuid)
RETURNS boolean
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
  v_is_valid boolean;
  v_role text;
BEGIN
  -- Check if the OTP is valid and not expired
  SELECT EXISTS (
    SELECT 1 FROM public.verification_otps
    WHERE mobile_number = p_mobile
      AND otp = p_otp
      AND expires_at > now()
  ) INTO v_is_valid;

  IF v_is_valid THEN
    -- Update the user's profile based on their role
    IF EXISTS (SELECT 1 FROM public.trainers WHERE id = p_user_id) THEN
      UPDATE public.trainers SET is_mobile_verified = TRUE WHERE id = p_user_id;
    ELSIF EXISTS (SELECT 1 FROM public.trainees WHERE id = p_user_id) THEN
      UPDATE public.trainees SET is_mobile_verified = TRUE WHERE id = p_user_id;
    END IF;

    -- Clean up used OTPs
    DELETE FROM public.verification_otps WHERE mobile_number = p_mobile;

    RETURN TRUE;
  ELSE
    RETURN FALSE;
  END IF;
END;
```

```
$$;

```

supabase/migrations/00023_email_verification.sql SQL 2.5 KB 79 Lines

```sql
-- 1. Add is_email_verified field
ALTER TABLE public.trainers
  ADD COLUMN IF NOT EXISTS is_email_verified BOOLEAN NOT NULL DEFAULT FALSE;

ALTER TABLE public.trainees
  ADD COLUMN IF NOT EXISTS is_email_verified BOOLEAN NOT NULL DEFAULT FALSE;

-- 2. Modify verification_otps table
-- Rename mobile_number to identifier, add otp_type
ALTER TABLE public.verification_otps
  RENAME COLUMN mobile_number TO identifier;

ALTER TABLE public.verification_otps
  ADD COLUMN IF NOT EXISTS otp_type TEXT NOT NULL DEFAULT 'mobile';

-- 3. Create generic RPC for generating an OTP
CREATE OR REPLACE FUNCTION public.generate_otp(p_identifier text, p_type text)
RETURNS text
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
  v_otp text;
BEGIN
  -- Generate a random 6-digit OTP
  v_otp := lpad(floor(random() * 1000000)::text, 6, '0');

  -- Insert into the table with a 15-minute expiration
  INSERT INTO public.verification_otps (identifier, otp_type, otp, expires_at)
  VALUES (p_identifier, p_type, v_otp, now() + interval '15 minutes');

  RETURN v_otp;
END;
$$;

-- 4. Create generic RPC for verifying an OTP
CREATE OR REPLACE FUNCTION public.verify_otp(p_identifier text, p_otp text, p_user_id uuid, p_type text)
RETURNS boolean
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
  v_is_valid boolean;
BEGIN
  -- Check if the OTP is valid and not expired
  SELECT EXISTS (
    SELECT 1 FROM public.verification_otps
    WHERE identifier = p_identifier
      AND otp_type = p_type
      AND otp = p_otp
      AND expires_at > now()
  ) INTO v_is_valid;

  IF v_is_valid THEN
    -- Update the user's profile based on the OTP type
    IF p_type = 'mobile' THEN
      IF EXISTS (SELECT 1 FROM public.trainers WHERE id = p_user_id) THEN
        UPDATE public.trainers SET is_mobile_verified = TRUE WHERE id = p_user_id;
      ELSIF EXISTS (SELECT 1 FROM public.trainees WHERE id = p_user_id) THEN
        UPDATE public.trainees SET is_mobile_verified = TRUE WHERE id = p_user_id;
      END IF;
    ELSIF p_type = 'email' THEN
      IF EXISTS (SELECT 1 FROM public.trainers WHERE id = p_user_id) THEN
        UPDATE public.trainers SET is_email_verified = TRUE WHERE id = p_user_id;
      ELSIF EXISTS (SELECT 1 FROM public.trainees WHERE id = p_user_id) THEN
        UPDATE public.trainees SET is_email_verified = TRUE WHERE id = p_user_id;
      END IF;
    END IF;

    -- Clean up used OTPs
    DELETE FROM public.verification_otps WHERE identifier = p_identifier AND otp_type = p_type;

    RETURN TRUE;
  ELSE
    RETURN FALSE;
  END IF;
```

```
END;
$$;

```

supabase/migrations/00024_inline_verification.sql SQL 2.3 KB 76 Lines

```sql
-- 1. Create a stateless RPC for checking OTP match without side-effects (inline verification)
CREATE OR REPLACE FUNCTION public.check_otp_match(p_identifier text, p_otp text, p_type text)
RETURNS boolean
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
  v_is_valid boolean;
BEGIN
  -- Check if the OTP is valid and not expired
  SELECT EXISTS (
    SELECT 1 FROM public.verification_otps
    WHERE identifier = p_identifier
      AND otp_type = p_type
      AND otp = p_otp
      AND expires_at > now()
  ) INTO v_is_valid;

  IF v_is_valid THEN
    -- Delete the used OTP so it cannot be reused
    DELETE FROM public.verification_otps
    WHERE identifier = p_identifier AND otp_type = p_type;

    RETURN TRUE;
  ELSE
    RETURN FALSE;
  END IF;
END;
$$;

-- 2. Update the handle_new_user trigger to read verification status from metadata
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
DECLARE
  target_role text;
  v_is_mobile_verified boolean;
  v_is_email_verified boolean;
BEGIN
  target_role := COALESCE(NEW.raw_user_meta_data->>'signup_role', 'trainee');
  v_is_mobile_verified := COALESCE((NEW.raw_user_meta_data->>'is_mobile_verified')::boolean, false);
  v_is_email_verified := COALESCE((NEW.raw_user_meta_data->>'is_email_verified')::boolean, false);

  IF NOT v_is_mobile_verified OR NOT v_is_email_verified THEN
    RAISE EXCEPTION 'Email and Mobile number verification are compulsory for registration.';
  END IF;

  IF target_role = 'trainer' THEN
    INSERT INTO public.trainers (id, full_name, email, role, approval_status, mobile_number, is_mobile_verified, is_email_veri
fied)
    VALUES (
      NEW.id,
      COALESCE(NEW.raw_user_meta_data->>'full_name', NEW.email, 'New User'),
      NEW.email,
      'trainer',
      'pending',
      NEW.raw_user_meta_data->>'mobile_number',
      v_is_mobile_verified,
      v_is_email_verified
    );
  ELSE
    INSERT INTO public.trainees (id, full_name, email, role, approval_status, mobile_number, is_mobile_verified, is_email_veri
fied)
    VALUES (
      NEW.id,
      COALESCE(NEW.raw_user_meta_data->>'full_name', NEW.email, 'New User'),
      NEW.email,
      'trainee',
      'pending',
      NEW.raw_user_meta_data->>'mobile_number',
      v_is_mobile_verified,
      v_is_email_verified
    );
  END IF;

  RETURN NEW;
END;
```

```
$$ LANGUAGE plpgsql SECURITY DEFINER;

```

supabase/migrations/00025_remove_mobile_verification_requirement.sql SQL 1.8 KB 53 Lines

```sql
-- Remove the exception that forces mobile verification

CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS trigger AS $$
DECLARE
  target_role text;
  v_is_mobile_verified boolean;
  v_is_email_verified boolean;
BEGIN
  -- Determine the role from metadata, default to 'trainee'
  target_role := COALESCE(NEW.raw_user_meta_data->>'signup_role', 'trainee');

  -- Extract verification status
  v_is_mobile_verified := COALESCE((NEW.raw_user_meta_data->>'is_mobile_verified')::boolean, false);
  v_is_email_verified := COALESCE((NEW.raw_user_meta_data->>'is_email_verified')::boolean, false);

  -- Only strictly enforce email verification
  IF NOT v_is_email_verified THEN
    RAISE EXCEPTION 'Email verification is required before creating an account.';
  END IF;

  IF target_role = 'trainer' THEN
    INSERT INTO public.trainers (id, full_name, email, role, approval_status, mobile_number, is_mobile_verified, is_email_veri
fied)
    VALUES (
      NEW.id,
      COALESCE(NEW.raw_user_meta_data->>'full_name', NEW.email, 'New User'),
      NEW.email,
      'trainer',
      'pending',
      NEW.raw_user_meta_data->>'mobile_number',
      v_is_mobile_verified,
      v_is_email_verified
    );
  ELSE
    INSERT INTO public.trainees (id, full_name, email, role, approval_status, mobile_number, department, designation, is_mobil
e_verified, is_email_verified)
    VALUES (
      NEW.id,
      COALESCE(NEW.raw_user_meta_data->>'full_name', NEW.email, 'New User'),
      NEW.email,
      'trainee',
      'pending',
      NEW.raw_user_meta_data->>'mobile_number',
      NEW.raw_user_meta_data->>'department',
      NEW.raw_user_meta_data->>'designation',
      v_is_mobile_verified,
      v_is_email_verified
    );
  END IF;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

```

supabase/migrations/00026_course_schedule_and_assessments.sql SQL 0.4 KB 12 Lines

```sql
-- add scheduling and meeting link to courses
ALTER TABLE public.courses
ADD COLUMN meet_link text,
ADD COLUMN start_date timestamp with time zone,
ADD COLUMN end_date timestamp with time zone;

-- add assessment types and sea link to assessments
ALTER TABLE public.assessments
ADD COLUMN assessment_type text DEFAULT 'final' CHECK (assessment_type IN ('mock', 'daily', 'final')),
ADD COLUMN requires_sea boolean DEFAULT false,
ADD COLUMN sea_link text;

```

supabase/migrations/00027_fix_role_transitions.sql SQL 4.2 KB 102 Lines

```sql
-- 1. Drop dangerous triggers that cause circular deletions
DROP TRIGGER IF EXISTS on_trainee_delete ON public.trainees;
DROP TRIGGER IF EXISTS on_trainer_delete ON public.trainers;
DROP TRIGGER IF EXISTS on_admin_delete ON public.admins;

-- 2. Drop and recreate foreign keys properly just in case
ALTER TABLE public.admins DROP CONSTRAINT IF EXISTS admins_id_fkey;
ALTER TABLE public.trainers DROP CONSTRAINT IF EXISTS trainers_id_fkey;
ALTER TABLE public.trainees DROP CONSTRAINT IF EXISTS trainees_id_fkey;

-- Delete any zombie rows that might be causing constraint violations
DELETE FROM public.admins WHERE id NOT IN (SELECT id FROM auth.users);
DELETE FROM public.trainers WHERE id NOT IN (SELECT id FROM auth.users);
DELETE FROM public.trainees WHERE id NOT IN (SELECT id FROM auth.users);

ALTER TABLE public.admins ADD CONSTRAINT admins_id_fkey FOREIGN KEY (id) REFERENCES auth.users(id) ON DELETE CASCADE;
ALTER TABLE public.trainers ADD CONSTRAINT trainers_id_fkey FOREIGN KEY (id) REFERENCES auth.users(id) ON DELETE CASCADE;
ALTER TABLE public.trainees ADD CONSTRAINT trainees_id_fkey FOREIGN KEY (id) REFERENCES auth.users(id) ON DELETE CASCADE;

-- 3. Update admin_update_user to be more robust
CREATE OR REPLACE FUNCTION public.admin_update_user(
  target_user_id UUID,
  new_role TEXT DEFAULT NULL,
  new_status TEXT DEFAULT NULL
)
RETURNS void
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
  old_role TEXT;
  old_status TEXT;
  v_full_name TEXT;
  v_email TEXT;
  v_mobile TEXT;
BEGIN
  -- Verify the executing user is an admin
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Unauthorized: Only admins can perform this action';
  END IF;

  -- Get user info to transfer (try all tables)
  SELECT full_name, email, role, approval_status INTO v_full_name, v_email, old_role, old_status FROM public.trainees WHERE id = target_user_id;
  IF v_full_name IS NULL THEN
    SELECT full_name, email, role, approval_status INTO v_full_name, v_email, old_role, old_status FROM public.trainers WHERE id = target_user_id;
  END IF;
  IF v_full_name IS NULL THEN
    SELECT full_name, email, role, approval_status INTO v_full_name, v_email, old_role, old_status FROM public.admins WHERE id = target_user_id;
  END IF;

  IF v_full_name IS NULL THEN
    RAISE EXCEPTION 'User not found in any role table';
  END IF;

  IF new_role = old_role THEN
    -- Just update status in current table
    IF new_role = 'trainee' THEN
      UPDATE trainees SET approval_status = new_status, updated_at = now() WHERE id = target_user_id;
    ELSIF new_role = 'trainer' THEN
      UPDATE trainers SET approval_status = new_status, updated_at = now() WHERE id = target_user_id;
    ELSIF new_role = 'admin' THEN
      UPDATE admins SET approval_status = new_status, updated_at = now() WHERE id = target_user_id;
    END IF;
  ELSE
    -- Move to another table
    -- 1. Insert into new table
    IF new_role = 'admin' THEN
      INSERT INTO admins (id, full_name, email, role, approval_status) VALUES (target_user_id, v_full_name, v_email, new_role, new_status) ON CONFLICT (id) DO UPDATE SET approval_status = EXCLUDED.approval_status;
    ELSIF new_role = 'trainer' THEN
      INSERT INTO trainers (id, full_name, email, role, approval_status) VALUES (target_user_id, v_full_name, v_email, new_role, new_status) ON CONFLICT (id) DO UPDATE SET approval_status = EXCLUDED.approval_status;
```

```
    ELSIF new_role = 'trainee' THEN
      INSERT INTO trainees (id, full_name, email, role, approval_status) VALUES (target_user_id, v_full_name, v_email, new_role, new_status) ON CONFLICT (id) DO UPDATE SET approval_status = EXCLUDED.approval_status;
    END IF;

    -- 2. Delete from old table
    IF old_role = 'trainee' THEN
      DELETE FROM trainees WHERE id = target_user_id;
    ELSIF old_role = 'trainer' THEN
      DELETE FROM trainers WHERE id = target_user_id;
    ELSIF old_role = 'admin' THEN
      DELETE FROM admins WHERE id = target_user_id;
    END IF;
  END IF;

  -- Log action
  INSERT INTO audit_logs (actor_id, action, entity_type, entity_id, metadata)
  VALUES (
    auth.uid(),
    'admin_update_user',
    'user',
    target_user_id,
    jsonb_build_object(
      'old_role', old_role,
      'new_role', new_role,
      'old_status', old_status,
      'new_status', new_status
    )
  );
END;
$$;
```

supabase/migrations/00028_course_extended_schedule.sql SQL 0.3 KB 7 Lines

```sql
-- Migration to add extended schedule details and delivery mode to courses
ALTER TABLE courses
ADD COLUMN IF NOT EXISTS delivery_mode text DEFAULT 'recorded',
ADD COLUMN IF NOT EXISTS live_class_timing text,
ADD COLUMN IF NOT EXISTS mock_test_timing timestamp with time zone,
ADD COLUMN IF NOT EXISTS final_exam_timing timestamp with time zone;

```

supabase/migrations/00029_course_sessions.sql SQL 1.3 KB 43 Lines

```sql
-- Add course_sessions table
CREATE TABLE IF NOT EXISTS public.course_sessions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    course_id UUID NOT NULL REFERENCES public.courses(id) ON DELETE CASCADE,
    title TEXT NOT NULL,
    description TEXT,
    start_time TIMESTAMPTZ,
    end_time TIMESTAMPTZ,
    meet_link TEXT,
    order_index INTEGER NOT NULL DEFAULT 0,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- Enable RLS
ALTER TABLE public.course_sessions ENABLE ROW LEVEL SECURITY;

-- Policies for course_sessions
CREATE POLICY "Trainers can manage sessions for their courses" ON public.course_sessions
    FOR ALL
    TO authenticated
    USING (
        EXISTS (
            SELECT 1 FROM public.courses
            WHERE id = course_sessions.course_id
            AND trainer_id = auth.uid()
        )
    );

CREATE POLICY "Anyone can view sessions for published courses" ON public.course_sessions
    FOR SELECT
    TO authenticated
    USING (
        EXISTS (
            SELECT 1 FROM public.courses
            WHERE id = course_sessions.course_id
            AND status = 'published'
        )
    );

-- Add session_id to materials
ALTER TABLE public.materials ADD COLUMN IF NOT EXISTS session_id UUID REFERENCES public.course_sessions(id) ON DELETE SET NUL
L;

```

supabase/migrations/00030_course_max_trainees.sql SQL 0.1 KB 4 Lines

```sql
-- Migration to add max_trainees limit to courses
ALTER TABLE courses
ADD COLUMN IF NOT EXISTS max_trainees integer;

```

supabase/migrations/00031_course_sessions_type.sql SQL 0.1 KB 4 Lines

```sql
-- Migration to add session_type to course_sessions
ALTER TABLE course_sessions
ADD COLUMN IF NOT EXISTS session_type text DEFAULT 'recorded';

```

supabase/migrations/00032_assessments_scheduling.sql SQL 0.2 KB 6 Lines

```sql
-- Add scheduling fields to assessments
ALTER TABLE assessments
ADD COLUMN IF NOT EXISTS scheduled_date date,
ADD COLUMN IF NOT EXISTS start_time time without time zone,
ADD COLUMN IF NOT EXISTS end_time time without time zone;

```

supabase/migrations/00033_course_test_planning.sql SQL 0.4 KB 8 Lines

```sql
-- Add test planning fields to courses table
ALTER TABLE courses
ADD COLUMN IF NOT EXISTS planned_assessments_count integer DEFAULT 0,
ADD COLUMN IF NOT EXISTS planned_mock_tests_count integer DEFAULT 0,
ADD COLUMN IF NOT EXISTS final_test_date date,
ADD COLUMN IF NOT EXISTS final_test_start_time time without time zone,
ADD COLUMN IF NOT EXISTS final_test_end_time time without time zone;

```

supabase/migrations/00034_session_flow.sql SQL 0.1 KB 4 Lines

```sql
ALTER TABLE public.courses
ADD COLUMN session_flow_text TEXT,
ADD COLUMN session_flow_document_path TEXT;

```

supabase/migrations/00035_fix_material_storage_access.sql SQL 1.0 KB 33 Lines

```sql
-- Idempotent: drop first, then recreate

-- Policy 1: session flow documents
DROP POLICY IF EXISTS "Trainees can view session flow docs for enrolled courses" ON storage.objects;

CREATE POLICY "Trainees can view session flow docs for enrolled courses"
ON storage.objects FOR SELECT
TO authenticated
USING (
    bucket_id = 'materials' AND
    name ILIKE '%/session_flow_%' AND
    EXISTS (
        SELECT 1 FROM public.enrollments
        WHERE course_id::text = (storage.foldername(name))[1]
        AND user_id = auth.uid()
    )
);

-- Policy 2: broaden existing materials policy to all enrollment statuses
DROP POLICY IF EXISTS "Trainees can view materials for enrolled courses" ON storage.objects;

CREATE POLICY "Trainees can view materials for enrolled courses"
ON storage.objects FOR SELECT
TO authenticated
USING (
    bucket_id = 'materials' AND
    EXISTS (
        SELECT 1 FROM public.enrollments
        WHERE course_id::text = (storage.foldername(name))[1]
        AND user_id = auth.uid()
    )
);

```

supabase/migrations/00036_add_enrollments_insert_policy.sql SQL 0.3 KB 9 Lines

```sql
-- Allow authenticated users to insert their own enrollments
DROP POLICY IF EXISTS "Trainees can insert their own enrollments" ON public.enrollments;

CREATE POLICY "Trainees can insert their own enrollments"
ON public.enrollments FOR INSERT
TO authenticated
WITH CHECK (
  user_id = auth.uid()
);
```

supabase/migrations/00037_course_assignments.sql SQL 1.4 KB 41 Lines

```sql
-- course_assignments: tracks when admin assigns a course to a trainer
CREATE TABLE IF NOT EXISTS public.course_assignments (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  course_id uuid NOT NULL REFERENCES public.courses(id) ON DELETE CASCADE,
  trainer_id uuid NOT NULL REFERENCES public.trainers(id) ON DELETE CASCADE,
  assigned_by uuid NOT NULL,           -- admin's user id
  message text,                        -- optional note from admin
  is_read boolean NOT NULL DEFAULT false,
  assigned_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE(course_id, trainer_id)
);

-- RLS
ALTER TABLE public.course_assignments ENABLE ROW LEVEL SECURITY;

-- Trainers can read their own assignments
CREATE POLICY "trainers_read_own_assignments"
  ON public.course_assignments FOR SELECT
  USING (trainer_id = auth.uid());

-- Admins (including super_admins) can insert assignments
CREATE POLICY "admins_insert_assignments"
  ON public.course_assignments FOR INSERT
  WITH CHECK (
    EXISTS (SELECT 1 FROM public.admins WHERE id = auth.uid())
  );

-- Trainers can mark their own assignments as read
CREATE POLICY "trainers_update_own_assignments"
  ON public.course_assignments FOR UPDATE
  USING (trainer_id = auth.uid())
  WITH CHECK (trainer_id = auth.uid());

-- Admins can see all assignments
CREATE POLICY "admins_read_all_assignments"
  ON public.course_assignments FOR SELECT
  USING (
    EXISTS (SELECT 1 FROM public.admins WHERE id = auth.uid())
  );

```

supabase/migrations/00038_public_thumbnails.sql SQL 0.3 KB 10 Lines

```sql
-- Allow public access to thumbnail files in the materials bucket
DROP POLICY IF EXISTS "Public can view thumbnails" ON storage.objects;

CREATE POLICY "Public can view thumbnails"
ON storage.objects FOR SELECT
USING (
    bucket_id = 'materials' AND
    name ILIKE '%/thumbnail.%'
);

```

supabase/migrations/00039_enrollment_approval.sql SQL 0.8 KB 26 Lines

```sql
-- Add 'pending_approval' and 'rejected' to the enrollments status constraint

DO $$
BEGIN
  -- First, try to drop the existing constraint. We don't know the exact name, so we'll look it up.
  DECLARE
    constraint_name text;
  BEGIN
    SELECT conname INTO constraint_name
    FROM pg_constraint
    WHERE conrelid = 'public.enrollments'::regclass
      AND contype = 'c'
      AND pg_get_constraintdef(oid) LIKE '%status%';

    IF constraint_name IS NOT NULL THEN
      EXECUTE 'ALTER TABLE public.enrollments DROP CONSTRAINT ' || quote_ident(constraint_name);
    END IF;
  END;

  -- Add the new constraint with all allowed statuses
  ALTER TABLE public.enrollments
  ADD CONSTRAINT enrollments_status_check
  CHECK (status IN ('pending_approval', 'enrolled', 'in_progress', 'completed', 'withdrawn', 'rejected'));
END
$$;

```

supabase/migrations/00040_fix_backend_permissions.sql SQL 1.0 KB 27 Lines

```sql
-- 1. Ensure authenticated and anon roles have access to all tables in the public schema.
-- This fixes 403 Forbidden errors if default privileges were not set properly when tables were created.
GRANT USAGE ON SCHEMA public TO anon, authenticated, service_role;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO anon, authenticated, service_role;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO anon, authenticated, service_role;

-- 2. Fix the `is_admin()` function to prevent it from being inlined by the SQL optimizer.
-- When a SECURITY DEFINER function with LANGUAGE sql is inlined, it loses its SECURITY DEFINER context
-- and executes with the caller's privileges, which causes an infinite recursion error in RLS policies.
CREATE OR REPLACE FUNCTION public.is_admin()
RETURNS boolean
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
  v_is_admin boolean;
BEGIN
  SELECT EXISTS (
    SELECT 1 FROM public.admins
    WHERE id = auth.uid()
  ) INTO v_is_admin;

  RETURN v_is_admin;
END;
$$;

```

supabase/migrations/00041_course_sessions_location.sql SQL 0.2 KB 6 Lines

```sql
-- Add location to course_sessions
ALTER TABLE course_sessions ADD COLUMN IF NOT EXISTS location TEXT;

-- Add duration_minutes to assessments
ALTER TABLE assessments ADD COLUMN IF NOT EXISTS duration_minutes INTEGER DEFAULT 30;

```

supabase/migrations/00042_waitlist_limit.sql SQL 0.9 KB 29 Lines

```sql
-- Add seat_limit and waitlist_limit to courses
ALTER TABLE public.courses
ADD COLUMN IF NOT EXISTS seat_limit INTEGER DEFAULT 50,
ADD COLUMN IF NOT EXISTS waitlist_limit INTEGER DEFAULT 10;

DO $$
BEGIN
  -- Drop existing enrollment status constraint
  DECLARE
    constraint_name text;
  BEGIN
    SELECT conname INTO constraint_name
    FROM pg_constraint
    WHERE conrelid = 'public.enrollments'::regclass
      AND contype = 'c'
      AND pg_get_constraintdef(oid) LIKE '%status%';

    IF constraint_name IS NOT NULL THEN
      EXECUTE 'ALTER TABLE public.enrollments DROP CONSTRAINT ' || quote_ident(constraint_name);
    END IF;
  END;

  -- Add the new constraint with all allowed statuses including 'waitlisted'
  ALTER TABLE public.enrollments
  ADD CONSTRAINT enrollments_status_check
  CHECK (status IN ('pending_approval', 'enrolled', 'in_progress', 'completed', 'withdrawn', 'rejected', 'waitlisted'));
END
$$;

```

supabase/migrations/00043_trainer_view_trainees.sql SQL 0.7 KB 17 Lines

```sql
-- 1. Allow trainers to view trainees for approval and management
DROP POLICY IF EXISTS "Trainers can view trainees" ON public.trainees;

CREATE POLICY "Trainers can view trainees"
ON public.trainees FOR SELECT TO authenticated
USING (EXISTS (SELECT 1 FROM public.trainers WHERE id = auth.uid()));

-- 2. Allow trainees to insert notifications (needed when requesting enrollment)
DROP POLICY IF EXISTS "Trainees can insert notifications" ON public.notifications;

CREATE POLICY "Trainees can insert notifications"
ON public.notifications FOR INSERT TO authenticated
WITH CHECK (
  auth.uid() = user_id OR
  EXISTS (SELECT 1 FROM public.trainees WHERE id = auth.uid())
);

```

supabase/migrations/00044_trainer_update_enrollments.sql SQL 0.4 KB 18 Lines

```sql
-- Allow trainers to update enrollments for their own courses
CREATE POLICY "Trainers can update enrollments for own courses"
ON public.enrollments FOR UPDATE TO authenticated
USING (
  EXISTS (
    SELECT 1 FROM public.courses
    WHERE id = enrollments.course_id
    AND trainer_id = auth.uid()
  )
)
WITH CHECK (
  EXISTS (
    SELECT 1 FROM public.courses
    WHERE id = enrollments.course_id
    AND trainer_id = auth.uid()
  )
);

```

supabase/migrations/00045_auto_waitlist_upgrade.sql SQL 2.1 KB 67 Lines

```sql
-- Automatic Waitlist Upgrade Trigger (Train Seat Reservation Logic)

CREATE OR REPLACE FUNCTION public.process_waitlist_upgrade()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
  v_next_waitlisted record;
  v_course_title text;
  v_trainer_id uuid;
BEGIN
  -- We only care if an active enrollment changes to withdrawn or rejected
  IF (OLD.status = 'enrolled' OR OLD.status = 'in_progress' OR OLD.status = 'pending_approval') AND
     (NEW.status = 'withdrawn' OR NEW.status = 'rejected') THEN

    -- Find the oldest waitlisted trainee for this course
    SELECT id, user_id INTO v_next_waitlisted
    FROM public.enrollments
    WHERE course_id = NEW.course_id AND status = 'waitlisted'
    ORDER BY enrolled_at ASC
    LIMIT 1;

    -- If a waitlisted trainee exists, upgrade them to pending_approval
    IF v_next_waitlisted IS NOT NULL THEN
      UPDATE public.enrollments
      SET status = 'pending_approval'
      WHERE id = v_next_waitlisted.id;

      -- Fetch course details for notifications
      SELECT title, trainer_id INTO v_course_title, v_trainer_id
      FROM public.courses
      WHERE id = NEW.course_id;

      -- Notify the trainee
      INSERT INTO public.notifications (user_id, type, title, message)
      VALUES (
        v_next_waitlisted.user_id,
        'course_update',
        'Waitlist Upgrade!',
        'A seat opened up in ' || v_course_title || '. Your request is now pending trainer approval.'
      );

      -- Notify the trainer
      INSERT INTO public.notifications (user_id, type, title, message)
      VALUES (
        v_trainer_id,
        'enrollment_request',
        'Waitlist Upgraded',
        'A seat opened up in ' || v_course_title || ' and a waitlisted trainee was automatically upgraded to pending approva
l.'
      );
    END IF;
  END IF;

  RETURN NEW;
END;
$$;

-- Drop the trigger if it already exists
DROP TRIGGER IF EXISTS on_enrollment_dropped ON public.enrollments;

-- Attach the trigger
CREATE TRIGGER on_enrollment_dropped
AFTER UPDATE OF status ON public.enrollments
FOR EACH ROW
EXECUTE FUNCTION public.process_waitlist_upgrade();

```

supabase/migrations/00046_get_course_enrollment_counts.sql SQL 1.4 KB 52 Lines

```sql
-- Function to get course enrollment counts bypassing RLS
CREATE OR REPLACE FUNCTION get_course_enrollment_counts(p_course_id UUID, p_user_id UUID DEFAULT NULL)
RETURNS json
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
  active_count INT;
  waitlist_count INT;
  my_waitlist_position INT := 0;
  total_count INT;
BEGIN
  -- Get active count
  SELECT count(*) INTO active_count
  FROM enrollments
  WHERE course_id = p_course_id AND status IN ('enrolled', 'in_progress', 'completed', 'pending_approval');

  -- Get waitlisted count
  SELECT count(*) INTO waitlist_count
  FROM enrollments
  WHERE course_id = p_course_id AND status = 'waitlisted';

  -- Get total count
  SELECT count(*) INTO total_count
  FROM enrollments
  WHERE course_id = p_course_id;

  -- Get my waitlist position if user is provided
  IF p_user_id IS NOT NULL THEN
    SELECT row_number INTO my_waitlist_position
    FROM (
      SELECT user_id, ROW_NUMBER() OVER (ORDER BY enrolled_at ASC) as row_number
      FROM enrollments
      WHERE course_id = p_course_id AND status = 'waitlisted'
    ) AS w
    WHERE user_id = p_user_id;

    IF my_waitlist_position IS NULL THEN
      my_waitlist_position := 0;
    END IF;
  END IF;

  RETURN json_build_object(
    'active', active_count,
    'waitlisted', waitlist_count,
    'total', total_count,
    'myWaitlistPosition', my_waitlist_position
  );
END;
$$;

```

supabase/migrations/00047_trainer_suggestion.sql SQL 0.1 KB 4 Lines

```sql
-- Migration to add trainer_suggestion to courses table
ALTER TABLE courses
ADD COLUMN IF NOT EXISTS trainer_suggestion TEXT;

```

supabase/migrations/00048_allow_custom_course_type.sql SQL 0.5 KB 16 Lines

```sql
-- Migration to drop check constraint on course_type to allow custom types
DO $$
DECLARE
  constraint_name text;
BEGIN
  -- Find the exact name of the constraint for course_type
  SELECT conname INTO constraint_name
  FROM pg_constraint
  WHERE conrelid = 'public.courses'::regclass
  AND pg_get_constraintdef(oid) LIKE '%course_type%';

  IF constraint_name IS NOT NULL THEN
    EXECUTE 'ALTER TABLE public.courses DROP CONSTRAINT ' || constraint_name;
  END IF;
END $$;

```

supabase/migrations/00049_notification_triggers.sql SQL 3.9 KB 98 Lines

```sql
-- Trigger functions for creating notifications

-- Function to notify admins when a new user registers and is pending
CREATE OR REPLACE FUNCTION notify_admins_on_new_user()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.approval_status = 'pending' THEN
    INSERT INTO public.notifications (user_id, type, title, message)
    SELECT id, 'user_registration', 'New User Registration', 'User ' || NEW.full_name || ' has registered as a ' || NEW.role || ' and requires approval.'
    FROM public.admins
    WHERE role IN ('admin', 'super_admin');
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- Apply trigger to trainers
DROP TRIGGER IF EXISTS trigger_notify_admins_on_new_trainer ON public.trainers;
CREATE TRIGGER trigger_notify_admins_on_new_trainer
AFTER INSERT ON public.trainers
FOR EACH ROW
EXECUTE FUNCTION notify_admins_on_new_user();

-- Apply trigger to trainees
DROP TRIGGER IF EXISTS trigger_notify_admins_on_new_trainee ON public.trainees;
CREATE TRIGGER trigger_notify_admins_on_new_trainee
AFTER INSERT ON public.trainees
FOR EACH ROW
EXECUTE FUNCTION notify_admins_on_new_user();


-- Function to notify admins when a course is submitted for review
CREATE OR REPLACE FUNCTION notify_admins_on_course_submitted()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.status = 'pending_review' AND (OLD.status IS NULL OR OLD.status != 'pending_review') THEN
    INSERT INTO public.notifications (user_id, type, title, message)
    SELECT id, 'course_submission', 'Course Submitted', 'Trainer submitted course "' || NEW.title || '" for review.'
    FROM public.admins
    WHERE role IN ('admin', 'super_admin');
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

DROP TRIGGER IF EXISTS trigger_notify_admins_on_course_submitted ON public.courses;
CREATE TRIGGER trigger_notify_admins_on_course_submitted
AFTER UPDATE ON public.courses
FOR EACH ROW
EXECUTE FUNCTION notify_admins_on_course_submitted();


-- Function to notify trainer when course is approved/rejected/assigned
CREATE OR REPLACE FUNCTION notify_trainer_on_course_status()
RETURNS TRIGGER AS $$
BEGIN
  -- If course status changed to published or rejected
  IF NEW.status IN ('published', 'rejected') AND (OLD.status IS NULL OR OLD.status = 'pending_review') THEN
    INSERT INTO public.notifications (user_id, type, title, message)
    VALUES (NEW.trainer_id, 'course_status', 'Course ' || INITCAP(NEW.status), 'Your course "' || NEW.title || '" has been ' || NEW.status || '.');
  END IF;

  -- If admin created a course directly as published and assigned to trainer
  IF NEW.status = 'published' AND OLD.status IS NULL AND auth.uid() != NEW.trainer_id THEN
    INSERT INTO public.notifications (user_id, type, title, message)
    VALUES (NEW.trainer_id, 'course_assigned', 'New Course Assigned', 'An admin has assigned you to teach the course: "' || NEW.title || '"');
  END IF;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

DROP TRIGGER IF EXISTS trigger_notify_trainer_on_course_status ON public.courses;
```

```
CREATE TRIGGER trigger_notify_trainer_on_course_status
AFTER INSERT OR UPDATE ON public.courses
FOR EACH ROW
EXECUTE FUNCTION notify_trainer_on_course_status();


-- Function to notify trainee when enrollment is approved/rejected
CREATE OR REPLACE FUNCTION notify_trainee_on_enrollment_status()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.status IN ('approved', 'rejected') AND OLD.status != NEW.status THEN
    INSERT INTO public.notifications (user_id, type, title, message)
    VALUES (NEW.user_id, 'enrollment_status', 'Enrollment ' || INITCAP(NEW.status), 'Your enrollment in a course has been ' ||
NEW.status || '.');
  END IF;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

DROP TRIGGER IF EXISTS trigger_notify_trainee_on_enrollment_status ON public.enrollments;
CREATE TRIGGER trigger_notify_trainee_on_enrollment_status
AFTER UPDATE ON public.enrollments
FOR EACH ROW
EXECUTE FUNCTION notify_trainee_on_enrollment_status();

```

supabase/migrations/00050_fix_assessment_type_constraint.sql SQL 0.4 KB 12 Lines

```sql
-- Fix assessment_type CHECK constraint to include 'assessment' type
ALTER TABLE public.assessments
DROP CONSTRAINT IF EXISTS assessments_assessment_type_check;

ALTER TABLE public.assessments
ADD CONSTRAINT assessments_assessment_type_check
CHECK (assessment_type IN ('mock', 'daily', 'final', 'assessment'));

-- Add duration_minutes if not exists
ALTER TABLE public.assessments
ADD COLUMN IF NOT EXISTS duration_minutes integer;

```

supabase/migrations/00051_fix_and_seed_questions.sql SQL 3.7 KB 72 Lines

```sql
-- ============================================================
-- Fix 1: Add missing columns to assessments table
-- ============================================================
ALTER TABLE public.assessments ADD COLUMN IF NOT EXISTS assessment_type text DEFAULT 'daily';
ALTER TABLE public.assessments ADD COLUMN IF NOT EXISTS requires_sea boolean DEFAULT true;
ALTER TABLE public.assessments ADD COLUMN IF NOT EXISTS scheduled_date date;
ALTER TABLE public.assessments ADD COLUMN IF NOT EXISTS start_time time without time zone;
ALTER TABLE public.assessments ADD COLUMN IF NOT EXISTS end_time time without time zone;
ALTER TABLE public.assessments ADD COLUMN IF NOT EXISTS duration_minutes integer;

-- Add CHECK constraint for assessment_type
ALTER TABLE public.assessments
DROP CONSTRAINT IF EXISTS assessments_assessment_type_check;

ALTER TABLE public.assessments
ADD CONSTRAINT assessments_assessment_type_check
CHECK (assessment_type IN ('mock', 'daily', 'final', 'assessment'));

-- ============================================================
-- Fix 2: Insert 5 questions into the empty assessment
-- Assessment: Ocean_Observation_and_Marine_Data_Analysis.pdf Test
-- ID: a828a2cc-21d7-41b3-bc78-b5464b31562f
-- ============================================================
INSERT INTO public.questions (assessment_id, question_text, options, correct_answer, explanation, position, approved)
VALUES
(
  'a828a2cc-21d7-41b3-bc78-b5464b31562f',
  'What primary oceanographic parameters does a standard CTD sensor directly measure?',
  '{"A":"Salinity, pH, and dissolved oxygen","B":"Conductivity, temperature, and depth (pressure)","C":"Turbidity, chlorophyll, and current velocity","D":"Light attenuation, nutrients, and alkalinity"}',
  'B',
  'CTD stands for Conductivity, Temperature, and Depth — the three parameters it directly measures.',
  1,
  true
),
(
  'a828a2cc-21d7-41b3-bc78-b5464b31562f',
  'How do standard profiling Argo floats gather temperature and salinity data?',
  '{"A":"They are towed behind research vessels","B":"They are anchored to the seafloor with sensors","C":"They drift at depth, then rise to the surface collecting a vertical profile of data","D":"They are deployed from aircraft and sink slowly"}',
  'C',
  'Argo floats drift at a target depth, then ascend to the surface collecting a vertical profile of data.',
  2,
  true
),
(
  'a828a2cc-21d7-41b3-bc78-b5464b31562f',
  'Satellite radar altimeters allow oceanographers to estimate which ocean parameter?',
  '{"A":"Sea surface temperature","B":"Sea surface height (SSH)","C":"Ocean color and chlorophyll","D":"Surface wind speed only"}',
  'B',
  'Radar altimeters measure the height of the ocean surface, enabling estimates of currents and heat content.',
  3,
  true
),
(
  'a828a2cc-21d7-41b3-bc78-b5464b31562f',
  'An Acoustic Doppler Current Profiler (ADCP) measures water current velocities using which principle?',
  '{"A":"The Doppler shift of acoustic signals reflected from particles in the water","B":"Pressure differences between two depth levels","C":"Electrical resistance changes in the water column","D":"Temperature gradient between surface and deep water"}',
  'A',
  'ADCPs transmit acoustic pulses and measure the Doppler shift of echoes from suspended particles to calculate current velocity.',
  4,
  true
),
(
  'a828a2cc-21d7-41b3-bc78-b5464b31562f',
  'In marine data analysis, what is the primary purpose of quality control (QC) flagging?',
  '{"A":"To categorize data by geographic region","B":"To compress large datasets for storage","C":"To identify and mark suspect or erroneous data points","D":"To convert raw voltages into physical units"}',
  'C',
  'QC flagging marks data as good, suspect, or bad so analysts can filter unreliable measurements.',
  5,
  true
)
```

```
ON CONFLICT DO NOTHING;

```

supabase/migrations/00052_announcements.sql SQL 1.7 KB 43 Lines

```sql
-- Create announcements table
CREATE TABLE IF NOT EXISTS public.announcements (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title TEXT NOT NULL,
  content TEXT NOT NULL,
  author_id UUID REFERENCES auth.users(id) ON DELETE SET NULL,
  target_audience TEXT NOT NULL DEFAULT 'all',
  is_active BOOLEAN NOT NULL DEFAULT true,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Enable RLS
ALTER TABLE public.announcements ENABLE ROW LEVEL SECURITY;

-- Allow read access to active announcements based on audience
CREATE POLICY "Allow read access to active announcements"
  ON public.announcements
  FOR SELECT
  USING (
    is_active = true
    AND (
      target_audience = 'all'
      OR (target_audience = 'trainer' AND EXISTS (SELECT 1 FROM public.trainers WHERE id = auth.uid()))
      OR (target_audience = 'trainee' AND EXISTS (SELECT 1 FROM public.trainees WHERE id = auth.uid()))
    )
  );

-- Allow admins to manage announcements
CREATE POLICY "Allow admins to manage announcements"
  ON public.announcements
  FOR ALL
  TO authenticated
  USING (public.is_admin())
  WITH CHECK (public.is_admin());

-- Insert a mock announcement for the landing page
INSERT INTO public.announcements (title, content, is_active)
VALUES
  ('MoES Certification Track 2027 Announced', 'The Ministry of Earth Sciences has officially launched the new certification track for field meteorologists and data analysts.', true),
  ('Doppler Radar Maintenance Downtime', 'Scheduled maintenance for East Coast Doppler radar telemetry feeds will occur on Oct 1st. Simulations will use cached datasets during this time.', true)
ON CONFLICT DO NOTHING;

```

supabase/migrations/00053_course_communication.sql SQL 4.6 KB 153 Lines

```sql
-- Course Announcements Table
CREATE TABLE IF NOT EXISTS public.course_announcements (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  course_id UUID NOT NULL REFERENCES public.courses(id) ON DELETE CASCADE,
  trainer_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  content TEXT NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Course Messages (Chat/Q&A) Table
CREATE TABLE IF NOT EXISTS public.course_messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  course_id UUID NOT NULL REFERENCES public.courses(id) ON DELETE CASCADE,
  sender_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  content TEXT NOT NULL,
  is_private BOOLEAN NOT NULL DEFAULT false,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Enable RLS
ALTER TABLE public.course_announcements ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.course_messages ENABLE ROW LEVEL SECURITY;

-- Add realtime publication for chat
-- Check if course_messages is already in realtime publication, if not, add it
DO $$
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM pg_publication_tables
    WHERE pubname = 'supabase_realtime' AND tablename = 'course_messages'
  ) THEN
    ALTER PUBLICATION supabase_realtime ADD TABLE public.course_messages;
  END IF;
END
$$;

-------------------------------------------------------------
-- Policies for course_announcements
-------------------------------------------------------------

-- Trainers can insert/update announcements for their courses
CREATE POLICY "Trainers can manage course announcements"
  ON public.course_announcements
  FOR ALL
  TO authenticated
  USING (
    auth.uid() = trainer_id
    AND
    EXISTS (SELECT 1 FROM public.courses WHERE id = course_id AND trainer_id = auth.uid())
  );

-- Admins can manage all announcements
CREATE POLICY "Admins can manage course announcements"
  ON public.course_announcements
  FOR ALL
  TO authenticated
  USING (public.is_admin());

-- Enrolled trainees can view announcements for their courses
CREATE POLICY "Trainees can view course announcements"
  ON public.course_announcements
  FOR SELECT
  TO authenticated
  USING (
    EXISTS (
      SELECT 1 FROM public.enrollments
      WHERE enrollments.course_id = course_announcements.course_id
      AND enrollments.user_id = auth.uid()
      AND enrollments.status IN ('enrolled', 'completed', 'in_progress')
    )
  );

-- Also allow Trainers to view announcements for their courses
CREATE POLICY "Trainers can view course announcements"
```

```
  ON public.course_announcements
  FOR SELECT
  TO authenticated
  USING (
    EXISTS (SELECT 1 FROM public.courses WHERE id = course_id AND trainer_id = auth.uid())
  );

-------------------------------------------------------------
-- Policies for course_messages
-------------------------------------------------------------

-- Anyone enrolled can read public messages or their own private messages
CREATE POLICY "Trainees can read course messages"
  ON public.course_messages
  FOR SELECT
  TO authenticated
  USING (
    EXISTS (
      SELECT 1 FROM public.enrollments
      WHERE enrollments.course_id = course_messages.course_id
      AND enrollments.user_id = auth.uid()
      AND enrollments.status IN ('enrolled', 'completed', 'in_progress')
    )
    AND (is_private = false OR sender_id = auth.uid())
  );

-- Trainers can read ALL messages for their courses
CREATE POLICY "Trainers can read all course messages"
  ON public.course_messages
  FOR SELECT
  TO authenticated
  USING (
    EXISTS (
      SELECT 1 FROM public.courses
      WHERE courses.id = course_messages.course_id
      AND courses.trainer_id = auth.uid()
    )
  );

-- Admins can read all messages
CREATE POLICY "Admins can read all course messages"
  ON public.course_messages
  FOR SELECT
  TO authenticated
  USING (public.is_admin());

-- Trainees can insert messages for courses they are enrolled in
CREATE POLICY "Trainees can insert course messages"
  ON public.course_messages
  FOR INSERT
  TO authenticated
  WITH CHECK (
    sender_id = auth.uid()
    AND
    EXISTS (
      SELECT 1 FROM public.enrollments
      WHERE enrollments.course_id = course_messages.course_id
      AND enrollments.user_id = auth.uid()
      AND enrollments.status IN ('enrolled', 'completed', 'in_progress')
    )
  );

-- Trainers can insert messages for their courses
CREATE POLICY "Trainers can insert course messages"
  ON public.course_messages
  FOR INSERT
  TO authenticated
  WITH CHECK (
    sender_id = auth.uid()
    AND
    EXISTS (
      SELECT 1 FROM public.courses
      WHERE courses.id = course_messages.course_id
      AND courses.trainer_id = auth.uid()
    )
  );

```

supabase/migrations/00053_home_page_settings.sql SQL 6.5 KB 113 Lines

```sql
-- ============================================================================
-- Capacity Connect: Complete Home Page Management Table & RLS Policies
-- Migration: 00053_home_page_settings.sql
-- ============================================================================

-- 1. Create table if not exists
CREATE TABLE IF NOT EXISTS public.home_page_settings (
    id TEXT PRIMARY KEY DEFAULT 'default_settings',
    featured_programs_enabled BOOLEAN NOT NULL DEFAULT true,
    featured_programs_tag TEXT NOT NULL DEFAULT 'Top Certified Tracks',
    featured_programs_title TEXT NOT NULL DEFAULT 'Featured Learning Programs',
    featured_programs_subtitle TEXT NOT NULL DEFAULT 'Explore government-certified, high-impact earth science and meteorological training tracks led by premier MoES scientific institutions.',
    featured_programs_btn_text TEXT NOT NULL DEFAULT 'Explore All Courses',
    featured_programs_btn_link TEXT NOT NULL DEFAULT '/courses',
    featured_programs_items JSONB DEFAULT '[]'::jsonb,

    announcements_bar_enabled BOOLEAN NOT NULL DEFAULT true,
    announcements_bar_label TEXT NOT NULL DEFAULT 'Announcements',
    announcements_bar_speed INTEGER NOT NULL DEFAULT 24,
    announcements_items JSONB DEFAULT '[]'::jsonb,

    upcoming_tracks_enabled BOOLEAN NOT NULL DEFAULT true,
    upcoming_tracks_tag TEXT NOT NULL DEFAULT 'Upcoming Announcements',
    upcoming_tracks_title TEXT NOT NULL DEFAULT 'Specialized Earth Sciences Tracks.',
    upcoming_tracks_subtitle TEXT NOT NULL DEFAULT 'Pre-register for next-generation cohorts and masterclasses.',
    upcoming_tracks_btn_text TEXT NOT NULL DEFAULT 'View Complete Catalog',
    upcoming_tracks_items JSONB DEFAULT '[]'::jsonb,

    updated_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL,
    updated_by UUID REFERENCES auth.users(id) ON DELETE SET NULL
);

-- 2. Add any missing columns for existing tables
ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS featured_programs_enabled BOOLEAN NOT NULL DEFAULT true;
ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS featured_programs_tag TEXT NOT NULL DEFAULT 'Top Certified Tracks';
ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS featured_programs_title TEXT NOT NULL DEFAULT 'Featured Learning Programs';
ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS featured_programs_subtitle TEXT NOT NULL DEFAULT 'Explore government-certified, high-impact earth science and meteorological training tracks led by premier MoES scientific institutions.';
ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS featured_programs_btn_text TEXT NOT NULL DEFAULT 'Explore All Courses';
ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS featured_programs_btn_link TEXT NOT NULL DEFAULT '/courses';
ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS featured_programs_items JSONB DEFAULT '[]'::jsonb;

ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS announcements_bar_enabled BOOLEAN NOT NULL DEFAULT true;
ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS announcements_bar_label TEXT NOT NULL DEFAULT 'Announcements';
ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS announcements_bar_speed INTEGER NOT NULL DEFAULT 24;
ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS announcements_items JSONB DEFAULT '[]'::jsonb;

ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS upcoming_tracks_enabled BOOLEAN NOT NULL DEFAULT true;
ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS upcoming_tracks_tag TEXT NOT NULL DEFAULT 'Upcoming Announcements';
ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS upcoming_tracks_title TEXT NOT NULL DEFAULT 'Specialized Earth Sciences Tracks.';
ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS upcoming_tracks_subtitle TEXT NOT NULL DEFAULT 'Pre-register for next-generation cohorts and masterclasses.';
ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS upcoming_tracks_btn_text TEXT NOT NULL DEFAULT 'View Complete Catalog';
ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS upcoming_tracks_items JSONB DEFAULT '[]'::jsonb;
ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS updated_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL;
ALTER TABLE public.home_page_settings ADD COLUMN IF NOT EXISTS updated_by UUID;

-- 3. Create alias view "home_page"
CREATE OR REPLACE VIEW public.home_page AS
SELECT * FROM public.home_page_settings;
```

```sql
-- 4. Grant table privileges to anon, authenticated, and service_role

GRANT ALL ON TABLE public.home_page_settings TO anon, authenticated, service_role, postgres;
GRANT ALL ON TABLE public.home_page TO anon, authenticated, service_role, postgres;

-- 5. Enable Row Level Security (RLS)

ALTER TABLE public.home_page_settings ENABLE ROW LEVEL SECURITY;

-- 6. Allow all operations (select, insert, update, upsert) for anon and authenticated users

DROP POLICY IF EXISTS "Allow all operations for home_page_settings" ON public.home_page_settings;
DROP POLICY IF EXISTS "Allow public read access on home_page_settings" ON public.home_page_settings;
DROP POLICY IF EXISTS "Allow authenticated admin update on home_page_settings" ON public.home_page_settings;

CREATE POLICY "Allow all operations for home_page_settings"
ON public.home_page_settings
FOR ALL
TO public, anon, authenticated
USING (true)
WITH CHECK (true);

-- 7. Seed initial default configuration row

INSERT INTO public.home_page_settings (
    id,
    featured_programs_enabled,
    featured_programs_tag,
    featured_programs_title,
    featured_programs_subtitle,
    featured_programs_btn_text,
    featured_programs_btn_link,
    announcements_bar_enabled,
    announcements_bar_label,
    announcements_bar_speed,
    upcoming_tracks_enabled,
    upcoming_tracks_tag,
    upcoming_tracks_title,
    upcoming_tracks_subtitle,
    upcoming_tracks_btn_text
) VALUES (
    'default_settings',
    true,
    'Top Certified Tracks',
    'Featured Learning Programs',
    'Explore government-certified, high-impact earth science and meteorological training tracks led by premier MoES scientific institutions.',
    'Explore All Courses',
    '/courses',
    true,
    'Announcements',
    24,
    true,
    'Upcoming Announcements',
    'Specialized Earth Sciences Tracks.',
    'Pre-register for next-generation cohorts and masterclasses.',
    'View Complete Catalog'
) ON CONFLICT (id) DO NOTHING;

```

supabase/migrations/00054_homepage_storage_bucket.sql SQL 1.8 KB 44 Lines

```sql
-- ============================================================================
-- Capacity Connect: Supabase Storage Bucket for Homepage
-- Migration: 00054_homepage_storage_bucket.sql
-- ============================================================================

-- 1. Create or update the 'Homepage' bucket
INSERT INTO storage.buckets (id, name, public, file_size_limit, allowed_mime_types)
VALUES (
    'Homepage',
    'Homepage',
    true,
    10485760, -- 10MB limit
    ARRAY['image/png', 'image/jpeg', 'image/webp', 'image/gif', 'image/svg+xml']
)
ON CONFLICT (id) DO UPDATE SET
    public = true,
    file_size_limit = 10485760,
    allowed_mime_types = ARRAY['image/png', 'image/jpeg', 'image/webp', 'image/gif', 'image/svg+xml'];

-- 2. Storage RLS Policies: Allow public read access to everyone
DROP POLICY IF EXISTS "Public can view Homepage assets" ON storage.objects;
CREATE POLICY "Public can view Homepage assets"
ON storage.objects FOR SELECT
USING (bucket_id = 'Homepage');

-- 3. Storage RLS Policies: Allow authenticated users to upload and manage assets
DROP POLICY IF EXISTS "Authenticated users can upload Homepage assets" ON storage.objects;
CREATE POLICY "Authenticated users can upload Homepage assets"
ON storage.objects FOR INSERT
TO authenticated
WITH CHECK (bucket_id = 'Homepage');

DROP POLICY IF EXISTS "Authenticated users can update Homepage assets" ON storage.objects;
CREATE POLICY "Authenticated users can update Homepage assets"
ON storage.objects FOR UPDATE
TO authenticated
USING (bucket_id = 'Homepage');

DROP POLICY IF EXISTS "Authenticated users can delete Homepage assets" ON storage.objects;
CREATE POLICY "Authenticated users can delete Homepage assets"
ON storage.objects FOR DELETE
TO authenticated
USING (bucket_id = 'Homepage');

```

supabase/migrations/00055_fix_missing_grants.sql SQL 0.6 KB 9 Lines

```sql
-- Fix missing permissions for recently created tables
-- Re-run the global table grants to cover tables created since migration 00040
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO anon, authenticated, service_role;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO anon, authenticated, service_role;

-- Also set default privileges so future tables automatically get these permissions
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO anon, authenticated, service_role;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT USAGE, SELECT ON SEQUENCES TO anon, authenticated, service_role;

```

supabase/migrations/00056_announcements_author_name.sql SQL 0.1 KB 2 Lines

```sql
ALTER TABLE public.announcements ADD COLUMN author_name TEXT;

```

supabase/migrations/00057_course_messages_recipient.sql SQL 3.3 KB 77 Lines

```sql
-- Add recipient_id to course_messages to allow private messages to specific users
ALTER TABLE public.course_messages ADD COLUMN IF NOT EXISTS recipient_id UUID REFERENCES auth.users(id) ON DELETE SET NULL;

-- Update RLS for course_messages so trainees can read messages sent TO them
DROP POLICY IF EXISTS "Trainees can read course messages" ON public.course_messages;
CREATE POLICY "Trainees can read course messages"
  ON public.course_messages
  FOR SELECT
  TO authenticated
  USING (
    EXISTS (
      SELECT 1 FROM public.enrollments
      WHERE enrollments.course_id = course_messages.course_id
      AND enrollments.user_id = auth.uid()
      AND enrollments.status IN ('enrolled', 'completed', 'in_progress')
    )
    AND (is_private = false OR sender_id = auth.uid() OR recipient_id = auth.uid())
  );

-- Trigger for Course Chat Notifications
CREATE OR REPLACE FUNCTION notify_on_course_message()
RETURNS TRIGGER AS $$
DECLARE
  v_course_title TEXT;
  v_sender_name TEXT;
BEGIN
  -- Get course title
  SELECT title INTO v_course_title FROM public.courses WHERE id = NEW.course_id;

  -- Get sender name
  SELECT full_name INTO v_sender_name FROM public.trainers WHERE id = NEW.sender_id;
  IF v_sender_name IS NULL THEN
    SELECT full_name INTO v_sender_name FROM public.trainees WHERE id = NEW.sender_id;
  END IF;
  IF v_sender_name IS NULL THEN
    SELECT full_name INTO v_sender_name FROM public.admins WHERE id = NEW.sender_id;
  END IF;
  IF v_sender_name IS NULL THEN
    v_sender_name := 'Someone';
  END IF;

  IF NEW.is_private = true THEN
    -- If it's private, it's either going to the trainer or to a specific trainee
    IF NEW.recipient_id IS NOT NULL THEN
      -- Notify the specific recipient
      INSERT INTO public.notifications (user_id, type, title, message)
      VALUES (NEW.recipient_id, 'new_message', 'Private Message in ' || v_course_title, v_sender_name || ' sent you a private message.');
    ELSE
      -- If no recipient_id but private, it usually means it's to the trainer
      INSERT INTO public.notifications (user_id, type, title, message)
      SELECT trainer_id, 'new_message', 'Private Message in ' || v_course_title, v_sender_name || ' sent you a private message.'
      FROM public.courses WHERE id = NEW.course_id;
    END IF;
  ELSE
    -- Group broadcast: notify all enrolled trainees EXCEPT the sender
    INSERT INTO public.notifications (user_id, type, title, message)
    SELECT user_id, 'new_message', 'New Message in ' || v_course_title, v_sender_name || ': ' || substring(NEW.content from 1 for 50)
    FROM public.enrollments
    WHERE course_id = NEW.course_id AND status IN ('enrolled', 'completed', 'in_progress') AND user_id != NEW.sender_id;

    -- Also notify the trainer if they aren't the sender
    INSERT INTO public.notifications (user_id, type, title, message)
    SELECT trainer_id, 'new_message', 'New Message in ' || v_course_title, v_sender_name || ': ' || substring(NEW.content from 1 for 50)
    FROM public.courses
    WHERE id = NEW.course_id AND trainer_id != NEW.sender_id;
  END IF;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

DROP TRIGGER IF EXISTS trigger_notify_on_course_message ON public.course_messages;
```

```
CREATE TRIGGER trigger_notify_on_course_message
AFTER INSERT ON public.course_messages
FOR EACH ROW
EXECUTE FUNCTION notify_on_course_message();
```

supabase/migrations/00058_fix_orphaned_assessments.sql SQL 0.3 KB 2 Lines

```sql
UPDATE public.assessments SET created_by = (SELECT trainer_id FROM public.courses WHERE public.courses.id = public.assessments.course_id) WHERE created_by IS NULL OR created_by != (SELECT trainer_id FROM public.courses WHERE public.courses.id = public.assessments.course_id);

```

supabase/migrations/00059_fix_assessment_delete_rls.sql SQL 1.7 KB 54 Lines

```sql
-- Fix assessment deletion: allow trainer who owns the course to delete its assessments
-- Simplified: just check course ownership via courses.trainer_id - no need for profiles check

-- Drop the overly restrictive combined policy
DROP POLICY IF EXISTS "Trainers can manage own assessments" ON public.assessments;

-- SELECT
DROP POLICY IF EXISTS "Trainers can view assessments for own courses" ON public.assessments;
CREATE POLICY "Trainers can view assessments for own courses"
ON public.assessments FOR SELECT
USING (
  EXISTS (
    SELECT 1 FROM public.courses
    WHERE courses.id = assessments.course_id
      AND courses.trainer_id = auth.uid()
  )
);

-- INSERT
DROP POLICY IF EXISTS "Trainers can insert assessments for own courses" ON public.assessments;
CREATE POLICY "Trainers can insert assessments for own courses"
ON public.assessments FOR INSERT
WITH CHECK (
  EXISTS (
    SELECT 1 FROM public.courses
    WHERE courses.id = course_id
      AND courses.trainer_id = auth.uid()
  )
);

-- UPDATE
DROP POLICY IF EXISTS "Trainers can update assessments for own courses" ON public.assessments;
CREATE POLICY "Trainers can update assessments for own courses"
ON public.assessments FOR UPDATE
USING (
  EXISTS (
    SELECT 1 FROM public.courses
    WHERE courses.id = assessments.course_id
      AND courses.trainer_id = auth.uid()
  )
);

-- DELETE
DROP POLICY IF EXISTS "Trainers can delete assessments for own courses" ON public.assessments;
CREATE POLICY "Trainers can delete assessments for own courses"
ON public.assessments FOR DELETE
USING (
  EXISTS (
    SELECT 1 FROM public.courses
    WHERE courses.id = assessments.course_id
      AND courses.trainer_id = auth.uid()
  )
);

```

supabase/migrations/00060_add_results_publish_date.sql SQL 0.1 KB 2 Lines

```sql
ALTER TABLE "public"."assessments" ADD COLUMN "results_publish_date" timestamp with time zone;

```

supabase/migrations/00060_course_modules.sql SQL 2.6 KB 66 Lines

```sql
-- ============================================================================

-- Capacity Connect: Course Modules & Storage Bucket

-- Migration: 00060_course_modules.sql

-- ============================================================================


-- 1. Create or update the 'Course modules' storage bucket

INSERT INTO storage.buckets (id, name, public, file_size_limit)
VALUES (
    'Course modules',
    'Course modules',
    true,
    104857600 -- 100MB limit
)
ON CONFLICT (id) DO UPDATE SET
    public = true,
    file_size_limit = 104857600;

-- 2. Storage RLS Policies for 'Course modules'

DROP POLICY IF EXISTS "Public can view Course modules" ON storage.objects;
CREATE POLICY "Public can view Course modules"
ON storage.objects FOR SELECT
USING (bucket_id = 'Course modules');

DROP POLICY IF EXISTS "Authenticated users can upload Course modules" ON storage.objects;
CREATE POLICY "Authenticated users can upload Course modules"
ON storage.objects FOR INSERT
TO authenticated
WITH CHECK (bucket_id = 'Course modules');

DROP POLICY IF EXISTS "Authenticated users can update Course modules" ON storage.objects;
CREATE POLICY "Authenticated users can update Course modules"
ON storage.objects FOR UPDATE
TO authenticated
USING (bucket_id = 'Course modules');

DROP POLICY IF EXISTS "Authenticated users can delete Course modules" ON storage.objects;
CREATE POLICY "Authenticated users can delete Course modules"
ON storage.objects FOR DELETE
TO authenticated
USING (bucket_id = 'Course modules');

-- 3. Add modules JSONB column to courses table for direct payload storage

ALTER TABLE courses ADD COLUMN IF NOT EXISTS modules JSONB DEFAULT '[]'::jsonb;

-- 4. Create dedicated course_modules table

CREATE TABLE IF NOT EXISTS course_modules (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    course_id UUID NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
    title TEXT NOT NULL,
    description TEXT,
    order_index INTEGER NOT NULL DEFAULT 0,
    is_final_assessment BOOLEAN NOT NULL DEFAULT false,
    content_items JSONB DEFAULT '[]'::jsonb,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- 5. Enable RLS on course_modules

ALTER TABLE course_modules ENABLE ROW LEVEL SECURITY;

DROP POLICY IF EXISTS "Public can view course modules" ON course_modules;
CREATE POLICY "Public can view course modules" ON course_modules FOR SELECT USING (true);

DROP POLICY IF EXISTS "Authenticated users can manage course modules" ON course_modules;
CREATE POLICY "Authenticated users can manage course modules" ON course_modules FOR ALL TO authenticated USING (true) WITH CHE
```

```
CK (true);
 66 |
```

supabase/migrations/00061_course_messages_fixes.sql SQL 2.2 KB 58 Lines

```sql
-- ============================================================================
-- Capacity Connect: Fix Course Messages RLS & Permissions
-- Migration: 00061_course_messages_fixes.sql
-- ============================================================================

-- Ensure course_messages table has recipient_id and RLS
ALTER TABLE public.course_messages ADD COLUMN IF NOT EXISTS recipient_id UUID REFERENCES auth.users(id) ON DELETE SET NULL;

-- Allow Admins to insert messages
DROP POLICY IF EXISTS "Admins can insert course messages" ON public.course_messages;
CREATE POLICY "Admins can insert course messages"
  ON public.course_messages
  FOR INSERT
  TO authenticated
  WITH CHECK (public.is_admin());

-- Ensure Trainers can read all course messages
DROP POLICY IF EXISTS "Trainers can read all course messages" ON public.course_messages;
CREATE POLICY "Trainers can read all course messages"
  ON public.course_messages
  FOR SELECT
  TO authenticated
  USING (
    EXISTS (
      SELECT 1 FROM public.courses
      WHERE courses.id = course_messages.course_id
      AND courses.trainer_id = auth.uid()
    )
  );

-- Ensure Trainees can read public messages, their own sent messages, or messages addressed to them
DROP POLICY IF EXISTS "Trainees can read course messages" ON public.course_messages;
CREATE POLICY "Trainees can read course messages"
  ON public.course_messages
  FOR SELECT
  TO authenticated
  USING (
    EXISTS (
      SELECT 1 FROM public.enrollments
      WHERE enrollments.course_id = course_messages.course_id
      AND enrollments.user_id = auth.uid()
      AND enrollments.status IN ('enrolled', 'completed', 'in_progress')
    )
    AND (is_private = false OR sender_id = auth.uid() OR recipient_id = auth.uid())
  );

-- Ensure Admins can read all messages
DROP POLICY IF EXISTS "Admins can read all course messages" ON public.course_messages;
CREATE POLICY "Admins can read all course messages"
  ON public.course_messages
  FOR SELECT
  TO authenticated
  USING (public.is_admin());

-- Grant table access
GRANT ALL ON TABLE public.course_messages TO authenticated;
GRANT ALL ON TABLE public.course_messages TO service_role;
```

supabase/migrations/00062_course_feedback.sql SQL 2.1 KB 57 Lines

```sql
-- ============================================================================
-- Capacity Connect: Course Feedback Table & Policies
-- Migration: 00062_course_feedback.sql
-- ============================================================================

CREATE TABLE IF NOT EXISTS public.course_feedback (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  course_id UUID NOT NULL REFERENCES public.courses(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  rating INTEGER NOT NULL CHECK (rating >= 1 AND rating <= 5),
  comments TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  CONSTRAINT course_feedback_course_user_unique UNIQUE (course_id, user_id)
);

-- Enable RLS
ALTER TABLE public.course_feedback ENABLE ROW LEVEL SECURITY;

-- Trainees can insert their own feedback
DROP POLICY IF EXISTS "Trainees can insert own feedback" ON public.course_feedback;
CREATE POLICY "Trainees can insert own feedback"
  ON public.course_feedback
  FOR INSERT
  TO authenticated
  WITH CHECK (user_id = auth.uid());

-- Trainees can update their own feedback
DROP POLICY IF EXISTS "Trainees can update own feedback" ON public.course_feedback;
CREATE POLICY "Trainees can update own feedback"
  ON public.course_feedback
  FOR UPDATE
  TO authenticated
  USING (user_id = auth.uid())
  WITH CHECK (user_id = auth.uid());

-- Authenticated users (Trainers, Trainees, Admins) can read feedback
DROP POLICY IF EXISTS "Anyone can read course feedback" ON public.course_feedback;
CREATE POLICY "Anyone can read course feedback"
  ON public.course_feedback
  FOR SELECT
  TO authenticated
  USING (true);

-- Admins can manage all feedback
DROP POLICY IF EXISTS "Admins can manage feedback" ON public.course_feedback;
CREATE POLICY "Admins can manage feedback"
  ON public.course_feedback
  FOR ALL
  TO authenticated
  USING (public.is_admin())
  WITH CHECK (public.is_admin());

-- Grants
GRANT ALL ON TABLE public.course_feedback TO authenticated;
GRANT ALL ON TABLE public.course_feedback TO service_role;
```

supabase/migrations/00063_comprehensive_profile_fields.sql SQL 3.9 KB 92 Lines

```sql
-- Migration 00063: Add comprehensive profile fields for trainers, trainees, and admins

-- 1. Add columns to trainers table
ALTER TABLE public.trainers
  ADD COLUMN IF NOT EXISTS full_name TEXT,
  ADD COLUMN IF NOT EXISTS email TEXT,
  ADD COLUMN IF NOT EXISTS qualifications TEXT,
  ADD COLUMN IF NOT EXISTS work_experience TEXT,
  ADD COLUMN IF NOT EXISTS years_of_experience INTEGER,
  ADD COLUMN IF NOT EXISTS skills TEXT[] DEFAULT '{}',
  ADD COLUMN IF NOT EXISTS interests TEXT[] DEFAULT '{}',
  ADD COLUMN IF NOT EXISTS phone TEXT,
  ADD COLUMN IF NOT EXISTS alternate_phone TEXT,
  ADD COLUMN IF NOT EXISTS address TEXT,
  ADD COLUMN IF NOT EXISTS city TEXT,
  ADD COLUMN IF NOT EXISTS state TEXT,
  ADD COLUMN IF NOT EXISTS country TEXT DEFAULT 'India',
  ADD COLUMN IF NOT EXISTS pincode TEXT,
  ADD COLUMN IF NOT EXISTS linkedin_url TEXT,
  ADD COLUMN IF NOT EXISTS website_url TEXT,
  ADD COLUMN IF NOT EXISTS github_url TEXT,
  ADD COLUMN IF NOT EXISTS availability TEXT DEFAULT 'available',
  ADD COLUMN IF NOT EXISTS department TEXT,
  ADD COLUMN IF NOT EXISTS designation TEXT;

-- 2. Add columns to trainees table
ALTER TABLE public.trainees
  ADD COLUMN IF NOT EXISTS full_name TEXT,
  ADD COLUMN IF NOT EXISTS email TEXT,
  ADD COLUMN IF NOT EXISTS qualifications TEXT,
  ADD COLUMN IF NOT EXISTS work_experience TEXT,
  ADD COLUMN IF NOT EXISTS years_of_experience INTEGER,
  ADD COLUMN IF NOT EXISTS skills TEXT[] DEFAULT '{}',
  ADD COLUMN IF NOT EXISTS interests TEXT[] DEFAULT '{}',
  ADD COLUMN IF NOT EXISTS phone TEXT,
  ADD COLUMN IF NOT EXISTS alternate_phone TEXT,
  ADD COLUMN IF NOT EXISTS address TEXT,
  ADD COLUMN IF NOT EXISTS city TEXT,
  ADD COLUMN IF NOT EXISTS state TEXT,
  ADD COLUMN IF NOT EXISTS country TEXT DEFAULT 'India',
  ADD COLUMN IF NOT EXISTS pincode TEXT,
  ADD COLUMN IF NOT EXISTS linkedin_url TEXT,
  ADD COLUMN IF NOT EXISTS website_url TEXT,
  ADD COLUMN IF NOT EXISTS github_url TEXT,
  ADD COLUMN IF NOT EXISTS bio TEXT,
  ADD COLUMN IF NOT EXISTS learning_goals TEXT,
  ADD COLUMN IF NOT EXISTS department TEXT,
  ADD COLUMN IF NOT EXISTS designation TEXT;

-- 3. Add columns to admins table
ALTER TABLE public.admins
  ADD COLUMN IF NOT EXISTS phone TEXT,
  ADD COLUMN IF NOT EXISTS alternate_phone TEXT,
  ADD COLUMN IF NOT EXISTS address TEXT,
  ADD COLUMN IF NOT EXISTS city TEXT,
  ADD COLUMN IF NOT EXISTS state TEXT,
  ADD COLUMN IF NOT EXISTS country TEXT DEFAULT 'India',
  ADD COLUMN IF NOT EXISTS pincode TEXT,
  ADD COLUMN IF NOT EXISTS qualifications TEXT,
  ADD COLUMN IF NOT EXISTS work_experience TEXT,
  ADD COLUMN IF NOT EXISTS years_of_experience INTEGER,
  ADD COLUMN IF NOT EXISTS skills TEXT[] DEFAULT '{}',
  ADD COLUMN IF NOT EXISTS interests TEXT[] DEFAULT '{}',
  ADD COLUMN IF NOT EXISTS bio TEXT,
  ADD COLUMN IF NOT EXISTS department TEXT,
  ADD COLUMN IF NOT EXISTS designation TEXT,
  ADD COLUMN IF NOT EXISTS linkedin_url TEXT,
  ADD COLUMN IF NOT EXISTS website_url TEXT,
  ADD COLUMN IF NOT EXISTS github_url TEXT;

-- 4. Enable RLS Update Policies so users can update their own data
```

```
DO $$
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM pg_policies WHERE tablename = 'trainers' AND policyname = 'Trainers can update own details'
  ) THEN
    CREATE POLICY "Trainers can update own details" ON public.trainers FOR UPDATE TO authenticated USING (auth.uid() = id);
  END IF;

  IF NOT EXISTS (
    SELECT 1 FROM pg_policies WHERE tablename = 'trainees' AND policyname = 'Trainees can update own details'
  ) THEN
    CREATE POLICY "Trainees can update own details" ON public.trainees FOR UPDATE TO authenticated USING (auth.uid() = id);
  END IF;

  IF NOT EXISTS (
    SELECT 1 FROM pg_policies WHERE tablename = 'admins' AND policyname = 'Admins can update own details'
  ) THEN
    CREATE POLICY "Admins can update own details" ON public.admins FOR UPDATE TO authenticated USING (auth.uid() = id);
  END IF;
END $$;

```

supabase/migrations/00064_course_certificates.sql SQL 4.5 KB 113 Lines

```sql
-- Migration 00064: Add certificate template support to courses and create storage buckets

-- 1. Add certificate fields to courses table
ALTER TABLE public.courses
  ADD COLUMN IF NOT EXISTS has_certificate BOOLEAN DEFAULT true,
  ADD COLUMN IF NOT EXISTS certificate_template_url TEXT,
  ADD COLUMN IF NOT EXISTS certificate_template_name TEXT,
  ADD COLUMN IF NOT EXISTS certificate_title TEXT;

-- 2. Add certificate fields to enrollments table
ALTER TABLE public.enrollments
  ADD COLUMN IF NOT EXISTS certificate_url TEXT,
  ADD COLUMN IF NOT EXISTS certificate_issued_at TIMESTAMPTZ,
  ADD COLUMN IF NOT EXISTS certificate_grade TEXT;

-- 3. Create or update certificates table to track issued certificates
CREATE TABLE IF NOT EXISTS public.certificates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  course_id UUID NOT NULL REFERENCES public.courses(id) ON DELETE CASCADE,
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  trainee_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  enrollment_id UUID REFERENCES public.enrollments(id) ON DELETE SET NULL,
  trainee_name TEXT,
  course_title TEXT,
  percentage NUMERIC(5,2) DEFAULT 100,
  file_url TEXT,
  file_name TEXT,
  issued_at TIMESTAMPTZ DEFAULT now(),
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Ensure columns exist in case table was created earlier with a different schema
ALTER TABLE public.certificates
  ADD COLUMN IF NOT EXISTS user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  ADD COLUMN IF NOT EXISTS trainee_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  ADD COLUMN IF NOT EXISTS trainee_name TEXT,
  ADD COLUMN IF NOT EXISTS course_title TEXT,
  ADD COLUMN IF NOT EXISTS percentage NUMERIC(5,2) DEFAULT 100,
  ADD COLUMN IF NOT EXISTS file_url TEXT,
  ADD COLUMN IF NOT EXISTS file_name TEXT,
  ADD COLUMN IF NOT EXISTS issued_at TIMESTAMPTZ DEFAULT now(),
  ADD COLUMN IF NOT EXISTS created_at TIMESTAMPTZ DEFAULT now();

-- Enable RLS on certificates
ALTER TABLE public.certificates ENABLE ROW LEVEL SECURITY;

-- Clean existing policies to prevent conflicts
DROP POLICY IF EXISTS "Authenticated users can read certificates" ON public.certificates;
DROP POLICY IF EXISTS "Users can insert their own certificate" ON public.certificates;
DROP POLICY IF EXISTS "Trainers and Admins can manage certificates" ON public.certificates;

-- Recreate Certificates RLS Policies
CREATE POLICY "Authenticated users can read certificates"
  ON public.certificates FOR SELECT
  TO authenticated
  USING (true);

CREATE POLICY "Users can insert their own certificate"
  ON public.certificates FOR INSERT
  TO authenticated
  WITH CHECK (
    auth.uid() = trainee_id OR
    auth.uid() = user_id OR
    auth.uid() IS NOT NULL
  );

-- 4. Create Storage Buckets for certificate templates and generated certificates
```

```sql
INSERT INTO storage.buckets (id, name, public)
VALUES
  ('Certificate template', 'Certificate template', true),
  ('certificates', 'certificates', true)
ON CONFLICT (id) DO UPDATE SET public = true;

-- Drop existing storage policies if they exist

DROP POLICY IF EXISTS "Public read certificate template" ON storage.objects;
DROP POLICY IF EXISTS "Authenticated upload certificate template" ON storage.objects;
DROP POLICY IF EXISTS "Authenticated update certificate template" ON storage.objects;
DROP POLICY IF EXISTS "Public read certificates" ON storage.objects;
DROP POLICY IF EXISTS "Authenticated upload certificates" ON storage.objects;
DROP POLICY IF EXISTS "Authenticated update certificates" ON storage.objects;

-- Storage Policies for 'Certificate template'

CREATE POLICY "Public read certificate template"
  ON storage.objects FOR SELECT
  TO public
  USING (bucket_id = 'Certificate template');

CREATE POLICY "Authenticated upload certificate template"
  ON storage.objects FOR INSERT
  TO authenticated
  WITH CHECK (bucket_id = 'Certificate template');

CREATE POLICY "Authenticated update certificate template"
  ON storage.objects FOR UPDATE
  TO authenticated
  USING (bucket_id = 'Certificate template');

-- Storage Policies for 'certificates'

CREATE POLICY "Public read certificates"
  ON storage.objects FOR SELECT
  TO public
  USING (bucket_id = 'certificates');

CREATE POLICY "Authenticated upload certificates"
  ON storage.objects FOR INSERT
  TO authenticated
  WITH CHECK (bucket_id = 'certificates');

CREATE POLICY "Authenticated update certificates"
  ON storage.objects FOR UPDATE
  TO authenticated
  USING (bucket_id = 'certificates');

```

supabase/migrations/00065_allow_trainer_insert_skills.sql SQL 0.6 KB 18 Lines

```sql
-- ============================================================================
-- Capacity Connect: Allow authenticated trainers to insert skills
-- Migration: 00065_allow_trainer_insert_skills.sql
-- ============================================================================

DO $$
BEGIN
  -- Drop restrictive insert policies if any
  DROP POLICY IF EXISTS "Authenticated users can insert skills" ON public.skills;

  -- Allow all authenticated trainers and users to insert new skills
  CREATE POLICY "Authenticated users can insert skills"
  ON public.skills FOR INSERT
  TO authenticated
  WITH CHECK (true);
END
$$;

```

supabase/migrations/00066_fix_learning_goals_and_availability.sql SQL 0.4 KB 12 Lines

```sql
-- Migration 00066: Ensure learning_goals and availability exist across role tables to avoid any schema cache mismatches

ALTER TABLE public.trainers
  ADD COLUMN IF NOT EXISTS learning_goals TEXT;

ALTER TABLE public.admins
  ADD COLUMN IF NOT EXISTS learning_goals TEXT,
  ADD COLUMN IF NOT EXISTS availability TEXT DEFAULT 'available';

ALTER TABLE public.trainees
  ADD COLUMN IF NOT EXISTS availability TEXT DEFAULT 'available';

```

supabase/migrations/00067_course_edit_requests.sql SQL 1.7 KB 43 Lines

```sql
-- Migration 00067: Add Course Edit Permission Request Workflow columns and table

ALTER TABLE public.courses
  ADD COLUMN IF NOT EXISTS edit_request_status TEXT DEFAULT 'none',
  ADD COLUMN IF NOT EXISTS edit_request_reason TEXT,
  ADD COLUMN IF NOT EXISTS edit_request_at TIMESTAMPTZ,
  ADD COLUMN IF NOT EXISTS edit_window_expires_at TIMESTAMPTZ,
  ADD COLUMN IF NOT EXISTS edit_window_duration_hours INTEGER,
  ADD COLUMN IF NOT EXISTS admin_edit_notes TEXT;

-- Create course_edit_requests audit & history table

CREATE TABLE IF NOT EXISTS public.course_edit_requests (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  course_id UUID NOT NULL REFERENCES public.courses(id) ON DELETE CASCADE,
  trainer_id UUID NOT NULL REFERENCES public.trainers(id) ON DELETE CASCADE,
  reason TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'pending', -- 'pending', 'approved', 'rejected', 'completed'
  duration_hours INTEGER DEFAULT 24,
  expires_at TIMESTAMPTZ,
  admin_notes TEXT,
  reviewed_by UUID REFERENCES public.admins(id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- Enable RLS

ALTER TABLE public.course_edit_requests ENABLE ROW LEVEL SECURITY;

-- Policies for course_edit_requests

DROP POLICY IF EXISTS "Trainers can view and create their course edit requests" ON public.course_edit_requests;
CREATE POLICY "Trainers can view and create their course edit requests"
  ON public.course_edit_requests
  FOR ALL
  TO authenticated
  USING (
    trainer_id = auth.uid()
    OR EXISTS (SELECT 1 FROM public.admins WHERE id = auth.uid())
  )
  WITH CHECK (
    trainer_id = auth.uid()
    OR EXISTS (SELECT 1 FROM public.admins WHERE id = auth.uid())
  );

```

supabase/migrations/00068_enroll_and_drop_functions.sql SQL 3.0 KB 95 Lines

```sql
-- Migration 00068: Add enroll_trainee and drop_enrollment database functions + RLS policies

-- 1. Ensure Trainees can update their own enrollment records (e.g. progress, withdrawal)
DROP POLICY IF EXISTS "Trainees can update own enrollments" ON public.enrollments;
CREATE POLICY "Trainees can update own enrollments"
ON public.enrollments FOR UPDATE TO authenticated
USING (user_id = auth.uid())
WITH CHECK (user_id = auth.uid());

-- 2. Ensure Trainees can delete/withdraw their own enrollment records
DROP POLICY IF EXISTS "Trainees can delete own enrollments" ON public.enrollments;
CREATE POLICY "Trainees can delete own enrollments"
ON public.enrollments FOR DELETE TO authenticated
USING (user_id = auth.uid());

-- 3. Function: public.enroll_trainee
CREATE OR REPLACE FUNCTION public.enroll_trainee(p_course_id UUID, p_user_id UUID)
RETURNS JSONB
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
  v_course RECORD;
  v_enrolled_count INT;
  v_initial_status TEXT;
  v_enrollment_id UUID;
BEGIN
  -- Fetch course info
  SELECT id, title, trainer_id, max_trainees, status INTO v_course
  FROM public.courses
  WHERE id = p_course_id;

  IF v_course.id IS NULL THEN
    RAISE EXCEPTION 'Course not found';
  END IF;

  -- Check current enrolled count
  SELECT COUNT(*) INTO v_enrolled_count
  FROM public.enrollments
  WHERE course_id = p_course_id AND status IN ('enrolled', 'in_progress', 'completed');

  IF v_course.max_trainees IS NOT NULL AND v_enrolled_count >= v_course.max_trainees THEN
    v_initial_status := 'waitlisted';
  ELSE
    v_initial_status := 'pending_approval';
  END IF;

  -- Upsert enrollment
  INSERT INTO public.enrollments (course_id, user_id, status, enrolled_at, progress_percent)
  VALUES (p_course_id, p_user_id, v_initial_status, now(), 0)
  ON CONFLICT (course_id, user_id)
  DO UPDATE SET
    status = EXCLUDED.status,
    enrolled_at = now(),
    progress_percent = 0
  RETURNING id INTO v_enrollment_id;

  -- Notify Trainer
  IF v_course.trainer_id IS NOT NULL THEN
    INSERT INTO public.notifications (user_id, type, title, message)
    VALUES (
      v_course.trainer_id,
      'enrollment_request:' || p_course_id,
      'New Enrollment Request',
      'A trainee requested to enroll in ' || v_course.title
    );
  END IF;

```

```
  RETURN jsonb_build_object(
    'success', true,
    'enrollment_id', v_enrollment_id,
    'status', v_initial_status
  );
END;
$$;

-- 4. Function: public.drop_enrollment

CREATE OR REPLACE FUNCTION public.drop_enrollment(p_course_id UUID, p_user_id UUID)
RETURNS JSONB
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
BEGIN
  UPDATE public.enrollments
  SET status = 'withdrawn'
  WHERE course_id = p_course_id AND user_id = p_user_id;

  RETURN jsonb_build_object('success', true);
END;
$$;

-- Grant execute permissions

GRANT EXECUTE ON FUNCTION public.enroll_trainee(UUID, UUID) TO authenticated, anon, service_role;
GRANT EXECUTE ON FUNCTION public.drop_enrollment(UUID, UUID) TO authenticated, anon, service_role;

```

supabase/migrations/00069_contact_messages.sql SQL 1.2 KB 44 Lines

```sql
-- Migration: 00069_contact_messages.sql

-- Description: Create contact_messages table for public contact submissions and admin management

CREATE TABLE IF NOT EXISTS public.contact_messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  email TEXT NOT NULL,
  department TEXT,
  subject TEXT,
  message TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'unread', -- 'unread', 'read', 'resolved'
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Enable RLS

ALTER TABLE public.contact_messages ENABLE ROW LEVEL SECURITY;

-- Allow anyone (public/anon/authenticated) to insert messages

CREATE POLICY "Allow public insert to contact_messages"
ON public.contact_messages
FOR INSERT
TO public
WITH CHECK (true);

-- Allow authenticated admins to view and manage messages

CREATE POLICY "Allow admins all on contact_messages"
ON public.contact_messages
FOR ALL
TO authenticated
USING (
  EXISTS (
    SELECT 1 FROM public.admins WHERE admins.id = auth.uid()
  )
)
WITH CHECK (
  EXISTS (
    SELECT 1 FROM public.admins WHERE admins.id = auth.uid()
  )
);

-- Grant table privileges

GRANT ALL ON public.contact_messages TO anon, authenticated, service_role;

```

supabase/migrations/00070_announcement_and_session_notifications.sql SQL 5.4 KB 145 Lines

```sql
-- Notification triggers for Course Announcements, Platform Announcements, and Course Sessions

-- 1. Function to notify enrolled trainees when a Course Announcement is created

CREATE OR REPLACE FUNCTION notify_on_course_announcement()
RETURNS TRIGGER AS $$
DECLARE
  v_course_title TEXT;
  v_course_trainer_id UUID;
BEGIN
  -- Get course title and assigned trainer

  SELECT title, trainer_id INTO v_course_title, v_course_trainer_id
  FROM public.courses
  WHERE id = NEW.course_id;

  -- 1. Notify all enrolled trainees for this course

  INSERT INTO public.notifications (user_id, type, title, message)
  SELECT
    e.user_id,
    'course_announcement:' || NEW.course_id::text,
    '📢 Announcement: ' || NEW.title,
    'New update in "' || COALESCE(v_course_title, 'Course') || '": ' || SUBSTRING(NEW.content FROM 1 FOR 140)
  FROM public.enrollments e
  WHERE e.course_id = NEW.course_id
    AND e.status IN ('enrolled', 'in_progress', 'completed')
    AND e.user_id != NEW.trainer_id;

  -- 2. If announcement created by admin or someone other than assigned trainer, notify trainer too

  IF v_course_trainer_id IS NOT NULL AND v_course_trainer_id != NEW.trainer_id THEN
    INSERT INTO public.notifications (user_id, type, title, message)
    VALUES (
      v_course_trainer_id,
      'course_announcement:' || NEW.course_id::text,
      '📢 Announcement Posted: ' || NEW.title,
      'An announcement was posted in your course "' || COALESCE(v_course_title, 'Course') || '".'
    );
  END IF;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

DROP TRIGGER IF EXISTS trigger_notify_on_course_announcement ON public.course_announcements;
CREATE TRIGGER trigger_notify_on_course_announcement
AFTER INSERT ON public.course_announcements
FOR EACH ROW
EXECUTE FUNCTION notify_on_course_announcement();


-- 2. Function to notify enrolled trainees & trainers when a Course Session is scheduled or updated

CREATE OR REPLACE FUNCTION notify_on_course_session()
RETURNS TRIGGER AS $$
DECLARE
  v_course_title TEXT;
  v_course_trainer_id UUID;
  v_time_text TEXT;
BEGIN
  -- Get course info

  SELECT title, trainer_id INTO v_course_title, v_course_trainer_id
  FROM public.courses
  WHERE id = NEW.course_id;

  IF NEW.start_time IS NOT NULL THEN
    v_time_text := ' scheduled for ' || TO_CHAR(NEW.start_time, 'Mon DD, YYYY HH12:MI AM');
  ELSE
    v_time_text := '';
  END IF;

  -- Only notify on INSERT or if start_time / title changed significantly
```

```
  IF TG_OP = 'INSERT' OR (TG_OP = 'UPDATE' AND (OLD.start_time IS DISTINCT FROM NEW.start_time OR OLD.title IS DISTINCT FROM NEW.title OR OLD.meet_link IS DISTINCT FROM NEW.meet_link)) THEN

    -- Notify all active enrolled trainees

    INSERT INTO public.notifications (user_id, type, title, message)
    SELECT
      e.user_id,
      'course_session:' || NEW.course_id::text,
      CASE WHEN TG_OP = 'INSERT' THEN '🗓️ New Session: ' || NEW.title ELSE '🗓️ Session Updated: ' || NEW.title END,
      'Live session in "' || COALESCE(v_course_title, 'Course') || '"' || v_time_text || '.'
    FROM public.enrollments e
    WHERE e.course_id = NEW.course_id
      AND e.status IN ('enrolled', 'in_progress', 'completed');

    -- If updated/created and assigned trainer exists, also notify trainer (if session was updated by admin)

    IF v_course_trainer_id IS NOT NULL AND auth.uid() != v_course_trainer_id THEN
      INSERT INTO public.notifications (user_id, type, title, message)
      VALUES (
        v_course_trainer_id,
        'course_session:' || NEW.course_id::text,
        CASE WHEN TG_OP = 'INSERT' THEN '🗓️ New Session: ' || NEW.title ELSE '🗓️ Session Updated: ' || NEW.title END,
        'A session was created/updated in your course "' || COALESCE(v_course_title, 'Course') || '"' || v_time_text || '.'
      );
    END IF;

  END IF;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

DROP TRIGGER IF EXISTS trigger_notify_on_course_session ON public.course_sessions;
CREATE TRIGGER trigger_notify_on_course_session
AFTER INSERT OR UPDATE ON public.course_sessions
FOR EACH ROW
EXECUTE FUNCTION notify_on_course_session();


-- 3. Function to notify users when a Platform Announcement is created

CREATE OR REPLACE FUNCTION notify_on_platform_announcement()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.is_active = true THEN
    -- If targeting all or trainees

    IF NEW.target_audience IN ('all', 'trainee') THEN
      INSERT INTO public.notifications (user_id, type, title, message)
      SELECT
        id,
        'platform_announcement',
        '📢 ' || NEW.title,
        SUBSTRING(NEW.content FROM 1 FOR 140)
      FROM public.trainees
      WHERE approval_status = 'approved';
    END IF;

    -- If targeting all or trainers

    IF NEW.target_audience IN ('all', 'trainer') THEN
      INSERT INTO public.notifications (user_id, type, title, message)
      SELECT
        id,
        'platform_announcement',
        '📢 ' || NEW.title,
        SUBSTRING(NEW.content FROM 1 FOR 140)
      FROM public.trainers
      WHERE approval_status = 'approved';
    END IF;
  END IF;

  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

DROP TRIGGER IF EXISTS trigger_notify_on_platform_announcement ON public.announcements;
CREATE TRIGGER trigger_notify_on_platform_announcement
```

```
AFTER INSERT ON public.announcements
FOR EACH ROW
EXECUTE FUNCTION notify_on_platform_announcement();

```

supabase/migrations/00071_achievements_section_settings.sql SQL 1.5 KB 33 Lines

```sql
-- ============================================================================
-- Capacity Connect: Add Achievements & Results Section Columns
-- Migration: 00071_achievements_section_settings.sql
-- ============================================================================

-- 1. Add Section 4 (Achievements & Results) columns to home_page_settings
ALTER TABLE public.home_page_settings
ADD COLUMN IF NOT EXISTS achievements_enabled BOOLEAN NOT NULL DEFAULT true;

ALTER TABLE public.home_page_settings
ADD COLUMN IF NOT EXISTS achievements_tag TEXT NOT NULL DEFAULT 'Proven National Impact';

ALTER TABLE public.home_page_settings
ADD COLUMN IF NOT EXISTS achievements_title TEXT NOT NULL DEFAULT 'Inspired trainees. Inspired results';

ALTER TABLE public.home_page_settings
ADD COLUMN IF NOT EXISTS achievements_subtitle TEXT NOT NULL DEFAULT 'National Benchmark Results & Field Recognitions';

ALTER TABLE public.home_page_settings
ADD COLUMN IF NOT EXISTS achievements_moes_badge TEXT NOT NULL DEFAULT '100% Verified Operational Skill Credentials';

ALTER TABLE public.home_page_settings
ADD COLUMN IF NOT EXISTS achievements_moes_desc TEXT NOT NULL DEFAULT 'Direct alignment with WMO, IMD & INCOIS forecast protocols.';

ALTER TABLE public.home_page_settings
ADD COLUMN IF NOT EXISTS achievements_stories JSONB DEFAULT '[]'::jsonb;

-- 2. Refresh permissions and RLS
GRANT ALL ON TABLE public.home_page_settings TO anon, authenticated, service_role, postgres;

-- 3. Notify postgrest schema reload
NOTIFY pgrst, 'reload schema';
```

supabase/migrations/00072_session_attendance.sql SQL 2.7 KB 77 Lines

```sql
-- Create session_attendance_meta table
CREATE TABLE IF NOT EXISTS public.session_attendance_meta (
    session_id UUID PRIMARY KEY REFERENCES public.course_sessions(id) ON DELETE CASCADE,
    total_generated INT NOT NULL DEFAULT 0,
    is_manual_mapping BOOLEAN NOT NULL DEFAULT false,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

ALTER TABLE public.session_attendance_meta ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Trainers can manage their session meta" ON public.session_attendance_meta
    FOR ALL USING (
        EXISTS (
            SELECT 1 FROM public.course_sessions cs
            JOIN public.courses c ON c.id = cs.course_id
            WHERE cs.id = session_attendance_meta.session_id
            AND c.trainer_id = auth.uid()
        )
    );

CREATE POLICY "Trainees can view their session meta" ON public.session_attendance_meta
    FOR SELECT USING (
        EXISTS (
            SELECT 1 FROM public.course_sessions cs
            JOIN public.enrollments e ON e.course_id = cs.course_id
            WHERE cs.id = session_attendance_meta.session_id
            AND e.user_id = auth.uid()
        )
    );

CREATE POLICY "Admins can do everything on session meta" ON public.session_attendance_meta
    FOR ALL USING (
        EXISTS (
            SELECT 1 FROM public.admins a WHERE a.id = auth.uid()
        )
    );

-- Create session_attendance table
CREATE TABLE IF NOT EXISTS public.session_attendance (
    session_id UUID REFERENCES public.course_sessions(id) ON DELETE CASCADE,
    user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
    entered_count INT NOT NULL DEFAULT 0,
    status_override TEXT,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now(),
    PRIMARY KEY (session_id, user_id)
);

ALTER TABLE public.session_attendance ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Trainers can manage attendance for their sessions" ON public.session_attendance
    FOR ALL USING (
        EXISTS (
            SELECT 1 FROM public.course_sessions cs
            JOIN public.courses c ON c.id = cs.course_id
            WHERE cs.id = session_attendance.session_id
            AND c.trainer_id = auth.uid()
        )
    );

CREATE POLICY "Trainees can view and update their own attendance" ON public.session_attendance
    FOR ALL USING (
        user_id = auth.uid()
    );

CREATE POLICY "Admins can do everything on attendance" ON public.session_attendance
    FOR ALL USING (
        EXISTS (
            SELECT 1 FROM public.admins a WHERE a.id = auth.uid()
        )
    );

-- Enable realtime for these tables
ALTER PUBLICATION supabase_realtime ADD TABLE public.session_attendance_meta;
```

```
76 | ALTER PUBLICATION supabase_realtime ADD TABLE public.session_attendance;
77 |
```

supabase/migrations/00073_check_email_exists.sql SQL 0.3 KB 17 Lines

```sql
CREATE OR REPLACE FUNCTION public.check_email_exists(p_email text)
RETURNS boolean
LANGUAGE plpgsql
SECURITY DEFINER
AS $$
DECLARE
  v_exists boolean;
BEGIN
  -- We check auth.users directly to be perfectly accurate
  SELECT EXISTS (
    SELECT 1 FROM auth.users WHERE email = p_email
  ) INTO v_exists;

  RETURN v_exists;
END;
$$;

```

supabase/migrations/create_super_admin_perfect.sql SQL 0.9 KB 29 Lines

```sql
DO $$
DECLARE
  v_user_id UUID := gen_random_uuid();
  v_email TEXT := 'superadmin@capacityconnect.com';
  v_password TEXT := '12345678';
BEGIN
  -- Insert the user
  INSERT INTO auth.users (
    id, instance_id, email, encrypted_password, email_confirmed_at,
    raw_app_meta_data, raw_user_meta_data, created_at, updated_at,
    role, confirmation_token, recovery_token, email_change_token_new, email_change
  ) VALUES (
    v_user_id, '00000000-0000-0000-0000-000000000000', v_email,
    crypt(v_password, gen_salt('bf')), now(),
    '{"provider": "email", "providers": ["email"]}',
    jsonb_build_object('full_name', 'System Super Admin', 'role', 'super_admin'),
    now(), now(), 'authenticated', '', '', '', ''
  );

  -- Since the trigger auto-creates the profile, we just update it
  UPDATE public.profiles
  SET
    role = 'super_admin',
    approval_status = 'approved',
    full_name = 'System Super Admin'
  WHERE id = v_user_id;

END $$;

```

supabase/migrations/fix_super_admin.sql SQL 1.0 KB 29 Lines

```sql
DO $$
DECLARE
  v_user_id UUID := gen_random_uuid();
  v_email TEXT := 'superadmin@capacityconnect.com';
  v_password TEXT := '12345678';
BEGIN
  -- 1. Create a brand new, clean user
  INSERT INTO auth.users (
    id, instance_id, email, encrypted_password, email_confirmed_at,
    raw_app_meta_data, raw_user_meta_data, created_at, updated_at,
    role, confirmation_token, recovery_token, email_change_token_new, email_change
  ) VALUES (
    v_user_id, '00000000-0000-0000-0000-000000000000', v_email,
    crypt(v_password, gen_salt('bf')), now(),
    '{"provider": "email", "providers": ["email"]}',
    jsonb_build_object('full_name', 'System Super Admin', 'role', 'super_admin'),
    now(), now(), 'authenticated', '', '', '', ''
  );

  -- 2. Update the profile that was automatically generated by your database triggers
  UPDATE public.profiles
  SET
    role = 'super_admin',
    approval_status = 'approved',
    full_name = 'System Super Admin'
  WHERE id = v_user_id;

END $$;
```

supabase/migrations/get_user.sql SQL 0.1 KB 2 Lines

```sql
SELECT id, email, encrypted_password FROM auth.users WHERE email = 'sanjithchandrasekar03@gmail.com';

```

supabase/migrations/manual_deployment.sql SQL 1.0 KB 36 Lines

```sql
-- Migration 35: Make thumbnails public
CREATE POLICY "Public Thumbnails Access"
ON storage.objects FOR SELECT
USING ( bucket_id = 'materials' );

-- Migration 36: Add pending_approval and rejected to enrollments
ALTER TABLE enrollments
DROP CONSTRAINT IF EXISTS enrollments_status_check;

ALTER TABLE enrollments
ADD CONSTRAINT enrollments_status_check
CHECK (status IN ('enrolled', 'completed', 'dropped', 'pending_approval', 'rejected'));

-- Migration 37: Fix backend permissions and RLS infinite recursion
GRANT USAGE ON SCHEMA public TO anon, authenticated, service_role;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO anon, authenticated, service_role;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO anon, authenticated, service_role;

CREATE OR REPLACE FUNCTION public.is_admin()
RETURNS boolean
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
  v_is_admin boolean;
BEGIN
  SELECT EXISTS (
    SELECT 1 FROM public.admins
    WHERE id = auth.uid()
  ) INTO v_is_admin;

  RETURN v_is_admin;
END;
$$;

```

supabase/migrations/schema.sql SQL 0.0 KB 1 Lines

```
1 |
```

supabase/migrations/update_super_admin.sql SQL 0.7 KB 26 Lines

```sql
DO $$
DECLARE
  v_user_id UUID;
  v_email TEXT := 'superadmin@capacityconnect.com';
  v_password TEXT := '12345678';
BEGIN
  -- Get the user ID
  SELECT id INTO v_user_id FROM auth.users WHERE email = v_email;

  IF v_user_id IS NOT NULL THEN
    -- Update the password and role
    UPDATE auth.users
    SET encrypted_password = crypt(v_password, gen_salt('bf')),
        raw_user_meta_data = jsonb_build_object('full_name', 'System Super Admin', 'role', 'super_admin')
    WHERE id = v_user_id;

    -- Update the profile
    UPDATE public.profiles
    SET
      role = 'super_admin',
      approval_status = 'approved',
      full_name = 'System Super Admin'
    WHERE id = v_user_id;
  END IF;
END $$;

```

supabase/migrations/upgrade_only.sql SQL 0.1 KB 4 Lines

```sql
UPDATE public.profiles
SET role = 'super_admin'
WHERE email = 'sanjithchandrasekar03@gmail.com';

```

scripts/db-audit.mjs MJS 3.7 KB 102 Lines

```mjs
import { createClient } from '@supabase/supabase-js'

const SUPABASE_URL = process.env.VITE_SUPABASE_URL
const SERVICE_KEY = process.env.SUPABASE_SERVICE_KEY || process.env.VITE_SUPABASE_ANON_KEY

// Use service_role key if available (bypasses RLS)
const s = createClient(SUPABASE_URL, SERVICE_KEY, {
  auth: { autoRefreshToken: false, persistSession: false }
})

async function run() {
  const tables = ['assessments', 'questions', 'assessment_attempts', 'attempt_answers', 'enrollments']

  console.log('\n=== TABLE COLUMNS VIA information_schema ===\n')

  for (const tableName of tables) {
    const { data, error } = await s
      .from('information_schema.columns' as any)
      .select('column_name, data_type, is_nullable, column_default')
      .eq('table_schema', 'public')
      .eq('table_name', tableName)
      .order('ordinal_position' as any)

    if (error) {
      console.log(`❌ ${tableName}: cannot query columns — ${error.message}`)
    } else if (!data || data.length === 0) {
      console.log(`⚠️  ${tableName}: table may NOT EXIST in public schema (no columns found)`)
    } else {
      console.log(`✅ ${tableName} (${data.length} columns):`)
      data.forEach((col: any) => {
        console.log(`   ${col.column_name} (${col.data_type})${col.is_nullable === 'NO' ? ' NOT NULL' : ''}`)
      })
    }
    console.log()
  }

  // Try direct row counts ignoring RLS
  console.log('\n=== ROW COUNTS ===\n')
  for (const tableName of tables) {
    const { count, error } = await s
      .from(tableName as any)
      .select('*', { count: 'exact', head: true })
    if (error) {
      console.log(`❌ ${tableName}: ${error.message}`)
    } else {
      console.log(`📊 ${tableName}: ${count} rows`)
    }
  }

  // Check recent assessment_attempts
  console.log('\n=== RECENT ASSESSMENT ATTEMPTS (last 5) ===\n')
  const { data: recent, error: recErr } = await s
    .from('assessment_attempts' as any)
    .select('id, assessment_id, user_id, score, passed, grade_status, submitted_at, answers')
    .order('submitted_at', { ascending: false, nullsFirst: false })
    .limit(5)

  if (recErr) {
    console.log('❌', recErr.message)
  } else {
    console.log(`Found ${recent.length} attempt(s):`)
    recent.forEach((r: any) => {
      const hasAnswers = r.answers && Object.keys(r.answers || {}).length > 0
      console.log(`  [${r.id.slice(0,8)}] score=${r.score} | passed=${r.passed} | status=${r.grade_status} | submitted=${r.submitted_at?.slice(0,16)} | answers_col=${hasAnswers ? 'HAS DATA' : 'empty/null'}`)
    })
  }

  // Check attempt_answers
  console.log('\n=== RECENT ATTEMPT_ANSWERS (last 5) ===\n')
  const { data: ans, error: ansErr } = await s
    .from('attempt_answers' as any)
    .select('*')
    .limit(5)

  if (ansErr) {
```

```
    console.log('❌', ansErr.message)
  } else {
    console.log(`Found ${ans.length} answer(s):`)
    ans.forEach((a: any) => console.log('  ', JSON.stringify(a)))
  }

  // Verify questions have correct_answer and options
  console.log('\n=== QUESTIONS SAMPLE ===\n')
  const { data: qs, error: qsErr } = await s
    .from('questions' as any)
    .select('id, assessment_id, question_text, correct_answer, options')
    .limit(3)

  if (qsErr) {
    console.log('❌', qsErr.message)
  } else {
    console.log(`Found ${qs.length} question(s):`)
    qs.forEach((q: any) => {
      const optKeys = Object.keys(q.options || {}).filter(k => !['_question_type','_difficulty','_section'].includes(k))
      const correctExists = q.correct_answer && q.options && q.correct_answer in q.options
      console.log(`  [${q.id.slice(0,6)}] correct_answer="${q.correct_answer}" | option_keys=[${optKeys.join(',')}] | correct_
in_options=${correctExists}`)
    })
  }
}

run().catch(e => { console.error('Fatal:', e.message); process.exit(1) })
```

scripts/debug-score.mjs MJS 2.4 KB 65 Lines

```mjs
import { createClient } from '@supabase/supabase-js'

const SUPABASE_URL = process.env.VITE_SUPABASE_URL
const SUPABASE_SERVICE_KEY = process.env.SUPABASE_SERVICE_KEY || process.env.VITE_SUPABASE_ANON_KEY

const supabase = createClient(SUPABASE_URL, SUPABASE_SERVICE_KEY)

async function run() {
  const { data: attempts, error: attemptsErr } = await supabase
    .from('assessment_attempts')
    .select('*, attempt_answers(*)')
    .order('created_at', { ascending: false })
    .limit(1)

  if (attemptsErr) { console.error(attemptsErr); return }
  const attempt = attempts[0]
  if (!attempt) return console.log("No attempts")

  console.log("Attempt score in DB:", attempt.score)

  const questionIds = attempt.attempt_answers.map(a => a.question_id)
  const { data: questions } = await supabase.from('questions').select('*').in('id', questionIds)

  let correctCount = 0;
  for (const q of questions) {
    const ans = attempt.attempt_answers.find(a => a.question_id === q.id)
    if (!ans) continue;

    let studentAns = ans.selected_answer || ''
    if (typeof studentAns !== 'string' && Array.isArray(studentAns)) studentAns = studentAns.join(',')
    else studentAns = String(studentAns)
    studentAns = studentAns.trim()

    const correctKeys = (q.correct_answer || '').split(',').map(k => k.trim())
    const correctTexts = correctKeys.map(k => (q.options || {})[k] || k).filter(Boolean).sort().join('|||')
    const studentTexts = studentAns.split('|||').map(s => s.trim()).filter(Boolean).sort().join('|||')

    console.log(`Q: ${q.id.slice(0,5)}`)
    console.log(`  q.correct_answer:`, q.correct_answer)
    console.log(`  correctTexts: "${correctTexts}"`)
    console.log(`  studentAns: "${studentAns}"`)
    console.log(`  studentTexts: "${studentTexts}"`)

    if (studentTexts && correctTexts && studentTexts === correctTexts) {
      console.log(`  => MATCH!`)
      correctCount++
    } else {
      // fallback
      const studentKey = studentAns.split(',').map(s => s.trim()).sort().join(',')
      const correctKey = correctKeys.sort().join(',')
      if (studentKey !== '' && studentKey === correctKey) {
         console.log(`  => MATCH BY KEY!`)
         correctCount++
      } else {
         console.log(`  => WRONG`)
      }
    }
  }

  console.log("Total correct computed by Result logic:", correctCount)
  console.log("Total questions:", questions.length)
}

run()
```

scripts/fix-attempt-answers.mjs MJS 3.1 KB 93 Lines

```mjs
/**
 * Migration: Convert letter-key answers (A, B, C, D) to full option text values.
 * Run with: node fix-attempt-answers.mjs
 */
import { createClient } from '@supabase/supabase-js'

const SUPABASE_URL = process.env.VITE_SUPABASE_URL
const SUPABASE_SERVICE_KEY = process.env.SUPABASE_SERVICE_KEY || process.env.VITE_SUPABASE_ANON_KEY

if (!SUPABASE_URL || !SUPABASE_SERVICE_KEY) {
  console.error('Missing env vars. Usage:')
  console.error('VITE_SUPABASE_URL=... SUPABASE_SERVICE_KEY=... node fix-attempt-answers.mjs')
  process.exit(1)
}

const supabase = createClient(SUPABASE_URL, SUPABASE_SERVICE_KEY)
const INTERNAL_KEYS = new Set(['_question_type', '_difficulty', '_section'])
const LETTER_PATTERN = /^[A-F]$/

async function main() {
  console.log('Fetching all attempt answers...')
  const { data: attemptAnswers, error: attemptAnswersErr } = await supabase
    .from('attempt_answers')
    .select('id, attempt_id, question_id, selected_answer')

  if (attemptAnswersErr) { console.error('Error fetching attempt_answers:', attemptAnswersErr); process.exit(1) }
  console.log(`Found ${attemptAnswers.length} answers`)

  const allQuestionIds = new Set()
  for (const ans of attemptAnswers) {
    if (ans.question_id) allQuestionIds.add(ans.question_id)
  }

  console.log(`Fetching ${allQuestionIds.size} questions...`)
  const { data: questions, error: qErr } = await supabase
    .from('questions')
    .select('id, options, correct_answer')
    .in('id', [...allQuestionIds])

  if (qErr) { console.error('Error fetching questions:', qErr); process.exit(1) }

  const qMap = {}
  for (const q of questions) qMap[q.id] = q

  let updatedCount = 0

  for (const ans of attemptAnswers) {
    const qId = ans.question_id
    const rawAnswer = ans.selected_answer

    const q = qMap[qId]
    if (!q || !q.options) continue

    const answerStr = String(rawAnswer || '').trim()
    if (!answerStr) continue

    const parts = answerStr.split(',').map(s => s.trim())
    const allLetterKeys = parts.every(p => LETTER_PATTERN.test(p))

    if (allLetterKeys) {
      // Convert letters to text values
      const converted = parts.map(letter => {
        const text = q.options[letter]
        return (text && !INTERNAL_KEYS.has(letter)) ? text : letter
      })

      const newAnswer = converted.join('|||')

      if (newAnswer !== answerStr) {
        const { error: updateErr } = await supabase
          .from('attempt_answers')
          .update({ selected_answer: newAnswer })
          .eq('id', ans.id)

        if (updateErr) {
          console.error(`❌ Failed answer ${ans.id.slice(0,8)}:`, updateErr.message)
```

```
        } else {
          console.log(`✅ Updated answer for Q ${qId.slice(0,8)}: ${answerStr} -> ${newAnswer}`)
          updatedCount++
        }
      }
    }
  }

  // Note: we're not recalculating the overall score in the assessment_attempts table in this script
  // because that would require joining attempt_answers by attempt_id.
  // The immediate UI problem is just fixing the stored answer strings.

  console.log(`\nDone. Updated ${updatedCount} answers in DB.`)
}

main().catch(err => { console.error(err); process.exit(1) })

```

scripts/generate_code_pdf.cjs CJS 18.9 KB 613 Lines

```
const fs = require('fs');
const path = require('path');
const { execFileSync } = require('child_process');

console.log('--- Capacity Connect Code-to-PDF Generator Initialized ---');

const ROOT_DIR = path.resolve(__dirname, '..');
const CHROME_PATH = 'C:\\Program Files\\Google\\Chrome\\Application\\chrome.exe';
const EDGE_PATH = 'C:\\Program Files (x86)\\Microsoft\\Edge\\Application\\msedge.exe';
const BROWSER_PATH = fs.existsSync(CHROME_PATH) ? CHROME_PATH : EDGE_PATH;

const IGNORE_DIRS = new Set([
  'node_modules',
  '.git',
  'dist',
  'scratch',
  '.gemini',
  '.system_generated'
]);

const IGNORE_FILES = new Set([
  'package-lock.json',
  'lint.log',
  'diff_branch.txt',
  'commit_diff.txt',
  'commit_diff_utf8.txt',
  'diff_full.txt',
  'db_dump.sql',
  '.env.local' // protect local environment secrets
]);

const ALLOWED_EXTS = new Set([
  '.ts', '.tsx', '.js', '.jsx', '.cjs', '.mjs',
  '.css', '.html', '.sql', '.json', '.md'
]);

function escapeHtml(str) {
  if (!str) return '';
  return str
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#039;');
}

// Ultra-fast inline syntax highlighting
function highlightCode(rawCode, ext) {
  const lines = rawCode.split('\n');
  const padLength = Math.max(3, lines.length.toString().length);
  const out = [];

  for (let i = 0; i < lines.length; i++) {
    const line = lines[i];
    const num = (i + 1).toString().padStart(padLength, ' ');
    let esc = escapeHtml(line);

    if (ext === '.sql') {
      if (/^\s*--/.test(line)) {
        esc = `<span class="c-cmt">${esc}</span>`;
      } else {
        esc = esc.replace(/\b(SELECT|FROM|WHERE|INSERT|INTO|UPDATE|DELETE|CREATE|TABLE|ALTER|DROP|FUNCTION|RETURNS|TRIGGER|POLICY|ROW|LEVEL|SECURITY|ENABLE|GRANT|TO|AS|BEGIN|END|DECLARE|RETURN|IF|THEN|ELSE|ELSIF|EXECUTE|LANGUAGE|PLPGSQL|SECURITY|DEFINER|SET|DEFAULT|PRIMARY|KEY|FOREIGN|REFERENCES|NOT|NULL|UNIQUE|INDEX|ON|JOIN|LEFT|INNER|OUTER|GROUP|BY|ORDER|LIMIT|OFFSET|UNION|ALL|OR|AND|CASE|WHEN|THEN|END|VIEW|CASCADE)\b/gi, '<span class="c-kw">$1</span>');
      }
    } else if (ext === '.json') {
      esc = esc
        .replace(/(&quot;.*?&quot;)(?=:)/g, '<span class="c-prop">$1</span>')
        .replace(/(:\s*)(&quot;.*?&quot;)/g, '$1<span class="c-str">$2</span>')
        .replace(/\b(true|false|null)\b/g, '<span class="c-num">$1</span>')
        .replace(/\b(-?\d+(\.\d+)?)\b/g, '<span class="c-num">$1</span>');
    } else {
      // JS / TS / CSS
      if (/^\s*(\/\/|\/\*|\*)/.test(line)) {
        esc = `<span class="c-cmt">${esc}</span>`;
```

```
      } else {
        esc = esc.replace(/\b(import|export|from|default|const|let|var|function|return|if|else|for|while|switch|case|break|continue|try|catch|finally|throw|new|class|interface|type|extends|implements|as|async|await|typeof|instanceof|void|yield)\b/g, '<span class="c-kw">$1</span>');
        esc = esc.replace(/\b(true|false|null|undefined)\b/g, '<span class="c-num">$1</span>');
        esc = esc.replace(/\b(string|number|boolean|any|unknown|never|void|Record|Array|Promise|React|FC|JSX|HTMLElement)\b/g, '<span class="c-typ">$1</span>');
      }
    }

    out.push(`<span class="ln">${num} | </span>${esc}`);
  }

  return out.join('\n');
}

function scanFiles() {
  const allFiles = [];

  function walk(currentDir) {
    const entries = fs.readdirSync(currentDir, { withFileTypes: true });
    for (const entry of entries) {
      if (entry.isDirectory()) {
        if (!IGNORE_DIRS.has(entry.name)) {
          walk(path.join(currentDir, entry.name));
        }
      } else {
        const ext = path.extname(entry.name).toLowerCase();
        if (ALLOWED_EXTS.has(ext) && !IGNORE_FILES.has(entry.name)) {
          const fullPath = path.join(currentDir, entry.name);
          const relPath = path.relative(ROOT_DIR, fullPath).replace(/\\/g, '/');
          const stat = fs.statSync(fullPath);
          allFiles.push({
            fullPath,
            relPath,
            name: entry.name,
            ext,
            size: stat.size
          });
        }
      }
    }
  }

  walk(ROOT_DIR);
  return allFiles;
}

function categorizeFile(file) {
  const p = file.relPath;
  if (p === 'package.json' || p === 'vite.config.ts' || p === 'tailwind.config.js' || p.startsWith('tsconfig') || p === 'postcss.config.js' || p === 'components.json' || p === 'vercel.json' || p === 'README.md' || p === 'index.html') {
    return { section: '01. Configuration & Build Setup', order: 1 };
  }
  if (p === 'src/main.tsx' || p === 'src/App.tsx' || p === 'src/index.css' || p === 'src/App.css') {
    return { section: '02. Application Entry & Core Setup', order: 2 };
  }
  if (p.startsWith('src/integrations/') || p.startsWith('src/lib/') || p.startsWith('src/utils/')) {
    return { section: '03. Integrations, Supabase Client & Utilities', order: 3 };
  }
  if (p.startsWith('src/hooks/')) {
    return { section: '04. Custom Hooks & State Management', order: 4 };
  }
  if (p.startsWith('src/pages/')) {
    return { section: '05. Application Pages & Views', order: 5 };
  }
  if (p.startsWith('src/features/trainer/')) {
    return { section: '06. Features — Trainer Management', order: 6 };
  }
  if (p.startsWith('src/features/courses/')) {
    return { section: '07. Features — Courses, Sessions & Learning', order: 7 };
  }
  if (p.startsWith('src/features/admin/')) {
    return { section: '08. Features — Super Admin & Governance', order: 8 };
  }
  if (p.startsWith('src/features/notifications/') || p.startsWith('src/features/certificates/')) {
    return { section: '09. Features — Notifications & Certificates', order: 9 };
  }
  if (p.startsWith('src/components/ui/')) {
```

```
    return { section: '10. UI Primitives & Design System (Radix/Tailwind)', order: 10 };
  }
  if (p.startsWith('src/components/')) {
    return { section: '11. Shared Layout & Dialog Components', order: 11 };
  }
  if (p.startsWith('supabase/functions/')) {
    return { section: '12. Supabase Backend — Edge Functions', order: 12 };
  }
  if (p.startsWith('supabase/migrations/')) {
    return { section: '13. Supabase Backend — Database Migrations', order: 13 };
  }
  if (p.startsWith('scripts/') || !p.includes('/')) {
    return { section: '14. System, Audit & Database Scripts', order: 14 };
  }
  return { section: '15. Miscellaneous Code', order: 15 };
}

console.log('Scanning files...');
const files = scanFiles();

let totalLines = 0;
let totalBytes = 0;

for (const file of files) {
  const cat = categorizeFile(file);
  file.section = cat.section;
  file.order = cat.order;
  const content = fs.readFileSync(file.fullPath, 'utf8');
  file.lines = content.split('\n').length;
  file.content = content;
  totalLines += file.lines;
  totalBytes += file.size;
}

files.sort((a, b) => {
  if (a.order !== b.order) return a.order - b.order;
  return a.relPath.localeCompare(b.relPath);
});

console.log(`Found ${files.length} source files, totaling ${totalLines.toLocaleString()} lines (${(totalBytes / 1024 / 1024).toFixed(2)} MB).`);

function buildHtml(fileSubset, title, subtitle) {
  let html = `<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>${escapeHtml(title)}</title>
<style>
  @page {
    size: A4 portrait;
    margin: 14mm 10mm 14mm 10mm;
  }
  * {
    box-sizing: border-box;
    -webkit-print-color-adjust: exact !important;
    print-color-adjust: exact !important;
  }
  body {
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Arial, sans-serif;
    color: #0f172a;
    background: #ffffff;
    margin: 0;
    padding: 0;
  }

  /* Cover Page */
  .cover-page {
    page-break-after: always;
    min-height: 88vh;
    display: flex;
    flex-direction: column;
    justify-content: space-between;
    padding: 24px;
    border: 2px solid #cbd5e1;
    border-radius: 12px;
    background: #ffffff;
  }
  .cover-header {
```

```
    border-bottom: 3px solid #2563eb;
    padding-bottom: 20px;
  }
  .cover-badge {
    display: inline-block;
    background: #eff6ff;
    color: #1d4ed8;
    font-weight: 700;
    font-size: 11px;
    letter-spacing: 1px;
    text-transform: uppercase;
    padding: 4px 12px;
    border-radius: 20px;
    margin-bottom: 12px;
    border: 1px solid #bfdbfe;
  }
  .cover-title {
    font-size: 34px;
    font-weight: 800;
    color: #0f172a;
    margin: 0 0 8px 0;
  }
  .cover-subtitle {
    font-size: 16px;
    color: #475569;
    margin: 0;
  }

  .meta-grid {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 12px;
    margin: 20px 0;
  }
  .meta-card {
    background: #f8fafc;
    border: 1px solid #e2e8f0;
    border-radius: 8px;
    padding: 12px 16px;
  }
  .meta-label {
    font-size: 10px;
    text-transform: uppercase;
    letter-spacing: 0.5px;
    color: #64748b;
    font-weight: 700;
    margin-bottom: 3px;
  }
  .meta-value {
    font-size: 15px;
    font-weight: 700;
    color: #1e293b;
  }

  .tech-stack {
    display: flex;
    flex-wrap: wrap;
    gap: 6px;
    margin-top: 8px;
  }
  .tech-pill {
    background: #f1f5f9;
    color: #334155;
    font-size: 10px;
    font-weight: 600;
    padding: 3px 8px;
    border-radius: 4px;
    border: 1px solid #cbd5e1;
  }

  .cover-footer {
    border-top: 1px solid #e2e8f0;
    padding-top: 12px;
    font-size: 10px;
    color: #64748b;
    display: flex;
    justify-content: space-between;
  }

```

```
  /* Table of Contents */
  .toc-page {
    page-break-after: always;
    padding-top: 6px;
  }
  .toc-title {
    font-size: 20px;
    font-weight: 800;
    color: #0f172a;
    border-bottom: 2px solid #2563eb;
    padding-bottom: 6px;
    margin-bottom: 14px;
  }
  .toc-section {
    margin-bottom: 12px;
    page-break-inside: avoid;
  }
  .toc-section-header {
    font-size: 11px;
    font-weight: 700;
    color: #1e40af;
    background: #eff6ff;
    padding: 4px 8px;
    border-radius: 4px;
    margin-bottom: 4px;
    border-left: 3px solid #2563eb;
  }
  .toc-list {
    list-style: none;
    padding: 0;
    margin: 0;
  }
  .toc-item {
    display: flex;
    justify-content: space-between;
    font-size: 9.5px;
    padding: 2px 6px;
    border-bottom: 1px dotted #e2e8f0;
    font-family: "JetBrains Mono", "Consolas", monospace;
  }
  .toc-item:nth-child(even) {
    background: #f8fafc;
  }
  .toc-item-meta {
    color: #64748b;
    white-space: nowrap;
    margin-left: 10px;
  }

  /* Code Section */
  .file-container {
    page-break-before: always;
    margin-bottom: 16px;
  }
  .file-header {
    page-break-inside: avoid;
    page-break-after: avoid;
    background: #0f172a;
    color: #f8fafc;
    padding: 6px 10px;
    border-radius: 6px 6px 0 0;
    display: flex;
    justify-content: space-between;
    align-items: center;
    border-bottom: 2px solid #2563eb;
  }
  .file-path {
    font-family: "JetBrains Mono", "Consolas", monospace;
    font-size: 10.5px;
    font-weight: 700;
    color: #ffffff;
  }
  .file-badges {
    display: flex;
    gap: 6px;
  }
  .file-badge {
    background: #1e293b;
    color: #93c5fd;
```

```
    font-size: 8.5px;
    font-weight: 600;
    padding: 2px 6px;
    border-radius: 4px;
    border: 1px solid #334155;
    font-family: sans-serif;
  }
  .file-badge-accent {
    background: #1e3a8a;
    color: #bfdbfe;
    border-color: #3b82f6;
  }

  .code-pre {
    margin: 0;
    padding: 6px 8px;
    background: #fafafa;
    border: 1px solid #e2e8f0;
    border-top: none;
    border-radius: 0 0 6px 6px;
    font-family: "Consolas", "Courier New", monospace;
    font-size: 7.2pt;
    line-height: 1.34;
    white-space: pre-wrap;
    word-break: break-all;
    tab-size: 2;
    color: #1e293b;
  }

  .ln {
    color: #94a3b8;
    user-select: none;
    display: inline-block;
  }

  /* Syntax Highlighting */
  .c-kw { color: #7c3aed; font-weight: 600; }
  .c-str { color: #059669; }
  .c-cmt { color: #64748b; font-style: italic; }
  .c-typ { color: #0284c7; font-weight: 600; }
  .c-num { color: #d97706; font-weight: 600; }
  .c-prop { color: #2563eb; }
</style>
</head>
<body>

<!-- Cover Page -->
<div class="cover-page">
  <div class="cover-header">
    <div class="cover-badge">Source Code Documentation</div>
    <h1 class="cover-title">Capacity Connect</h1>
    <p class="cover-subtitle">${escapeHtml(subtitle)}</p>
  </div>

  <div class="meta-grid">
    <div class="meta-card">
      <div class="meta-label">Total Files Included</div>
      <div class="meta-value">${fileSubset.length.toLocaleString()} Files</div>
    </div>
    <div class="meta-card">
      <div class="meta-label">Total Code Lines</div>
      <div class="meta-value">${fileSubset.reduce((acc, f) => acc + f.lines, 0).toLocaleString()} Lines</div>
    </div>
    <div class="meta-card">
      <div class="meta-label">Export Date &amp;amp; Time</div>
      <div class="meta-value">${new Date().toLocaleString('en-US', { dateStyle: 'long', timeStyle: 'short' })}</div>
    </div>
    <div class="meta-card">
      <div class="meta-label">Technology Stack</div>
      <div class="meta-value">React 19, TypeScript, Supabase</div>
    </div>
  </div>

  <div style="background: #ffffff; border: 1px solid #e2e8f0; border-radius: 8px; padding: 14px 18px;">
    <div class="meta-label">Architecture &amp;amp; Core Dependencies</div>
    <div class="tech-stack">
      <span class="tech-pill">React 19</span>
      <span class="tech-pill">TypeScript</span>
      <span class="tech-pill">Vite</span>
```

```
      <span class="tech-pill">Tailwind CSS</span>
      <span class="tech-pill">Supabase Backend</span>
      <span class="tech-pill">PostgreSQL &amp;amp; RLS Policies</span>
      <span class="tech-pill">Supabase Edge Functions (Deno)</span>
      <span class="tech-pill">Radix UI Primitives</span>
      <span class="tech-pill">TanStack React Query</span>
      <span class="tech-pill">React Hook Form + Zod</span>
      <span class="tech-pill">Three.js / React Three Fiber</span>
      <span class="tech-pill">TensorFlow.js (BlazeFace/COCO-SSD)</span>
    </div>
  </div>

  <div class="cover-footer">
    <span>Capacity Connect Application Repository</span>
    <span>Directory: E:\\Projects\\capacity connect</span>
  </div>
</div>

<!-- Table of Contents -->
<div class="toc-page">
  <div class="toc-title">Table of Contents</div>
`;

  const subSections = {};
  for (const f of fileSubset) {
    if (!subSections[f.section]) subSections[f.section] = [];
    subSections[f.section].push(f);
  }

  for (const [secTitle, secFiles] of Object.entries(subSections)) {
    const secLines = secFiles.reduce((acc, f) => acc + f.lines, 0);
    html += `
  <div class="toc-section">
    <div class="toc-section-header">${escapeHtml(secTitle)} (${secFiles.length} files, ${secLines.toLocaleString()} lines)</di
v>
    <ul class="toc-list">
    `;
    for (const f of secFiles) {
      html += `
      <li class="toc-item">
        <span>${escapeHtml(f.relPath)}</span>
        <span class="toc-item-meta">${f.lines.toLocaleString()} lines | ${(f.size / 1024).toFixed(1)} KB</span>
      </li>
      `;
    }
    html += `
    </ul>
  </div>
    `;
  }

  html += `
</div>

<!-- Source Code Content -->
`;

  for (const f of fileSubset) {
    html += `
<div class="file-container">
  <div class="file-header">
    <div class="file-path">${escapeHtml(f.relPath)}</div>
    <div class="file-badges">
      <span class="file-badge">${f.ext.toUpperCase().replace('.', '')}</span>
      <span class="file-badge">${(f.size / 1024).toFixed(1)} KB</span>
      <span class="file-badge file-badge-accent">${f.lines.toLocaleString()} Lines</span>
    </div>
  </div>
  <pre class="code-pre"><code>${highlightCode(f.content, f.ext)}</code></pre>
</div>
`;
  }

  html += `
</body>
</html>
`;
  return html;
}
```

```
542 |
543 | function renderPdf(htmlPath, pdfPath) {
544 |   const headerTemplate = `
545 |     <div style="font-size: 8px; width: 100%; display: flex; justify-content: space-between; padding: 0 10mm; color: #64748b; f
ont-family: sans-serif; border-bottom: 0.5px solid #e2e8f0; padding-bottom: 2px;">
546 |       <span style="font-weight: 700; color: #1e293b;">CAPACITY CONNECT</span>
547 |       <span>Source Code Documentation</span>
548 |     </div>
549 |   `;
550 |
551 |   const footerTemplate = `
552 |     <div style="font-size: 8px; width: 100%; display: flex; justify-content: space-between; padding: 0 10mm; color: #64748b; f
ont-family: sans-serif; border-top: 0.5px solid #e2e8f0; padding-top: 2px;">
553 |       <span>Capacity Connect Repository</span>
554 |       <span>Page <span class="pageNumber"></span> of <span class="totalPages"></span></span>
555 |     </div>
556 |   `;
557 |
558 |   console.log(`Rendering PDF via browser (${path.basename(BROWSER_PATH)})...`);
559 |   const t0 = Date.now();
560 |   execFileSync(BROWSER_PATH, [
561 |     '--headless=new',
562 |     '--disable-gpu',
563 |     '--display-header-footer',
564 |     `--header-template=${headerTemplate}`,
565 |     `--footer-template=${footerTemplate}`,
566 |     `--print-to-pdf=${pdfPath}`,
567 |     htmlPath
568 |   ], { maxBuffer: 1024 * 1024 * 100, timeout: 600000 });
569 |
570 |   const duration = ((Date.now() - t0) / 1000).toFixed(1);
571 |   const sizeMb = (fs.statSync(pdfPath).size / 1024 / 1024).toFixed(2);
572 |   console.log(`Generated: ${path.basename(pdfPath)} (${sizeMb} MB in ${duration}s)`);
573 | }
574 |
575 | function main() {
576 |   const outputDir = ROOT_DIR;
577 |
578 |   // 1. Frontend & Core Application
579 |   const frontendFiles = files.filter(f => !f.relPath.startsWith('supabase/'));
580 |   // 2. Supabase Backend & Database
581 |   const backendFiles = files.filter(f => f.relPath.startsWith('supabase/') || f.relPath.startsWith('scripts/'));
582 |
583 |   console.log('\n--- 1. Generating Volume 1: Frontend Application PDF ---');
584 |   const vol1HtmlPath = path.join(outputDir, 'Capacity_Connect_Frontend_Application.html');
585 |   const vol1PdfPath = path.join(outputDir, 'Capacity_Connect_Frontend_Application.pdf');
586 |   fs.writeFileSync(vol1HtmlPath, buildHtml(frontendFiles, 'Capacity Connect — Frontend Application', 'Frontend Application Arc
hitecture, Pages, Features & Components'));
587 |   renderPdf(vol1HtmlPath, vol1PdfPath);
588 |   fs.unlinkSync(vol1HtmlPath);
589 |
590 |   console.log('\n--- 2. Generating Volume 2: Supabase Backend & Database PDF ---');
591 |   const vol2HtmlPath = path.join(outputDir, 'Capacity_Connect_Backend_and_Database.html');
592 |   const vol2PdfPath = path.join(outputDir, 'Capacity_Connect_Backend_and_Database.pdf');
593 |   fs.writeFileSync(vol2HtmlPath, buildHtml(backendFiles, 'Capacity Connect — Backend & Database', 'Supabase Database Schema, M
igrations, Edge Functions & Maintenance Scripts'));
594 |   renderPdf(vol2HtmlPath, vol2PdfPath);
595 |   fs.unlinkSync(vol2HtmlPath);
596 |
597 |   console.log('\n--- 3. Generating Complete Codebase Master PDF ---');
598 |   const masterHtmlPath = path.join(outputDir, 'Capacity_Connect_Source_Code_Complete.html');
599 |   const masterPdfPath = path.join(outputDir, 'Capacity_Connect_Source_Code_Complete.pdf');
600 |   fs.writeFileSync(masterHtmlPath, buildHtml(files, 'Capacity Connect — Complete Codebase', 'Complete Source Code Documentatio
n (All 238 Files, 53,700+ Lines)'));
601 |   renderPdf(masterHtmlPath, masterPdfPath);
602 |   fs.unlinkSync(masterHtmlPath);
603 |
604 |   console.log('\n======================================================');
605 |   console.log('SUCCESS! All PDF documents generated:');
606 |   console.log('1. Capacity_Connect_Frontend_Application.pdf');
607 |   console.log('2. Capacity_Connect_Backend_and_Database.pdf');
608 |   console.log('3. Capacity_Connect_Source_Code_Complete.pdf');
609 |   console.log('======================================================\n');
610 | }
611 |
612 | main();
613 |
```

supabase/.temp/linked-project.json JSON 0.1 KB 1 Lines


```
1 | {"ref":"gezrhprdrlsswfnuffvf","name":"capacity-connect","organization_id":"xfmexvtpgnymbtqtnxty","organization_slug":"xfmexvtpgnymbtqtnxty"}
```